Accelerating the Inference Era with AI-Driven, Globally Optimized HW/SW Co-Design

Miria Feng
Electrical Engineering
Stanford University
miria0@stanford.edu

Fangzhao Zhang
Electrical Engineering
Stanford University
zfzhao@stanford.edu

Adrian Lafuente
Computer Science
Stanford University
agamarra@stanford.edu

Mert Pilanci
Electrical Engineering
Stanford University
pilanci@stanford.edu

Azalia Mirhoseini
Computer Science
Stanford University
azalia@stanford.edu

Abstract

Inference demand is projected to grow $10{,}000\times$ in five years, motivating custom accelerators. Coordinating hardware, schedules and compiler transformations creates a combinatorial design space exceeding $\mathcal{O}(10^{2300})$. Sequential pipelines discard alternatives before their global memory effects are known. We introduce CHEETAH, an LLM-guided framework built around one master LP relaxation. Time-indexed placement and rematerialization expose reuse across operations and decode steps; McCormick envelopes couple tiling with fusion; shared hardware variables allocate compute, capacity and bandwidth under area and power budgets. Disaggregated epigraphs and implementation menus encode competing service costs. An LLM architect proposes changes using resource prices and trial diagnostics, while operator-based optimization, integral recovery and replay produce designs separately from relaxation bounds. The original prefill-oriented campaign averages $22.07\times$ speedup and reaches $40.1\times$ over modeled TPU-v3 within six hours of search. A full-inference DeepSeek-V3 design passes a declared BF16 physical scenario and predicts $20.83\times$ overall against a retained host-backed reference; reproducing FAST's modeled TPU-v3 baseline remains open. Its fixed-chip software/temporal LP has a checked 0.0847% objective interval. Matched controls quantify batching, storage fusion, panel reuse, KV residency and temporal placement.

1 Introduction

AI inference demand is expected to grow by $10{,}000\times$ over the next five years [Computing Research Association, 2026]. General-purpose accelerators leave significant efficiency gains unrealized, which motivates accelerators specialized to their target workloads. Specialization, however, requires coordinating datapaths, memory capacity, mappings, and compiler transformations, and the resulting design space can exceed $\mathcal{O}(10^{2300})$ [Zhang et al., 2022]. Engineers typically tackle this problem in stages: propose hardware, map each operation, then optimize data reuse and fusion. However this sequential method misses crucial global gains in co-design. For example, a locally faster tile can consume the memory needed to retain a neighboring tensor, and a slightly slower tile can therefore yield a faster complete schedule.

In this paper, we introduce CHEETAH: **C**o-designed **H**ardware **E**xploration via **E**fficient **T**hroughput and **H**ybrid-search, a novel mathematically principled hardware-software co-design

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

framework with LLM guided search efficiency. The inner loop consists of one master convex program which exposes complex interactions jointly. Its implementation choices determine work, traffic, memory, time-indexed states track residency and reuse. Its shared hardware variables allocate compute, vector, memory, and bandwidth resources under area and power budgets. CHEETAH uses McCormick envelopes and disaggregated epigraph techniques to model HW/SW relationships in a rigorous and realistic way. The large scale convex program also yields bounds for its declared domain, while integral recovery and replay verify each chip designed and complementing software schedule. These are distinct outputs, since a single low fractional objective alone is not a realistic executable physical design.

The CHEETAH formulation operates on a graph of independent logical invocations and typed storage objects. Convolutions, dense and expert GEMMs, attention, and vector operations contribute implementation-specific coefficients and dependencies. Optional batches and fusion groups cover original invocations, and KV prefixes, expert weights, and long skip connections enter the same capacity and readiness accounting. This makes the program reusable across architectures without assuming that identical shapes must share a decision.

We separate inference acceleration gains betwee prefill and decode. Prefill amortizes weight traffic across many tokens, whil autoregressive decode repeatedly accesses weights and an expanding KV history. Batching helps, but sparse routing also increases the number of experts touched. Under independent uniform top-k routing, the expected union is $n[1 - (1 - k/n)^B]$: with $n = 256$, $k = 8$, it grows from 8 experts at batch one to about 163 at batch 32. Thus batching does not guarantee a compute-bound decode phase. CHEETAH therefore jointly chooses resource provisioning, compatible batches, and temporal residency. Our full-inference experiment uses actual routes, retains the complete KV history, and charges every restore. Its gains do not assume that all weights or KV fit on chip.

Since the complete CHEETAH program is too large to materialize, in order to this large scale formulation we introduce a custom adapted OCSolver, which evaluates linear operators and adjoints on compact working supports, using GPU first-order updates, reversible scaling, and structured temporal incidence. An LLM architect outer loop then reads resource prices and trial diagnostics to propose target specific implementations or hardware search neighborhoods. This reduces the reliance on brute force large space search effectively. Crucially, the hardware-software interactions and decisions are captured in the CHEETAH inner loop formulation.

Contributions. CHEETAH combines a unified HW/SW design relaxation, operator-based optimization and certified bounds with an LLM architect that proposes implementations and hardware search neighborhoods.

1. **CHEETAH formulation.** We introduce time-indexed LP variables $p^k_{i,t}$ and rematerialization variables $r_{i,t}$ to enable cheap recompute operations instead of paying DRAM reload costs, supporting skip connections and multi-fanout nodes. We unify tiling and fusion through **McCormick envelopes** and resource epigraphs. Binary tile–residency products are exact at integral choices; continuous hardware/service products retain their explicit relaxation. These couplings expose tiling–placement trade-offs that sequential pipelines can discard.
2. **Operator-based GPU optimization.** OCS applies the master constraints and their adjoint without materializing the matrix. GPU working solves, structured pricing and integral recovery address complementary parts of the large native problem; Section 5 distinguishes accepted working supports from full-master closure.
3. **LLM Architect Outer Loop.** The architect interprets resource prices and a causal ledger of earlier trials to propose hardware and implementation changes. Deterministic checks admit proposals; the LLM itself supplies no certificate. This couples architecture exploration with the optimizer's diagnostics rather than relying only on black-box scores.
4. **Bounds and executable validation.** The frozen 21-branch native DeepSeek master has a certified 0.0850% LP objective gap (rounded). Analytical replay and selected control/datapath checks support a separate recovered design; global discrete optimality remains open. The original experiments retain their certificates and physical-validation evidence.
5. **Inference speedup.** The accepted prefill-oriented campaign reports an average $22.07\times$ predicted speedup and up to $40.1\times$ under its original contracts (Figure 2). The new checkpoint-bound campaign uses matched BF16/FP32 arithmetic for custom and reference hardware,

with full-inference phase and same-chip feature controls. Its archived full-input DeepSeek comparison gives $31.71\times$ against an earlier host-backed reference; this remains conditional on the unreconciled baseline and BF16 physical-cost assumptions (Appendix P.1). Section 6 reports the uniform vision and Llama extensions separately from the historical results.

The following paper is organized as follows. Section 2 discusses related work, Section 3 summarizes preliminary background, Section 4 presents the CHEETAH master and its features, and Section 5 describes OCS, full-domain certification and hardware search. Section 6 reports experimental results and ablations, and the appendix provides containment theorems, physical-validation details, economics, co-design blueprints, and reproduction evidence.

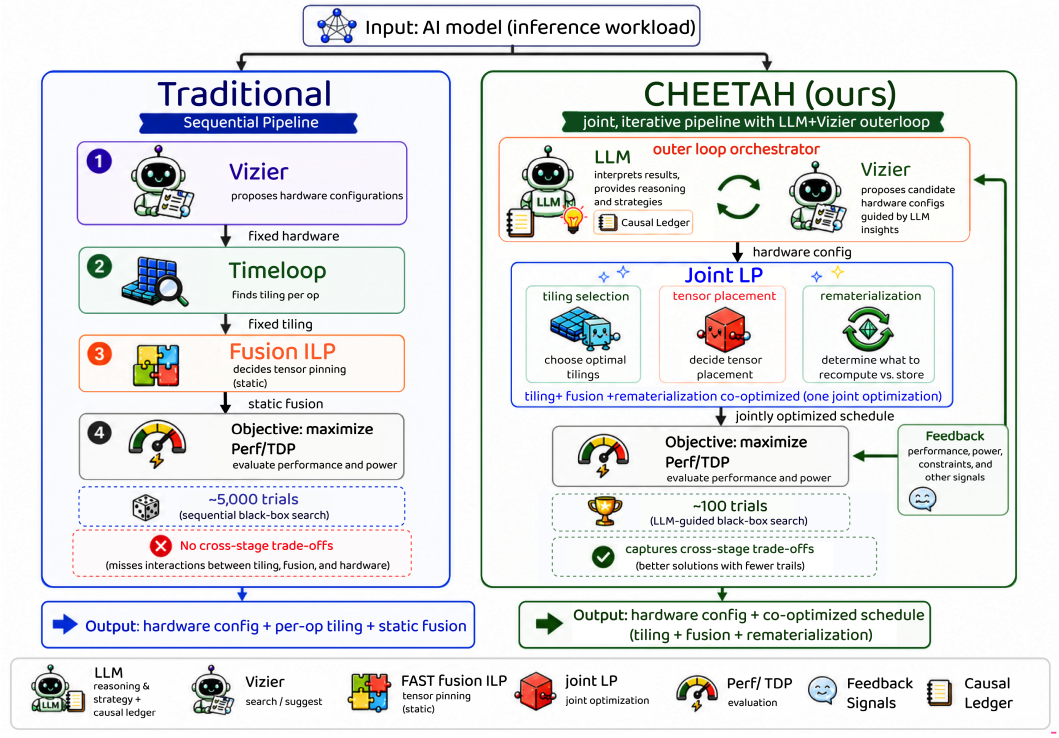


Figure 1: Heuristics based Sequential design vs Our Unified Optimization Paradigm. (Left) Baseline methods decouple hardware search, operation-level tiling, and static fusion, precluding cross-stage trade-offs. (Right) CHEETAH presents a unified LLM-driven mathematical framework that jointly optimizes tiling, placement, and more while learning from a causal ledger.

2 Related Work

Accelerator mappers such as Timeloop, CoSA, and ZigZag explore tiling and mapping; FAST connects hardware search, mapping, and fusion in a staged flow [Parashar et al., 2019, Huang et al., 2021, Mei et al., 2021, Zhang et al., 2022]. DOSA jointly differentiates hardware and mapping choices, while Gemini explores chiplet architectures and layer pipelines [Hong et al., 2023, Cai et al., 2024]. TileFlow and FADiff address fusion-aware dataflows and joint mapping–fusion optimization [Zheng et al., 2023, Jia et al., 2025]. CHEETAH couples these choices to temporal storage and shared hardware in an LP relaxation; this is a distinction in formulation, not a claim that prior work lacks joint optimization altogether.

Capability comparison table. Table 1 distinguishes searched decisions from restricted or user-specified support. DeFiNES chooses caching versus overlap recomputation, Fast-and-Fusiest searches dependency-respecting orders within its fused mapspace. Gemini's fusion and order support concerns layer pipelines. Pluto optimizes affine schedules, Tiramisu also exposes explicit storage, copies, and redundant computation through its scheduling language. These differ from unrestricted cost-driven

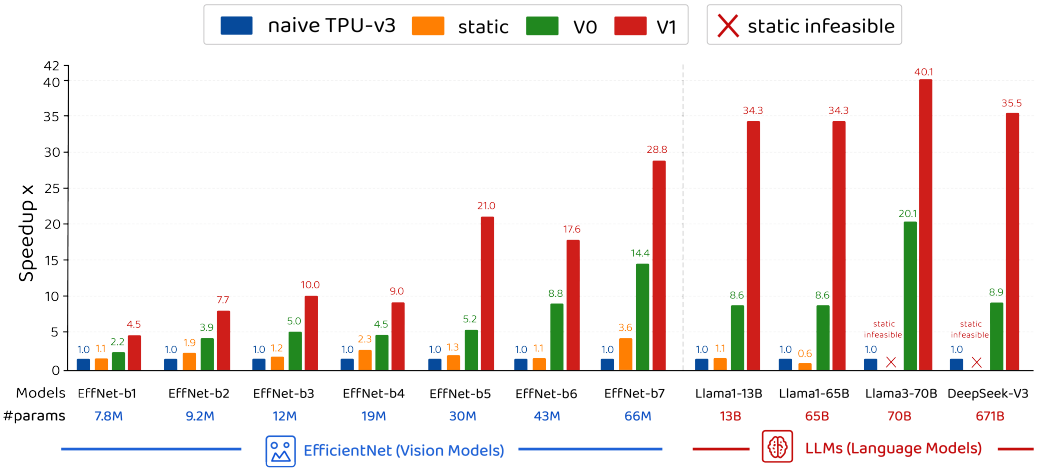


Figure 2: Retained original prefill-oriented results over the modeled TPU-v3 baseline. Legacy labels in the artwork identify restricted controls: static fusion, fixed-tile temporal placement (V0), and joint tiling/fusion (V1). These results retain their original numerical and workload contracts; Section 6 reports the new uniform BF16 comparisons and separates prefill from decode.

temporal placement. CoSA's guarantee concerns a per-layer proxy objective, and Fast-and-Fusiest's concerns its fixed-hardware mapspace, but neither is a bound over a joint hardware–schedule domain. The last column denotes one joint mathematical program, rather than staged co-search. DOSA's qualifying program is nonconvex.

First-order solver methods [Applegate et al., 2022] and McCormick envelopes [McCormick, 1976] supply the optimization building blocks. LLM proposal systems such as FunSearch and AlphaEvolve [Romera-Paredes et al., 2024, Novikov et al., 2025] inform the outer-loop role: propose changes, then evaluate them independently. Further comparisons appear in Appendix D.

3 Preliminaries

3.1 Sequential Design and Information Loss

Accelerator design commonly proceeds from a resource budget to a hardware template, then to operation mappings and a memory schedule. Engineers iterate these choices using heuristics, analytical models, simulation, and compiler feedback. Automated flows replace individual choices with search tools, but an early commitment still limits what downstream stages can recover. A mapper that exports only its fastest isolated tile discards alternatives that might reduce workspace, enable fusion, or retain data for a later consumer. Joint design keeps those alternatives available until their complete resource costs are known in a global comprehensive end-to-end design.

3.2 Linear Relaxation and Recovery

An LP minimizes a linear objective subject to linear rows and coordinate bounds. Discrete implementation selections are relaxed to $[0, 1]$; bounded products are represented by convex envelopes. The resulting optimum is a lower bound only for the domain actually covered by the formulation and its certificate. Fractional selections need not describe a chip. Recovery chooses integral hardware and implementations, then checks dependencies, storage, transfers, and predicted latency. A feasible recovered design is useful even when the remaining optimality interval is wide.

3.3 Capability Comparison

Table 1 compares decision spaces, rather than claiming that every listed capability has a measured gain. CHEETAH's row describes the master formulation; Section 6.5 identifies the subset exercised by the current full-inference execution. Related Work explains restricted cells.

Table 1: Decision spaces in the cited systems. ✓: formulated; ○: restricted or user-specified; ×: not in the cited method. Guarantees retain their own objective and domain. CHEETAH's row demonstrates our unified formulation scope.

	Schedule decisions					Search		Guarantee	
System	Per-op tiling	Fusion	Temporal placement	Remat.	Op. order	Mapping	HW sizing	Opt. guarantee	Joint HW/SW program
Per-op mappers									
Timeloop [Parashar et al., 2019]	✓	×	×	×	×	✓	×	×	×
CoSA [Huang et al., 2021]	✓	×	×	×	×	✓	×	○	×
Fusion-aware mappers									
DeFiNES [Mei et al., 2023]	✓	✓	○	○	○	✓	×	×	×
Fast-and-Fusiest [Andrulis et al., 2026]	✓	✓	○	×	✓	✓	×	○	×
Hardware design-space exploration									
ZigZag [Mei et al., 2021]	✓	×	×	×	×	✓	✓	×	×
DOSA [Hong et al., 2023]	✓	×	×	×	×	✓	✓	×	✓
Gemini [Cai et al., 2024]	✓	○	○	×	○	✓	✓	×	×
CODEBench [Tuli et al., 2023]	○	×	×	×	×	×	✓	×	×
FAST [Zhang et al., 2022]	✓	✓	○	×	×	✓	✓	×	×
Loop compilers									
Pluto [Bondhugula et al., 2008]	○	✓	×	×	✓	○	×	×	×
Tiramisu [Baghdadi et al., 2019]	○	○	○	○	○	○	×	×	×
CHEETAH (our framework)	✓	✓	✓	✓	✓	✓	✓	○	✓

4 CHEETAH Formulation for HW/SW Co-Design

CHEETAH minimizes predicted workload inference latency under a declared hardware budget. Its inputs are independent logical invocations, a finite implementation library, and a numerical contract. Each whole-program branch specifies compatible hardware and execution semantics; hardware is shared across all invocations within that branch. The same master template admits the retained joint tiling–placement model and causal execution branches. The strengthened physical branches add direct installed-resource ceilings. The earlier DeepSeek certificate applies to its frozen family of 21 native branches, not automatically to these new rows.

4.1 Pruning, Variables, and the Encoded Features

Pareto pruning pre-pass. For each operation, we discard infeasible tiles and configurations dominated in every relevant cost, traffic, workspace, and interface dimension. Dominance is tested within the same implementation contract and hardware class: a lower isolated latency alone is insufficient. Timeloop or a native analytical recipe supplies the retained coefficients once; the solver then selects among them without invoking the mapper at each iteration. Reuse across hardware settings requires that the same dominance relation remains valid.

Let $i \in \mathcal{N}$ index logical nodes (invocations and required conversions), $j \in \mathcal{J}$ storage objects, $e \in \mathcal{E}_b = \{0, \dots, E_b - 1\}$ events in branch b, and τ memory tiers. An implementation k covers members S_k and specifies inputs, outputs, workspace $W_{k\tau}$, and resource demands. States also have the exit index E_b. $\mathcal{K}_b$ is the complete finite implementation set of branch b, generated on demand without tying same-shaped invocations. Table 2 defines its coordinates.

Partition the declared family as $\mathcal{B} = \mathcal{B}_C \,\dot\cup\, \mathcal{B}_P$: causal branches and, when included, retained joint tiling–placement branches. The objective charges causal event durations and retained operation latencies plus rematerialization. κ_b converts ns to ms; α_b converts the retained block's cost units to ms. Each x_b contains the complete branch coordinates, already scaled by λ_b; the objective does not multiply them by λ_b again. Right-hand sides and finite bounds scale likewise. In the causal rows below we suppress branch indices and write $E = E_b$. $\mathcal{R}_k$ lists required object/tier pairs, $\mathcal{D}_k$ completed controls, and D^r_e is linear resource demand. $\mathsf{Cap}_{b,\tau e}$ includes live objects, incoming copies, active outputs, workspace and dedicated reservations. B_b, J_b contain the remaining state, backing and implementation rows, expanded in Appendix O. For $b \in \mathcal{B}_{\text{phys}} \subseteq \mathcal{B}_C$, the installed chip $\bar{h}_b$ fixes resource rates $\bar{R}_{br}$, and $\mathsf{D}^{r,\text{phys}}_{be}$ is the total issued work or traffic sharing resource r at event e. $\mathsf{A}^\lambda_b = a_{b0}\lambda_b + a_b^T h_b$ and $\mathsf{P}^\lambda_b = p_{b0}\lambda_b + p_b^T h_b$ denote area and peak power.

Table 2: Master variables. Unscaled selection/state masses lie in $[0, 1]$; branch coordinates are scaled by λ_b.

Variable	Indices	Meaning
h_b	shared chip	Compute/vector resources, capacity and service rates; fixed to $\lambda_b \bar{h}_b$ in physical branches
λ_b, x_b	branch b	Whole-program weight and scaled branch coordinates
y_k	implementation k	Tile, mapping, batch or fusion selection
$s_{ke}, w_{ke}, f_{ke}, a_{ke}$	implementation, event	Start, work, finish and active mass
D_{ie}, β_{je}	invocation/object, event	Logical completion and generated data
$p_{j\tau e}, v_{j\tau e}$	object, tier, event	Resident state and eviction
$q_{j,\tau\to\tau',e}$	object, link, event	Paid transfer and destination reservation
$\eta_{je}, \rho_{je}, \delta_{je}$	parameter, event	Staging occupancy, receive and drain
A_e, χ_e	event	Total active mass and array-workspace product
Q^r_e, u_r, Z^r_e, d_e	resource, event	Demand, inverse rate, service and nonnegative event duration
$T_i, p_{et}, p^W_{it}, r_{it}, w_{ick}$	retained schedule block	Latency, residency, recomputation and tile–residency products

$$
\begin{aligned}
\min_{\lambda,\{x_b\}} \quad & \sum_{b\in\mathcal{B}_C} \kappa_b \sum_{e\in\mathcal{E}_b} d_{be} + \sum_{b\in\mathcal{B}_P} \alpha_b \left(\sum_i T_{bi} + \sum_{i,t} c^{\mathrm{remat}}_{bi} r_{bit} \right) \\
\text{s.t.} \quad & \lambda_b \ge 0, \quad \sum_b \lambda_b = 1, \\
& \sum_{k\in\mathcal{K}_b : i\in S_k} y_k = \lambda_b, \\
& \sum_e s_{ke} = \sum_e w_{ke} = \sum_e f_{ke} = y_k, \\
& a_{ke} = a_{k,e-1} + s_{ke} - f_{k,e-1}, \quad w_{ke} = f_{ke}, \quad 0 \le w_{ke}, f_{ke} \le a_{ke} \le y_k, \\
& a_{ke} \le p_{j\tau e} - v_{j\tau e} \ ((j,\tau)\in\mathcal{R}_k), \qquad a_{ke} \le D_{ie} \ (i \in \mathcal{D}_k), \\
& D_{i,e+1} = D_{ie} + \sum_{k : i\in S_k} f_{ke}, \quad D_{i0} = 0, \ D_{iE} = \lambda_b, \\
& 0 \le v_{j\tau e} \le p_{j\tau e} \le \beta_{je}, \qquad 0 \le q_{j,\tau\to\tau',e} \le p_{j\tau e} - v_{j\tau e}, \\
& A_e = \sum_k a_{ke} \le \lambda_b, \qquad (A_e, h_{b,\mathrm{arrays}}, \chi_e) \in \mathcal{W}^{\lambda_b}_b, \\
& \mathsf{Cap}_{b,\tau e}(p, v, q, a, \chi, \eta, \rho, \delta) \le C^{\lambda}_{\tau}(h_b), \\
& Q^r_e = \mathsf{D}^r_e(q, w, \rho, \delta), \quad (h_{br}, u_r) \in \mathcal{R}^{\lambda_b}_b, \quad (Q^r_e, u_r, Z^r_e) \in \mathcal{M}^{\lambda_b}_{br}, \\
& 0 \le Z^r_e \le d_e, \qquad \mathsf{S}^v_{be}(q, w, \rho, \delta) \le d_e \quad \text{(fixed-rate services } v\text{)}, \\
& h_b = \lambda_b \bar{h}_b, \quad \mathsf{D}^{r,\mathrm{phys}}_{be} \le \bar{R}_{br} d_e \quad (b \in \mathcal{B}_{\mathrm{phys}}, \ r \in \mathcal{R}_{\mathrm{phys}}), \\
& \mathsf{A}^{\lambda}_b \le 600\lambda_b, \quad \mathsf{P}^{\lambda}_b \le 300\lambda_b, \quad \mathsf{P}^{\lambda}_b \le 2\mathsf{A}^{\lambda}_b \quad (b \in \mathcal{B}_{\mathrm{phys}}), \\
& F_b h_b \le g_b \lambda_b, \quad B_b x_b = b_b \lambda_b, \quad J_b x_b \le j_b \lambda_b, \\
& A^P_b x_b = a^P_b \lambda_b, \quad G^P_b x_b \le g^P_b \lambda_b \quad \text{(retained schedule branches)}, \\
& \ell_{b\nu} \lambda_b \le x_{b\nu} \le u_{b\nu} \lambda_b \quad \text{(each finite original bound)}.
\end{aligned}
\tag{Master}
$$

The displayed cover, lifecycle, service and B_b, J_b rows apply to native causal branches; A^P_b, G^P_b apply only to retained schedule branches, when included. $a_{k,-1} = f_{k,-1} = 0$; $\mathcal{W}, \mathcal{R}, \mathcal{M}$ are linear workspace-product, reciprocal and service-product envelopes. Causal copy, conversion, KV restoration and permitted repeat costs enter the shared service demands and hence d_e; adding their service times again would double-count overlap already represented by the event epigraph. The objective covers the declared task boundary, including preparation when that boundary requires it. It minimizes latency, not area, energy, Perf/TDP or solver runtime. Setting $\lambda_b = 1$ and all other branches to zero preserves that branch's hardware and objective. Fractional branch weights describe an LP relaxation; recovery selects one program. Bounds must be recomputed for the chosen family, including every branch.

4.2 Features and Their Mathematical Encoding

Joint tiling and fusion. For an isolated operation, the cover row reduces to $\sum_c y_{i,c} = 1$. A tile's traffic savings depend on residency. In the retained schedule block, w_{ick} represents $y_{ic}p_{\mathrm{assoc}(i,k),o(i)}$ through

$$0 \le w_{ick}, \quad w_{ick} \le y_{ic}, \quad w_{ick} \le p, \quad w_{ick} \ge y_{ic} + p - 1. \tag{2}$$

These McCormick inequalities give the convex hull of one bounded bilinear graph and fix the product when either binary-domain factor is integral [McCormick, 1976]; they do not make every fractional solution executable. Tile-dependent footprints use the same construction. The displayed unit-bound rows are for a branch at weight one; homogenization replaces 1 by λ_b. Native fused atoms carry their complete input, output, workspace and service recipe, so fusion cannot silently remove work.

Shared hardware and resource epigraphs. Compute, vector lanes, SRAM, and delivered memory service are shared decisions under linear area/power proxies $F_b h_b \le g_b$. Finite implementation or technology menus use disaggregated copies with linking equalities and scaled bounds. Resource epigraphs charge the maximum of concurrent service demands, while demands on the same port add. Native rows aggregate work and transfer demands per event before applying the service envelopes. Arbitrary executable overlap is not inferred from fractional sharing; native replay uses its declared inter-kernel ordering and internal panel pipelines.

Hard rooflines and physical budgets. The strengthened fixed-chip branches enforce resource ceilings *inside the LP*: $\sum_k F^{\mathrm{issued}}_k w_{ke} \le \bar{C}_b d_e$ for matrix work, and $\mathsf{D}^{r,\mathrm{phys}}_{be} \le \bar{R}_{br} d_e$ for shared HBM, host, vector, DMA-staging and SRAM-bank-port service. Padded issued work bounds useful work; simultaneous kernels and copies share the same supply. Summing over events therefore gives MFU and accounted MBU at most one. Affine installed-component costs enforce area $\le 600\,\mathrm{mm}^2$, peak power $\le 300\,\mathrm{W}$ and chip-average density $\le 2\,\mathrm{W/mm}^2$, without discounting peak power by MFU. The same identified template supplies combined host/HBM pin, controller, whole-bank capacity/count/aspect, logic-depth and NoC-size constraints in $F_b h_b \le g_b \lambda_b$. Fixed rates keep these ceilings linear; a variable-rate McCormick envelope alone cannot enforce the exact physical product. These are hard limits of the declared analytical model, with the same predicates used in recovery. Aggregate bank and NoC proxies do not prove conflict-free arbitrary placement. Appendix O.5 gives the rows and distinguishes their current implementation from the frozen certificate.

Reciprocal and product envelopes. For a rate $r \in [\ell, h]$, $\mathcal{R}$ bounds $u \approx 1/r$ by tangents $u \ge 2/a - r/a^2$ and the global secant $u \le (\ell + h - r)/(\ell h)$. $\mathcal{M}$ bounds $Z \approx Qu$ by the four McCormick inequalities on $[0, \bar{Q}] \times [1/h, 1/\ell]$. All coefficients are constants, so the program is linear. Outward coefficient rounding preserves graph containment. These are relaxations of physical service; replay reinstates the exact rates and products. Selecting a discrete bandwidth fixes that rate, but continuous-rate envelopes remain relaxations even when implementation selectors are integral.

Time-indexed placement and KV across steps. Residency $p_{j\tau e}$ changes with events rather than being fixed for the whole run. KV production creates immutable append objects; each decode attention operation depends on its required prefix. The state rows preserve earlier entries while the prefix grows. Capacity accounts for resident data, incoming copies, and active workspace simultaneously. Long skip connections and multi-fanout tensors use the same object identities and consumer readiness.

Spill, restoration, and rematerialization. Copies retain a valid source, reserve their destination, consume service, and become readable only after completion. Boundary rows require the declared terminal backing. A permitted repeat executes the original producer again and pays its work, input and workspace demands; it restores storage without advancing primary logical completion twice. Keeping full KV history in a slower tier is therefore different from deleting history. The native results credit paid KV restoration and retention; no native recomputation gain or lossy KV budget is claimed.

Batching and causal scheduling. A batch column covers several compatible invocations and loads shared parameters once where its recipe permits. Cover constraints retain every member; start/finish and readiness rows prevent using future tokens or expert selections before their router dependencies complete. Event placement can vary within the finite library. The current native execution uses a recovered serial schedule; a larger scheduling domain does not imply that its optimum has been found.

Implementation menus and recovery. Parameter staging has explicit receive, occupancy and drain states. Full-K panels and FP8-to-BF16 receiver implementations pay buffer, port and conversion costs. Selected-reader backing keeps the raw or converted representation needed by each consumer; it cannot discard both. These rows connect hardware provisioning to software reuse. Precision, sparsity, array aspect, memory technology, and validated overlap policies enter as implementation choices with their own numerical and resource contracts. Accuracy-changing cache eviction or approximate reuse needs separate quality evidence; it is pinned off in the full-input native result. Larger cross-GEMM fusion and unrestricted overlap have no credited gain there. Recovery pins hardware and mappings, schedules transfers, and replays the original obligations. The resulting chip and schedule have a predicted latency under the analytical model. Section 5 distinguishes hardware branch-and-bound, continuous LP certification, and integral recovery.

4.3 LLM Architect Diagnostic Outer Search

The LLM architect reads a ledger of resource prices, slacks, invalid trials, and replay outcomes, then proposes candidate implementations or search neighborhoods. Deterministic validation checks each proposal before evaluation. The original campaign uses Vizier [Golovin et al., 2017] to search hardware neighborhoods; the native master also represents hardware choices directly. Dual prices guide search, but an LLM proposal is not a complete pricing oracle: a full-domain optimality claim requires checking all omitted alternatives. The original controller and prompts are documented in Appendix J.

5 Operator-Based Optimization and Design Recovery

We combine OCS working solves, structured primal improvements and complete-domain bounds for the continuous LP, then search and replay integral designs. These stages share coefficients but have distinct stopping criteria.

Operators replace matrices. For a working LP $\min_x f(x) + g(Kx)$, $f(x) = c^T x + I_{[\ell,u]}(x)$ and g encodes row equalities and inequalities. An operator computes Kx without storing its matrix; its adjoint $K^T y$ accumulates row prices back to coordinates, including shared hardware and capacity. Generated incidence and coefficient reuse preserve independent invocation decisions.

GPU first-order working solves. Our project copy of OCS 0.3.2 uses centered Halpern primal–dual hybrid gradient updates with fixed diagonal steps and block-average restarts [Chambolle and Pock, 2011, Halpern, 1967, Applegate et al., 2022]. Each step applies K, K^T, objective-shifted box projection and the appropriate dual sign projection. JAX [Bradbury et al., 2018] compiles the update blocks. Reversible power-of-two scaling reduces coefficient disparity; outward row/column sums choose steps with $\|\Sigma^{1/2} K T^{1/2}\|_2 \leq 0.9$. All residual and bound checks return to original coordinates.

Working supports and implicit full-domain points. A checked lift embeds each restricted support into the source LP. One expanded-residency support has $36{,}508$ columns and $313{,}061$ rows; OCS reached a checked 0.0774% working interval in 313.25 seconds on an A100, including preparation and checking. This alone is not a full-master bound. For the full input, we improve independently identified owner retention, original tiles and streamed-weight choices, then represent temporal fields implicitly as weighted translations. GPU evaluation streams edge events while the central affine envelope is summed analytically. Capacity is checked on the resulting fractional point; no full event-expanded vector is allocated.

Complete-domain LP certificate. Coverage prices are checked against every implementation in the frozen generators. Work-mass and service rows extend those inequalities to every original event; the minimum of the 21 branch bounds gives the family lower bound. Together with the admitted implicit primal, this yields $L = 734{,}095.710904$ ms and $U = 734{,}720.389756$ ms, hence $(U - L)/U = 0.0850226646\%$: a certified 0.0850% LP objective gap, rounded. The certificate closes the continuous objective interval through specialized primal transformations and algebraic bounds, not a claim that generic PDHG iterates converged on the entire expansion (Appendix P.4). This certificate concerns the frozen native instance; it is not transferred to the strengthened physical branches in

Eq. (Master). A new source with the corrected BF16 chip instead gives a checked 0.0846842% interval over its complete implementation and temporal domain (Appendix P.5). Hardware is fixed in this result; the numerical reconstruction engine has not converged over the joint master.

Hardware branch and bound. For integral recovery, a finite tree partitions array counts, vector lanes, memory bandwidth, host channels and local-service units. Optimistic resources and capacity-priced retention paths bound each region; budget-infeasible regions or those unable to beat a replayed incumbent are pruned. Promising leaves are evaluated with actual capacity and implementation costs. The demonstrated tree covers a declared hardware grid and conditional software choices, not every discrete selector of the master. It does not re-solve the full temporal LP at every node. Appendix P.3 describes the bounds and the unresolved schedule leaves.

Recovery and replay. Recovery selects one branch, an integral cover, hardware allocations and legal storage/transfer actions. Analytical replay recomputes services at the selected rates, dependency readiness and capacities, yielding the reported chip latency. The LP certificate and recovered design are separate outputs: integer choices do not remove continuous reciprocal/product relaxation error, and the continuous LP interval is not a discrete-design optimality guarantee.

6 Experiments

We evaluate predicted latency, search cost, and recovered schedules. Studies include prefill/decode and fixed-hardware mapping. For example, the full-input DeepSeek case adds autoregressive decode and phase-specific controls. TPU-v3 provides a publicly documented reference baseline consistent with prior work [Jouppi et al., 2021, Zhang et al., 2022]. All speedups below are analytical and modeled results.

6.1 Experimental Setup

The suite comprises EfficientNet-B1–B7, Llama-1 13B/65B, Llama-3 70B, and DeepSeek-V3 [Tan and Le, 2019, Grattafiori et al., 2024, DeepSeek-AI, 2024] and Stable Diffusion 1.5 UNet. The campaign compares static pinning, fixed-tile temporal placement, and joint tiling under the same cost convention. These are restricted controls of CHEETAH, not separate proposed systems. Small programs use HiGHS [Huangfu and Hall, 2018]; operator-based native working programs use OCS. The new BF16 suite recovery enumerates a finite common-chip menu and minimizes independent implementation or stage choices within each candidate. These recovered executions are evaluated separately from continuous LP certificates. The full-input experiment binds actual checkpoint routes and preserves every logical invocation (Appendix P).

The new checkpoint-bound comparisons use the same numerical contract on both arms: BF16 matrix operands and activations, FP32 accumulation and the pinned implementation's sensitive operations, with TF32 and reduced-precision accumulation disabled. Checkpoint conversion and data movement are charged. Historical prefill results retain their original numerical and workload contracts; we do not combine their speedups with the new BF16 results.

Physical Design Validation. Since modeling simulations ultimately present a gap with real world physical performance, we validate all results through a five-level audit process: (i) Roofline Auditor, (ii) Campaign Validator, (iii) Causal Ledger, (iv) solver diagnostics, and (v) six pre-RTL gates. Each CHEETAH design must pass all five audit tiers before it is deemed feasible and reported.

The roofline check rejects costs below compute or memory service floors. Table 3 lists the architectural screening thresholds. These are modeling checks, not fabricated-chip timing or packaging sign-off. RTL and synthesis evidence is in Appendix B.

6.2 Reported Efficiency Quantities

Three quantities carry the efficiency claims below, and each is read off the scheduled execution rather than assumed. Let T be the duration of the interval being reported, C_{peak} the peak arithmetic throughput of the recovered chip, b_r the peak bandwidth of resource r, and P_{peak} its modeled peak

Table 3: Six architectural feasibility gates.

Gate	Check	Threshold
Pin count	HBM3-class signal pins at 0.8 GB/s/pin	$\leq 4{,}096$
Reticle area	Linear area model at 7 nm	≤ 600 mm^2
Logic depth	MAC-tree combinational depth	≤ 35 levels
NoC routing	$n_{\text{PEs}}\, n_{\text{banks}}$	$\leq 5 \times 10^6$ cells
Power density	Dynamic power / die area	≤ 2.0 W/mm^2
SRAM macros	64 KiB banks, aspect ratio 1:4–4:1	$\leq 4{,}096$ banks

power.

$$\text{MFU} = \frac{F_{\text{useful}}}{C_{\text{peak}} T}, \qquad \text{MBU}_r = \frac{B_{r,\text{read}} + B_{r,\text{write}}}{b_r T}, \qquad \text{Perf/TDP} = \frac{Q/T}{P_{\text{peak}}}. \tag{3}$$

Model FLOP utilization. MFU is the fraction of the chip's arithmetic capability that the schedule converts into useful work, where F_{useful} counts only the arithmetic the model requires. It is distinct from the work the arrays actually issue: padding, masked edge tiles and replicated operands are charged to the machine but are not useful, so $F^{\text{useful}} \leq F^{\text{issued}}$ always. Within a fixed-chip branch the per-event service rows enforce $\sum_i F^{\text{issued}}_{i,e} x_{i,e} \leq C_{\text{peak}} \Delta t_e$, which sums every concurrent implementation sharing the arrays in event e; because the installed rate is a branch constant there, summing over the event intervals guarantees an aggregate $\text{MFU} \leq 1$ by construction rather than by inspection. This guarantee is a property of the fixed-chip rows and is not inherited by the variable-hardware relaxation, where the corresponding product is bilinear and its McCormick envelope is necessary but not exact (Appendix O.5); the installed-rate condition is therefore re-checked exactly at execution admission.

Model bandwidth utilization. MBU_r is the same ratio for a memory resource, and it is computed from the *complete* scheduled traffic: operand rereads, KV transfers between timesteps, and spill and restore movement all count, since each occupies the interface whether or not it advances the model's arithmetic. Because the resources have different peak rates and different consequences for a schedule, HBM, the host link and on-chip SRAM are reported separately rather than aggregated into a single figure.

Performance per TDP. Perf/TDP divides delivered throughput, Q units of work per unit time, by modeled peak power. Three conditions govern its use as a comparison. The process, the power scope and the unit of work must match between the designs compared; a ratio across differing scopes is not a ratio of efficiencies. Average measured power is not a substitute for modeled peak TDP, and we do not interchange them. And because the objective minimizes latency rather than this ratio, a performance/power tradeoff is obtained by solving across power budgets and comparing recovered designs: for a fixed amount of work the ratio factors as $(\text{Perf/TDP})_{\text{design}}/(\text{Perf/TDP})_{\text{baseline}} = (T_{\text{baseline}}/T_{\text{design}}) \cdot (\text{TDP}_{\text{baseline}}/\text{TDP}_{\text{design}})$, so a latency objective does not by itself maximize it.

All three are reported over the prefill interval, the decode interval, and the whole task separately, because a quantity averaged across phases conceals the regime change between them. Values recovered from a fractional relaxation are reported as bounds, not as admitted physical designs; only points that survive integral recovery and replay are reported as designs.

6.3 Workload Results and Fixed-Hardware Controls

The original campaign reports an average $22.07\times$ predicted speedup over its modeled TPU-v3 baseline, reaching $40.1\times$ on Llama-3 70B. These are prefill-oriented results under that campaign's workload and traffic conventions. Appendix L shows the concrete hardware blueprints; Appendix C has the original plots and restricted controls.

Uniform BF16 extension. The following archived comparisons use the earlier host-backed reference and physical-cost coefficients. They are not yet verified comparisons against FAST's modeled TPU-v3 baseline. The BF16 power and interface correction in Appendix P.1 requires new chip recovery; the original numerical records are retained here with that scope. Table 4 applies the DeepSeek

Table 4: Archived BF16 vision predictions, including preparation and writeback. All latencies are ms. SW holds the modeled TPU-v3 resources fixed.

Workload	Reference	Reference + SW	Custom	Speedup
EfficientNet-B1	13.657	3.557	0.826	16.53×
EfficientNet-B2	17.124	4.372	1.011	16.94×
EfficientNet-B3	28.752	7.276	1.576	18.25×
EfficientNet-B4	59.880	15.673	3.128	19.14×
EfficientNet-B5	117.697	29.791	5.946	19.79×
EfficientNet-B6	200.960	53.037	9.859	20.38×
EfficientNet-B7	354.137	97.591	16.909	20.94×
SD1.5 UNet (50 calls)	109,439.059	55,112.649	2,712.737	40.34×

Table 5: Latency relative to the full BF16 design on the same chip. Larger values show the cost of removing a feature.

Workload	Materialized patches only	Best single matrix tile
EfficientNet-B1	2.00×	1.040×
EfficientNet-B2	2.07×	1.049×
EfficientNet-B3	2.15×	1.064×
EfficientNet-B4	2.29×	1.065×
EfficientNet-B5	2.38×	1.090×
EfficientNet-B6	2.44×	1.096×
EfficientNet-B7	2.48×	1.103×

comparison convention to fresh EfficientNet-B1–B7 and UNet captures. Each classifier result covers one complete classifier forward at its original resolution, cold checkpoint conversion/import and final output writeback. The reference uses the same frozen matrix mapper and scaling, 900 GB/s DRAM, 32 GB/s host service, 2,048 vector lanes and 8,192 GB/s local service as the reported DeepSeek comparison. These latter service rates are analytical assumptions. The software control retains the reference matrix implementations while keeping intermediate tensors in DRAM and vector temporaries in registers.

The custom search selects a common chip and independent per-invocation tiles from a finite recovery family. Convolution may materialize patches or read them directly; direct reads pay 24 scalar address/mask cycles per issued input element. The execution serializes transfers and kernels, retains all observed classifier allocations in DRAM, and charges FP32-checkpoint-to-BF16 conversion. Rational replay checks every original invocation, dependency, allocation, transfer and selected service count. This is analytical recovery, not a full-master optimality certificate or a measurement of the custom kernels.

B1–B4 select 512 16×16 arrays and 114 MiB SRAM; B5–B7 select 896 arrays and 185.875 MiB SRAM. Both designs have 16,384 vector lanes, 2,608 GB/s DRAM and 32,768 GB/s local service. Their host interfaces supply 512 and 256 GB/s respectively. Area/power proxies are 411.64 mm^2/295.55 W and 523.44 mm^2/298.47 W under the earlier coefficient set. These old power values do not establish compliance with the repaired 300-W BF16 model. This comparison includes the explicit interface provisioning difference. The seven fixed-resource software controls improve predicted latency by 3.63–3.95×; full co-design gives 16.53–20.94×. Classifiers and denoising UNets have no autoregressive prefill/decode split.

Same-chip feature controls. Table 5 removes one implementation freedom at a time while retaining the recovered chip, all original operations and their order, DRAM residence, cold preparation and output writeback. Direct affine convolution reads avoid materializing patches, after paying their address generation and masked reads. Per-invocation tiling chooses different matrix tiles for different operations. Its control uses the best single tile from the same five-tile menu for the whole network, while preserving independent convolution dataflow choices. All resulting schedules pass rational replay. These effects interact and are not additive; the controls optimize their declared serial implementation menus, not every feature-disabled schedule.

Table 6: Archived ratios over the earlier host-backed TPU-v3-resource reference; baseline and BF16 physical-cost reconciliation remain open. Phase ratios do not add. Dashes denote inapplicable phases.

Workload	Overall	Prefill	Decode
EfficientNet-B1	16.53×	–	–
EfficientNet-B2	16.94×	–	–
EfficientNet-B3	18.25×	–	–
EfficientNet-B4	19.14×	–	–
EfficientNet-B5	19.79×	–	–
EfficientNet-B6	20.38×	–	–
EfficientNet-B7	20.94×	–	–
SD1.5 UNet (50 calls)	40.34×	–	–
Llama-1 13B	122.59×	48.00×	148.94×
Llama-1 65B	68.92×	25.36×	80.76×
Llama-3 70B	91.78×	47.14×	105.93×
DeepSeek-V3	31.71×	19.85×	37.93×

Denoising UNet. The UNet result covers all 50 captured denoising calls; conditioning, the scheduler and image decoding remain outside that declared boundary. Its chip has 1,024 arrays, 209.75 MiB SRAM and a 128-GB/s host interface, with the same vector, DRAM and local service rates. A paid 128-KiB GELU table is checked over every finite BF16 input and all original tensor layouts; positional tables are recomputed from their captured constant programs. Preparation contributes 32.508 ms and denoising with boundary transfers 2,680.229 ms. The result is an analytical UNet design, not a complete text-to-image timing measurement. On this same chip, restricting GELU to one banked lookup-table copy increases latency from 2,712.737 to 7,304.729 ms (2.693×); the full design chooses the number of table replicas per operation and pays their bank capacity and reload traffic. Materializing every convolution patch increases latency to 2,987.345 ms (1.101×). In contrast, the best common matrix tile gives 2,713.233 ms, only 1.00018× slower. Thus adaptive tiling contributes little for this UNet, whereas lookup replication is consequential. Each control retains the original 50 calls, numerical table, chip and boundary transfers; their effects are not additive.

Common comparison across the suite. Table 6 preserves the archived designs under the BF16/FP32 arithmetic and modeled reference convention used for DeepSeek. Each ratio compares the same complete declared task on the two chips; task sizes differ across models and are specified below. Cold preparation is included. The historical host-backed reference is not yet verified against FAST's modeled TPU-v3 baseline, and these chips require re-screening under the consistent BF16 physical-cost contract (Appendix P.1). The hardware and software controls following the table identify which changes contribute, without adding interacting speedup ratios.

Full-inference Llama designs. The same BF16/FP32 arithmetic and reference service rates apply to 32 independent requests per model, each producing 512 greedy output tokens. Llama-1 retains its original 1,536-token prompt; Llama-3 uses 4,096 tokens. Prefill emits the first output, followed by 511 decode forwards per request. Cold checkpoint conversion/import and final backing are charged in every arm. These are modeled extensions of the paper reference convention, not measured TPU serving latencies. The reference remains the original host-backed singleton schedule; the software arm holds all reference resources fixed.

Recovery selects a common chip and compatible linear batches. Between linear stages, each member executes its original dependent operations, keeping attention temporaries private. Gather/scatter, KV copies and all original token dependencies are retained. Whole immutable parameters may remain in DRAM across stages and timesteps after their first paid import. Their complete extents are conservatively reserved in addition to the original KV/activation peak. A bounded residency MILP selects these leases. The fixed-resource software arm also batches linears and shares weights within each stage, using memory-feasible complete-request cohorts and the same matrix mapper. These finite recovery searches provide analytical executions; the separate DeepSeek LP certificate is not transferred to them.

Llama-1 13B selects 512 16×16 arrays, 16,384 vector lanes, 114 MiB SRAM and a 512-GB/s host interface (411.64 mm^2/295.55 W proxies). Its 43,350,816 logical operations complete in 1 cohort(s) of 32 requests. It retains 275 parameters occupying 13.416 GiB; the conservative DRAM bound

Table 7: Uniform full-inference predictions in seconds for all 32 requests. Preparation is included in totals. SW keeps TPU-v3 resources fixed.

Model	Phase	Reference	Reference + SW	Custom	Speedup
Llama-1 13B	Prefill	980.380	720.526	20.425	48.00×
	Decode	17,668.410	1,317.840	118.631	148.94×
	Total	18,663.574	2,053.150	152.245	122.59×
Llama-1 65B	Prefill	4,407.777	pending	173.827	25.36×
	Decode	82,145.424	pending	1,017.176	80.76×
	Total	86,627.355	pending	1,256.908	68.92×
Llama-3 70B	Prefill	15,076.868	10,905.168	319.824	47.14×
	Decode	106,905.123	7,982.193	1,009.241	105.93×
	Total	121,981.992	18,887.361	1,329.065	91.78×

Table 8: Matched custom-chip controls. Extra predicted time in seconds relative to each retained design; effects are not additive.

Model	Control	Prefill extra	Decode extra	Latency ratio
Llama-1 13B	No linear batching	0.000	222.329	2.460×
	Singletons, no weight leases	2.298	1,417.469	10.326×
	KV spill/restore	0.115	137.112	1.901×
	No cross-step weight residency	0.000	20.976	1.138×
Llama-1 65B	No linear batching	0.000	1,324.921	2.054×
	Singletons, no weight leases	6.969	4,909.761	4.912×
	KV spill/restore	0.242	287.970	1.229×
	No cross-step weight residency	0.139	94.616	1.075×
Llama-3 70B	No linear batching	0.000	1,175.788	1.885×
	Singletons, no weight leases	12.443	7,551.977	6.692×
	KV spill/restore	0.124	134.051	1.101×
	No cross-step weight residency	0.000	17.593	1.013×

is 63.998 GiB within 64 GiB. Prefill/decode account for 5.19%/94.81% of predicted time saved; preparation is separate. Fixed-resource software improves the reference by 9.09×; the custom design is a further 13.49× faster.

Llama-1 65B selects 128 16×16 arrays, 12,288 vector lanes, 74.375 MiB SRAM and a 1,024-GB/s host interface (326.38 mm^2/298.15 W proxies). Its 114,898,464 logical operations complete in 4 cohort(s) of 8 requests. It retains 335 parameters occupying 23.045 GiB; the conservative DRAM bound is 64.000 GiB within 64 GiB. Prefill/decode account for 4.96%/95.03% of predicted time saved; preparation is separate.

Llama-3 70B selects 512 16×16 arrays, 16,384 vector lanes, 114 MiB SRAM and a 512-GB/s host interface (411.64 mm^2/295.55 W proxies). Its 133,248,544 logical operations complete in 1 cohort(s) of 32 requests. It retains 373 parameters occupying 11.241 GiB; the conservative DRAM bound is 63.997 GiB within 64 GiB. Prefill/decode account for 12.23%/87.77% of predicted time saved; preparation is separate. Fixed-resource software improves the reference by 6.46×; the custom design is a further 14.21× faster.

All custom designs retain 64-GiB DRAM and use the same paid memory-interface and local-service model as DeepSeek. These comparisons include differences in installed interface bandwidth. Local and A100 replays check service and logical memory; physical fragmentation and silicon timing are not measured.

In the historical controls, Table 9 holds each chip fixed and compares the joint schedule against streaming every tensor. The compact NPU has 32,768 INT8 MACs at 1 GHz, 64 GB/s bandwidth, and 16 MiB on-chip storage; TPU-v3 uses the original 900 GB/s, 32 MiB model. The larger NPU gains reflect its tighter memory bandwidth. No hardware-area normalization is needed within a same-chip pair.

6.4 Search Cost and LLM Guidance

Cached implementation coefficients remove repeated mapper calls from the inner search. In the original 50-trial comparison, joint tiling reduces Llama-70B search time from 58.42 to 1.17 hours

Table 9: Historical fixed-chip speedup, under the original numerical contracts.

Workload	TPU-v3	Compact NPU
Llama-1 13B	2.14×	31.4×
Llama-3 70B	2.51×	7.66×
DeepSeek-V3	2.22×	7.8×
SD1.5 UNet	1.50×	5.89×
Geometric mean	2.06×	10.3×

and DeepSeek-V3 from 69.39 to 2.17 hours relative to the fixed-tile temporal pipeline. These are search times, not inference speedups. The LLM-guided campaign reduces mean invalid trials from 24.5% to 5.9% while reaching the same best observed Perf/TDP score as its full-space Vizier control. This tests proposal efficiency, not proof of an optimal architecture or causal validity of every dual suggestion.

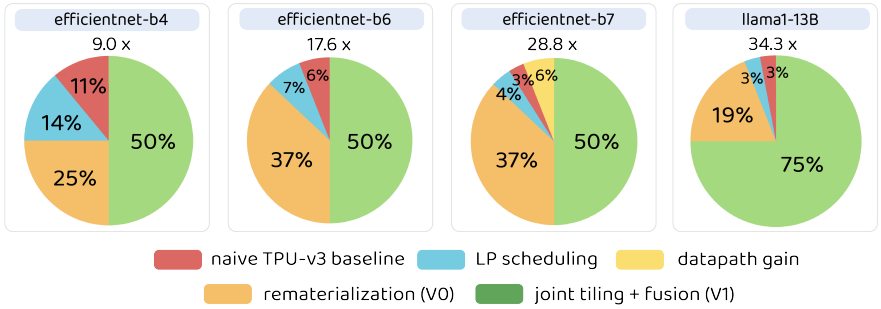


Figure 3: Retained historical prefill attribution under the original campaign's numerical and traffic conventions. The artwork's V1 label denotes joint tiling and fusion. New uniform BF16 phase and feature controls are reported separately in Tables 7, 8, and 5.

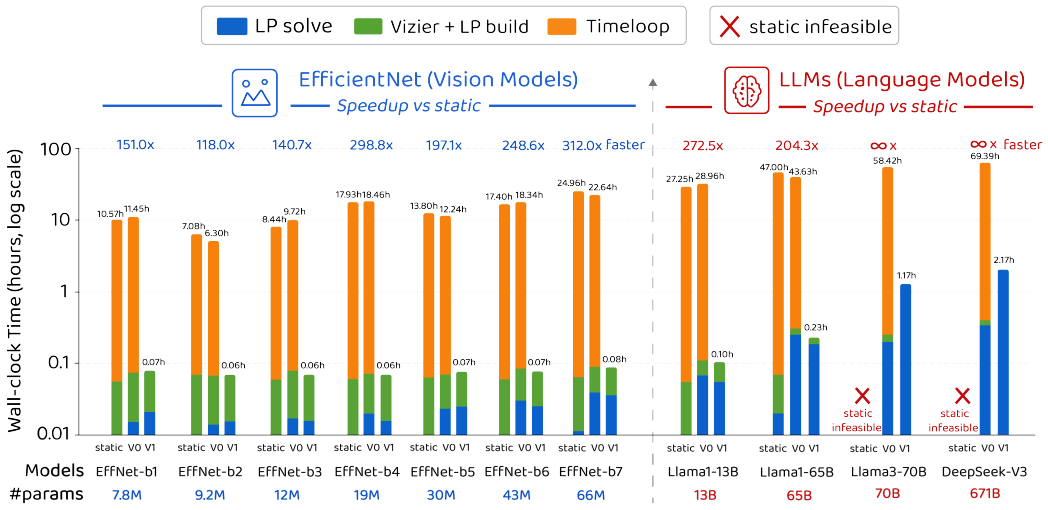


Figure 4: Wall-clock time decomposition and performance acceleration across formulations: CHEETAH-V1's internalized tiling eliminates the Timeloop bottleneck, achieving up to 100–300× faster search than the classical static method. Upper bars show wall-clock hours (log scale); annotations show speedup vs. static.

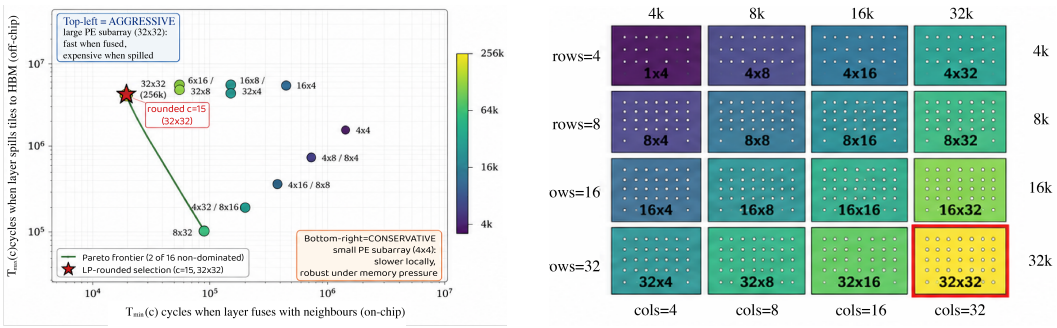


Figure 5: Example per-layer tiling cache for EfficientNet-B7 layer L4. Left: Timeloop-evaluated candidates plotted by fused/on-chip cost $T_{\min}(c)$ and spilled/off-chip cost $T_{\max}(c)$; the frontier shows non-dominated choices and the star is the LP-rounded selection. Right: the 16 tiling choices; labels such as 4×4, 8×8, and 32×32 denote active tile rows/columns inside the fixed accelerator, white dots indicate active PE/MAC sites. These are schedule choices, not different chips.

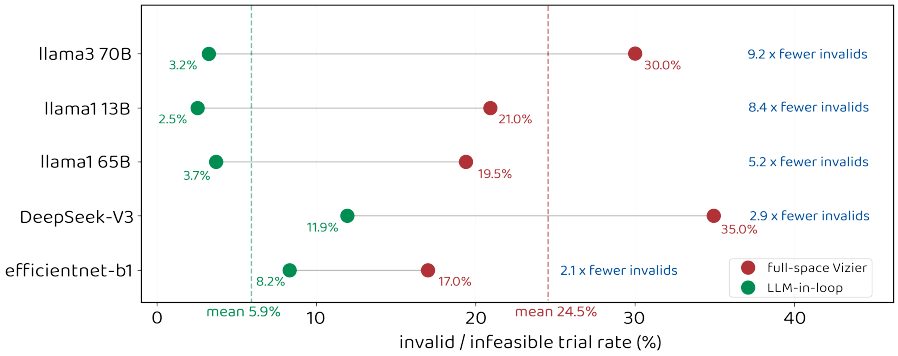


Figure 6: Invalid/infeasible trial rate (%) for full-space Vizier vs. LLM-in-loop across five workloads. The LLM Architect reduces the mean invalid rate from 24.5% to 5.9% (4.1× reduction), with the largest gains on LLM workloads where the hardware search space is most prone to infeasible configurations.

6.5 Full-Inference DeepSeek: One Chip, Prefill and Decode

We evaluate DeepSeek-V3 on 32 independent requests, each with 4,096 prompt and 512 output tokens. All 61 layers execute: prefill produces the first output, followed by 511 decode forwards. Captured expert IDs and routing weights define all 61,985,244 invocations in 41,924,510 execution groups. Original FP8 weights are converted to BF16 with the captured rounding policy; higher-precision tensors and complete KV history are preserved.

The recovered chip has 320 16×16 MAC arrays, 5,248 vector lanes, 1,024 paid FP8 receiver lanes, 78.25 MiB SRAM, 64 GiB DRAM and 2 TiB backing. At a 1 ns analytical clock, the corrected chip provides 1,630 GB/s DRAM, 640 GB/s external and 20,480 GB/s local service. It passes the shared BF16 scenario's area, peak-power, density, pins, bank, depth and NoC predicates: 321.69 mm^2 and 298.69 W. The BF16 peak coefficient is 1.2 mW/MAC; the area scale and interface organization remain explicit concept assumptions, not calibrated silicon estimates. Retiming preserves the original cover, addresses and outer dependency order; 384 regenerated pipeline recipes pass replay, and all other durations are conservatively extended. The historical TPU-v3 extension retains the reference compute/DRAM resources with 32 GB/s external and 8,192 GB/s local service, and 2,048 primitive vector cycles/ns. These interface assumptions are explicit parts of the prediction, not measurements of a calibrated TPU serving stack or fabricated custom chip. This extension is not yet a verified reconstruction of FAST's modeled TPU-v3 baseline, which includes ordinary XLA fusion. The ratios below use that retained reference with the corrected custom chip; they remain conditional comparisons, not established reproduction of FAST's modeled TPU-v3 baseline.

Table 10: Corrected custom-chip predictions (seconds); ratios use the retained, uncalibrated TPU-v3 extension.

Phase	TPU-v3 extension	Custom chip	Speedup
Prefill	13,387.193	1,078.914	12.408×
Decode	53,083.324	2,074.999	25.582×
Preparation	131.252	42.973	3.054×
Total	66,601.769	3,196.886	**20.833**×

Table 11: Archived-chip feature controls: predicted time saved (seconds).

Feature	Prefill	Decode
Matrix batching	43.173	856.070
SwiGLU primitive fusion	14.579	1.819
Other elementwise register storage	257.071	34.026
Full-K panel reuse	32.445	0
Per-use weighted decode tiles	0	0.866
Optional KV residency	0	15.726
Phase-reused placement (included above)	0	3.698

Phase times add; speedup is the ratio of total predicted inference latencies. Decode contributes 80.45% of time saved, prefill 19.41%, and preparation 0.14%. Initial conversion is charged in preparation; subsequent streamed conversion is charged to the consuming kernels. Optimization time is separate. The retained execution's state replay is reused with the checked retiming; the corrected fixed-chip LP separately has a 0.0846842% objective interval (Appendix P.5). Neither this interval nor the retiming establishes global integral optimality.

6.6 Feature Contributions on the Same Chip

Table 11 retains the archived chip and removes one mechanism at a time on the original numerical workload. These are matched execution controls, not globally optimized feature-off designs. Interacting rows are not an additive attribution of the headline speedup. They have not been retimed to the corrected chip in Table 10.

Batching and storage fusion. The no-batch control executes every original member separately and reselects its tile. Resident weights remain resident; each streamed member pays its own receive. The control expands to exactly $61{,}985{,}244$ kernels and takes $2{,}999.864$ seconds, versus $2{,}100.621$ with batching. Thus grouped execution and shared streamed-weight delivery give $1.612\times$ decode and $1.064\times$ prefill gains. SwiGLU and elementwise storage choices remove primitive spills while retaining arithmetic, BF16 rounding, and local/vector service. Full-K panels reuse weights across output-row tiles, paying SRAM and serialized panel waves. Holding one tile fixed per parameter family adds 0.866 seconds to decode; panel boundaries, buffers and residency stay fixed. This tests weighted decode tile adaptation, rather than every temporal choice in the master.

KV and time-indexed placement. Every arm retains complete cached decoding, growth, host backing and mandatory writes. Optional DRAM residency selects 246 request/layer/K-or-V prefix chains and pays 125,952 copies. A fixed whole-task partition fits 193 chains; releasing prefill temporaries before decode permits 246. Placement's 3.698-second gain is included in the 15.726-second KV gain. These modest marginal gains reflect limited DRAM capacity and the wide custom external interface. Independent checks replay changed copies and kernel traffic; Appendix P records exact inputs, costs, controls and their scope.

7 Conclusion

CHEETAH jointly represents hardware, implementation, and temporal memory choices in one master LP relaxation. Operator-based optimization, checked bounds, and integral replay produce concrete

designs. Full-input captures and matched analytical controls expose prefill, decode and memory-reuse effects. The revised DeepSeek concept passes a declared BF16 physical scenario and has a tightly bounded fixed-chip LP. Its comparison with FAST's modeled TPU-v3 baseline, calibrated cost assumptions and joint numerical convergence remain open; the accepted original campaign retains its separate results and evidence.

Acknowledgments. We gratefully acknowledge support from federal sources: NSF under No. 24-554 (AIMing) and DARPA under No. HR00112520038 (Fallingwater). We also gratefully acknowledge support from Stanford HAI, IBM (Code Generation), Lightspeed, Felicis, and Google. This work was also partially supported by IBM and Felicis as an affiliate member of Stanford Institute for Human-centered Artificial Intelligence (HAI)'s industry program.

This work was supported in part by the National Science Foundation (NSF) CAREER Award under Grant CCF-2236829, in part by the NSF under Grant CNS-2551282, in part by the National Institutes of Health under Grant 1R01AG08950901A1, in part by the Office of Naval Research under Grant N00014-24-1-2164, and in part by the Defense Advanced Research Projects Agency under Grant HR00112490441.

Miria Feng was supported in part by the Stanford Graduate Fellowship. We also thank Lucy Woof for her continuous support throughout, Dan Zhang for his invaluable guidance, Kevin Nam for feedback on an early draft and analysis, and ZhongWei Dang on RTL and physical design conversations.

References

T. Ajayi, V. A. Chhabria, M. Fogaça, S. Hashemi, A. Hosny, A. B. Kahng, M. Kim, J. Lee, U. Mallappa, M. Neseem, et al. Toward an open-source digital flow: First learnings from the OpenROAD project. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*, 2019.

T. Andrulis, M. Gilbert, V. Sze, and J. S. Emer. Fast and fusiest: An optimal fusion-aware mapper for accelerator modeling and evaluation. *arXiv preprint arXiv:2602.15166*, 2026.

D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O'Donoghue, and W. Schudy. Practical large-scale linear programming using primal-dual hybrid gradient, 2022. URL https://arxiv.org/abs/2106.04756.

R. Baghdadi, J. Ray, M. B. Romdhane, E. Del Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 193–205, 2019.

U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. A practical automatic polyhedral parallelizer and locality optimizer. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 101–113, 2008.

J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/google/jax.

J. Cai, Z. Wu, S. Peng, Y. Wei, Z. Tan, G. Shi, M. Gao, and K. Ma. Gemini: Mapping and architecture co-exploration for large-scale DNN chiplet accelerators. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024. arXiv:2312.16436.

A. Chambolle and T. Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of mathematical imaging and vision*, 40(1):120–145, 2011.

Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze. Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices, 2019. URL https://arxiv.org/abs/1807.07928.

L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric. ASAP7: A 7-nm FinFET predictive process design kit. In *Microelectronics Journal*, volume 53, pages 105–115, 2016.

Computing Research Association. How AI will reshape computer systems by 2035: A Jeffersonian dinner in San Francisco about our 10,000× future. CRA Industry Blog, Apr. 2026. URL https://cra.org/industry/2026/04/27/how-ai-will-reshape-computer-systems-by-2035-a-jeffersonian-dinner-in-san-francisco-about-our-10000x-future/. Accessed: 2026.

T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.

DeepSeek-AI. Deepseek-v3 technical report, 2024. URL https://arxiv.org/abs/2412.19437.

E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

D. Golovin, B. Solnik, S. Moitra, G. Kochanski, J. Karro, and D. Sculley. Google vizier: A service for black-box optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*, pages 1487–1495. ACM, 2017. doi: 10.1145/3097983.3098043. URL https://doi.org/10.1145/3097983.3098043.

A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

B. Halpern. Fixed points of nonexpanding maps. 1967.

C. Hong, Q. Huang, G. Dinh, M. Subedar, and Y. S. Shao. DOSA: Differentiable model-based one-loop search for DNN accelerators. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023. doi: 10.1145/3613424.3623797.

C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Q. Huang, M. Kang, G. Dinh, T. Norell, A. Kalaiah, J. Demmel, J. Wawrzynek, and Y. S. Shao. Cosa: Scheduling by constrained optimization for spatial accelerators, 2021. URL https://arxiv.org/abs/2105.01898.

Q. Huang, P.-A. Tsai, J. S. Emer, and A. Parashar. Mind the gap: Attainable data movement and operational intensity bounds for tensor algorithms. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024. Orojenesis: mapping-independent attainable off-chip traffic and operational-intensity bounds for tensor programs; used here to certify the compulsory-DRAM (attainable) floor.

Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

P. Jain, A. Jain, A. Nrusimha, A. Gholami, P. Abbeel, K. Keutzer, I. Stoica, and J. E. Gonzalez. Checkmate: Breaking the memory wall with optimal tensor rematerialization, 2020. URL https://arxiv.org/abs/1910.02653.

S. Jia, Z. Ling, C. Bai, K. Zhao, and J. Zhai. Fadiff: Fusion-aware differentiable optimization for dnn scheduling on tensor accelerators, 2025. URL https://arxiv.org/abs/2511.22348.

N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12. ACM, 2017. doi: 10.1145/3079856.3080246. Reports mean useful-MAC utilization and weight-fetch stall fractions for a production matrix-unit accelerator; also shows that enlarging the matrix unit can degrade delivered performance (sub-linear return on added MACs).

N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma, et al. Ten lessons from three generations shaped google's tpuv4i: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–14. IEEE, 2021.

W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

H. Lu and J. Yang. cupdlp.jl: A gpu implementation of restarted primal-dual hybrid gradient for linear programming in julia, 2024. URL https://arxiv.org/abs/2311.12180.

H. Lu, J. Yang, H. Hu, Q. Huangfu, J. Liu, T. Liu, Y. Ye, C. Zhang, and D. Ge. cupdlp-c: A strengthened implementation of cupdlp for linear programming by c language, 2024. URL https://arxiv.org/abs/2312.14832.

H. Lu, Z. Peng, and J. Yang. cupdlpx: A further enhanced gpu-based first-order solver for linear programming, 2025. URL https://arxiv.org/abs/2507.14051.

I. Magaki, M. Khazraee, L. V. Gutierrez, and M. B. Taylor. Asic clouds: Specializing the datacenter. *ACM SIGARCH Computer Architecture News*, 44(3):178–190, 2016.

G. P. McCormick. Computability of global solutions to factorable nonconvex programs: Part i: Convex underestimating problems. *Mathematical programming*, 10(1):147–175, 1976.

L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst. Zigzag: Enlarging joint architecture-mapping design space exploration for dnn accelerators. *IEEE Transactions on Computers*, 70(8): 1160–1174, 2021.

L. Mei, K. Goetschalckx, A. Symons, and M. Verhelst. DeFiNES: Enabling fast exploration of the depth-first scheduling space for DNN accelerators through analytical modeling. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 570–583. IEEE, 2023. doi: 10.1109/HPCA56546.2023.10071098. arXiv:2212.05344.

A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*, pages 304–315, 2019.

S. G. Patil, P. Jain, P. Dutta, I. Stoica, and J. E. Gonzalez. Poet: Training neural networks on tiny devices with integrated rematerialization and paging, 2022. URL https://arxiv.org/abs/2207.07697.

H. Peng, S. Davidson, R. Shi, S. L. Song, and M. Taylor. Chiplet cloud: Building ai supercomputers for serving large generative language models, 2024. URL https://arxiv.org/abs/2307.02666.

B. Rashidi, C. Gao, S. Lu, Z. Wang, C. Zhou, D. Niu, and F. Sun. Unico: Unified hardware software co-optimization for robust neural network acceleration. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 77–90, 2023.

B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G. Tell, Y. Zhang, W. J. Dally, J. Emer, C. T. Gray, B. Khailany, and S. W. Keckler. Simba: Scaling deep-learning inference with multi-chip-module-based architecture. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 14–27. ACM, 2019. doi: 10.1145/3352460.3358302. Network-on-package / network-on-chip provisioning (radix, link count, hierarchical crossbar+mesh) for multi-chiplet DNN inference.

W. Snyder and contributors. Verilator: Open-source SystemVerilog simulator and lint system. https://www.veripool.org/verilator/, 2025. Version 5.050, cycle-based two-state simulation.

B. F. Spector, S. Arora, A. Singhal, D. Y. Fu, and C. Ré. ThunderKittens: Simple, fast, and adorable AI kernels. *arXiv preprint arXiv:2410.20399*, 2024.

M. Tan and Q. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.

S. Tuli, C.-H. Li, R. Sharma, and N. K. Jha. Codebench: A neural architecture and hardware accelerator co-design framework. *ACM Transactions on Embedded Computing Systems*, 22(3): 1–30, Apr. 2023. ISSN 1558-3465. doi: 10.1145/3575798. URL http://dx.doi.org/10.1145/3575798.

R. Venkatesan, Y. S. Shao, M. Wang, J. Clemons, S. Dai, M. Fojtik, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, et al. Magnet: A modular accelerator generator for neural networks. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8. IEEE, 2019.

D. Zhang, S. Huda, E. Songhori, K. Prabhu, Q. Le, A. Goldie, and A. Mirhoseini. A full-stack search technique for domain optimized deep learning accelerators. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, page 27–42. ACM, Feb. 2022. doi: 10.1145/3503222.3507767. URL http://dx.doi.org/10.1145/3503222.3507767.

L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

S. Zheng, S. Chen, S. Gao, L. Jia, G. Sun, R. Wang, and Y. Liang. Tileflow: A framework for modeling fusion dataflow via tree-based analysis. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 1271–1288, 2023.

J. Zhu, C. Xue, Y. Chen, Z. Wang, C. Zhang, Y. Shen, Y. Chen, Z. Cheng, Y. Jiang, T. Wang, Y. Lin, W. Hu, B. Cui, R. Wang, Y. Liang, and G. Sun. Theseus: Exploring efficient wafer-scale chip design for large language models, 2024. URL https://arxiv.org/abs/2407.02079.

Appendix

Table of Contents

A Total Cost of Ownership and Return on Investment . . . 24

B Implementation-Level Validation: Cycle-Accurate RTL . . . 27

B.1 Physical-design validation: post-synthesis area and the linearity of the cost model . . 28

B.2 Measuring the compute-energy coefficients . . . 29

C Discussion and Plots . . . 31

C.1 What a Bound Does and Does Not Establish . . . 31

C.2 Interpreting Hardware and Software Gains . . . 31

D Extended Related Work . . . 32

E Modeling Properties and Containment . . . 33

E.1 Bounded Products and Epigraphs . . . 33

E.2 An Objective-Preserving Embedding . . . 33

E.3 Structure and Safe Pruning . . . 34

F Linear Program Formulation Details . . . 35

F.1 Relaxation and Integral Recovery . . . 35

F.2 Original Campaign Matrix Sizes . . . 35

F.3 LP Matrix Structure . . . 35

F.4 Current Solver Path . . . 35

G Designing for Manufacturability and Feasibility Audits . . . 37

G.1 Five Levels of Validation . . . 37

G.2 MFU and Latency Audits . . . 37

G.3 Six Physical-Design Feasibility Gates . . . 41

G.4 Why We Are Confident in the Software Side . . . 42

G.5 Why We Are Confident in the Hardware Side . . . 42

H Speedup Analysis: Baseline Deficit Attribution and Design-Parameter Sensitivity . . . 44

H.1 Per-Operation Deficit Attribution . . . 44

H.2 Two-Factor Speedup Decomposition . . . 45

H.3 One-Decision-at-a-Time Sensitivity Sweeps . . . 47

H.4 Decision-Impact Ranking . . . 48

I Qualitative Design Discussion . . . 50

I.1 Interesting Empirical Design Results . . . 50

I.2 Hardware and Vizier Search . . . 50

J LLM-Architect Outer Loop Details . . . 52

J.1 Profile-Conditioned LLM-Architect Application . . . 52

J.2 LLM in the Loop Prompts . . . 54

J.3 Resource Prices and Proposal Validation . . . 57

K Two Optimal Tiling Examples: joint-tile LP Solution at a Glance . . . 58

K.1 joint-tile LP Solution: Llama-3 70B on modeled H100 . . . 58

K.2 joint-tile LP Solution: DeepSeek-V3 on modeled RTX 5090 . . . 59
L Examples of CHEETAH Design Blueprints . . . 63
L.1 Variant 1: TSMC N7 + GDDR/LPDDR . . . 63
L.2 Variant 2: TSMC N5 + HBM3 . . . 68
M Limitations, Future Work, and Broader Impact . . . 72
N Historical Shape-Tied Control and Its Limits . . . 72
O Complete Rows of the Master Blocks . . . 73
O.1 The Retained Joint Tiling–Placement Block . . . 73
O.2 Native Cover, Lifecycle and Storage . . . 73
O.3 Capacities, Staging and Installed Hardware . . . 74
O.4 Native Resource Demands and Linear Service Envelopes . . . 75
O.5 Direct Roofline and Physical Constraints . . . 76
O.6 Feature Scope . . . 77
P Native Full-Inference Provenance and Reproduction . . . 78
P.1 Baseline and BF16 physical-cost reconciliation . . . 78
P.2 Archived native results . . . 78
P.3 End-to-End Solution and Recovery Workflow . . . 80
P.4 Archived certificate for the finite native master . . . 82
P.5 Corrected fixed-chip full-domain LP . . . 83
Q Coefficient Audit Trail: Where Every Constant Comes From . . . 84

A Total Cost of Ownership and Return on Investment

A co-designed accelerator only pays for itself if the recurring efficiency it buys eventually amortizes its one-time silicon cost. To evaluate this we adopt FAST's total-cost-of-ownership (TCO) methodology [Zhang et al., 2022] *verbatim* (the same cost model, the same non-recurring-engineering (NRE) decomposition, and the same return-on-investment (ROI) break-even analysis; their Eq. 1–2 and Sec. 5.1), so that our deployment economics are read on FAST's own terms and are directly comparable to their Table 4. We stress at the outset that this is the weakest-grounded part of our evaluation: absolute TCO depends on proprietary fleet data neither we nor FAST possess, several cost constants are assumption-flagged, and the speedup that drives the ROI depends strongly on which baseline it is measured against. We therefore do not report a single S: Appendix A stratifies every Perf/TDP figure over three denominators of increasing strength, and we carry the ROI through on the design that clears the strongest of them (Appendix A), stating plainly that the throughput-provisioned blueprints do not. We report ROI as an order-of-magnitude plausibility check, not a precise claim, and we state every caveat inline.

FAST's cost model (what we mirror). FAST models the three-year cost of a fleet that offloads n baseline accelerator-units of traffic onto a co-designed chip as

$$\mathrm{TCO}_{\mathrm{old}}(n) \;=\; C_{\mathrm{cap}}(n) \;+\; t_D\, C_{\mathrm{op}}(n), \tag{4}$$

capital cost plus lifetime ($t_D = 3$ yr) operating cost, and defines the ROI of committing the co-design NRE as

$$\mathrm{ROI} \;=\; \frac{\mathrm{TCO}_{\mathrm{old}}\,(S-1)}{\left(t_{\mathrm{design}}\, C_{\mathrm{eng}} + C_{\mathrm{mask}} + C_{\mathrm{IP}}\right) S}, \tag{5}$$

where S is the per-unit efficiency gain of the co-designed chip over the baseline; $\mathrm{ROI} > 1$ means the deployment is profitable. Because per-chip TCO is confidential, FAST uses $S =$ Perf/TDP as a proxy for Perf/TCO (the two are reported to be highly correlated for datacenter accelerators [Jouppi et al., 2017, 2021]) and anchors the baseline unit to a commodity DGX-A100 accelerator. We reuse this proxy, this DGX-anchored baseline unit, and FAST's engineering-NRE constant ($t_{\mathrm{design}}\, C_{\mathrm{eng}} =$ 65 engineer-years $\times$ \$240K $\times$ 1.65 overhead $=$ \$25.74M) unchanged. Inverting Eq. (5) at a target ROI multiple R gives the break-even deployment volume

$$n^{\star}(R) \;=\; \frac{R \cdot \mathrm{NRE} \cdot S}{(S-1) \cdot \mathrm{TCO}_{\mathrm{unit}}}, \qquad \mathrm{NRE} = t_{\mathrm{design}}\, C_{\mathrm{eng}} + C_{\mathrm{mask}} + C_{\mathrm{IP}}, \tag{6}$$

the number of baseline units whose traffic must be offloaded before the NRE is repaid. All arithmetic below is a pure evaluation of Eqs. (4) to (6) on the eight co-designed blueprints of Appendix L; no solver is involved.

Break-even under FAST parity. Under FAST's Perf/TDP proxy, our co-designed chips retain $S = 1.15$–$1.79\times$ Perf/TDP over a modeled TPU-v3 after normalizing away raw MAC scale, measured against the c0-naive denominator (D1) of Appendix A. These are *prefill* blueprints under that convention. The iso-resource analysis of Appendix A separately compares against a well-scheduled baseline; neither calculation uses the native full-inference timings. For the best chip (the DeepSeek-V3 [DeepSeek-AI, 2024] N7 design on TSMC N7 with LPDDR5X, $S = 1.79$), Eq. (6) gives a $1\times$-ROI break-even of $\mathbf{4{,}410}$ units (Table 12, Fig. 7). This sits at the *same order of magnitude* as FAST's own reported 2,164–3,534 units (their Table 4, at $S = 1.84$–3.91), and well within the thousands-to-tens-of-thousands scale of modern accelerator fleets. As Eq. (6) makes explicit and Fig. 7 shows, deployment volume dominates ROI: the ranking of chips by break-even volume is driven jointly by S and by NRE, and the N7-plus-commodity-memory family (N7) is uniformly ROI-preferred over the N5-plus-HBM3 family (N5), which carries $\sim$\$15M more mask NRE, 1.7–$2.5\times$ the unit TCO, and (except on the 13B) *lower* Perf/TDP, pushing its break-even to 13,000–19,000 units.

Reconciling the NRE. The NRE terms in Eq. (6) carry the largest uncertainty, and we flag them as assumptions [A] rather than measured quantities: the mask-set cost (\$15M at N7, \$30M at N5), the IP/PHY-licensing cost (\$10M, ASIC-Clouds-class [Magaki et al., 2016]), the defect density D_0 used in the die-yield model, and the PUE all follow public industry-class estimates, not vendor data. FAST published only its engineering-NRE constant, so rather than assert NRE parity we *reconcile*:

Table 12: FAST-parity break-even volumes (convention F: S = Perf/TDP proxy, DGX-A100-anchored baseline unit), computed from Eq. (6). NRE is \$50.7M for the N7 (N7) family and \$65.7M for the N5+HBM3 (N5) family. For reference, FAST's Table 4 reports 1×-ROI volumes of 2,164–3,534 units at $S = 1.84$–3.91. Cost constants are assumption-flagged; see Appendix A.

Chip (blueprint)	**Process / memory**	**S**	**1×**	**2×**	**4×**
DeepSeek-V3 N7	N7 / LPDDR5X	1.79	**4,410**	8,821	17,641
Llama1-65B N7	N7 / LPDDR5X	1.74	4,591	9,182	18,363
Llama3-70B N7	N7 / LPDDR5X	1.54	5,528	11,057	22,114
Llama1-13B N5	N5 / HBM3	1.57	6,920	13,841	27,681
Llama1-13B N7	N7 / GDDR6X	1.42	6,641	13,283	26,566
DeepSeek-V3 N5	N5 / HBM3	1.24	13,083	26,166	52,332
Llama1-65B N5	N5 / HBM3	1.20	15,079	30,158	60,317
Llama3-70B N5	N5 / HBM3	1.15	19,237	38,473	76,947

Figure 7: FAST-parity 1×-ROI break-even deployment volume per co-designed blueprint (convention F), from Eq. (6). The shaded band marks FAST's own reported range (2,164–3,534 units); our best chip (DeepSeek-V3 N7, 4,410 units) lands just above it, and drops to ∼3,650 units (*inside* FAST's range) when evaluated at FAST's implied ∼\$42M total NRE. The N7+commodity-memory (N7) family is uniformly ROI-preferred over N5+HBM3 (N5).

back-computing from FAST's own Table 4 break-even volumes and reported S implies a FAST total NRE of ∼\$42.0M. Our best chip carries a larger, deliberately conservative \$50.7M; re-evaluating Eq. (6) at FAST's implied ∼\$42M moves our best-chip break-even from 4,410 to ∼**3,650** units, now *inside* FAST's own 2,164–3,534 range. The Perf/TDP proxy S is independent of these [A] constants except through NRE, so the headline break-even row is robust to them: doubling the mask cost alone moves the 1×-ROI volume only to ∼5,700. We also disclose the honest error bar from the proxy itself: a fully modeled TCO on both sides (convention M) inflates break-even to ∼79,000–108,000 units (an ∼18× spread), which is precisely why FAST, and we, headline Perf/TDP and cite the Perf/TCO correlation rather than claim TCO precision.

The zero-NRE deployment option. A distinct and strictly cleaner deployment story requires *no silicon at all*. Because CHEETAH's scheduler is a drop-in software layer, applying the joint LP schedule to the *existing* TPU-v3 fleet (same chip, same TDP) delivers $S = 2.14$–$2.51\times$ Perf/TDP on the LLM workloads (2.21× on EfficientNet-B7 [Tan and Le, 2019], 1.50× on the Stable-Diffusion U-Net) at software-only engineering cost, with $C_{\text{mask}} = C_{\text{IP}} = 0$ (Fig. 8). Under Eq. (5) the NRE collapses to a software term and the ROI exceeds any silicon option at every volume. This is a genuine, low-risk path to fleet-wide efficiency that does not depend on any of the [A]-flagged silicon constants above.

Baseline stratification: three denominators, never mixed. Our S and FAST's S are *not measured against the same baseline*: CHEETAH's speedup is measured against a naive streaming schedule on the reference hardware, whereas FAST's is measured against a production-calibrated simulator. Rather than carry a single S and a caveat, we report Perf/TDP against three denominators of increasing strength and tag every number with the one it uses:

(D1) **c0-naive.** TPU-v3 running its deployed per-operation configuration. This is the FAST-comparable "TPU-v3 as deployed" convention.

(D2) **stream-naive.** The per-operation streaming optimum $\sum_i \min_c \max(C_{i,c}, \alpha B_{i,c})$: the strongest capacity-unaware analytic baseline.

(D3) **joint-scheduled.** A TPU-v3 running *CHEETAH's own* joint LP schedule. This is the well-scheduled denominator, and it is the correct hurdle rate for any silicon proposal, because Appendix A shows that schedule is available on the existing fleet at zero silicon NRE.

Against (D3) the co-designed *blueprints* of Appendix L do not clear the bar, and we state the arithmetic rather than a rounded caveat. Their $S = 1.15$–$1.79\times$ is measured against (D1), while the same schedule on unmodified TPU-v3 already buys 2.14–2.51×; the blueprints therefore sit at

Figure 8: Perf/TDP (FAST's TCO proxy) of the two deployment options against a TPU-v3 baseline of 1.0. Co-designed silicon retains 1.15–1.79× after normalizing away MAC scale; deploying CHEETAH *scheduling* on the existing TPU-v3 fleet delivers 2.14–2.51× at zero silicon NRE. The dashed band marks the 0.46–0.84× obtained when the blueprints are measured against a *joint-scheduled* TPU-v3 (denominator (D3)) rather than a naive one, and the marker at 3.00–3.21× is CHEETAH-iso against that same denominator (Appendix A).

Table 13: CHEETAH-iso (65,536 MACs = TPU-v3, 0.82× area, 128 GB/s) Perf/TDP against the three denominators of Appendix A, with the 1×-ROI break-even volume from Eq. (6) computed at the (D3) figure: the strongest baseline, a TPU-v3 running CHEETAH's own joint schedule. NRE is held at the N7 family's \$50.7M even though the die is smaller, which is conservative. FAST's Table 4 reports 2,164–3,534 units. Richer TDP convention ($\mathrm{TDP} = \mathrm{MACs} \cdot 2.3\times10^{-4} + 0.07\,\mathrm{W}/(\mathrm{GB/s}) \cdot \mathrm{BW}$) applied to both sides.

Workload	(D1) c0	(D2) stream	(D3) joint	1× break-even	In FAST range?
EfficientNet-B7	6.96×	3.36×	**3.21×**	2,827	yes
DeepSeek-V3	4.95×	3.05×	**3.00×**	2,919	yes

0.46–0.84× against (D3), $S - 1 < 0$, and Eq. (6) admits *no break-even at any deployment volume*. That is the correct reading of those chips, and it is not a defect of the formulation: the blueprints are provisioned for *throughput*, which Appendix H.2 shows is where 20.0–26.9× of their headline comes from, and raw throughput is not what a Perf/TDP proxy rewards. The blueprints' deployment case rests on per-chip throughput, latency, and the consequent reduction in fleet size and footprint, mirroring FAST's own observation that latency and model-enablement can justify even break-even ROI. It does not rest on Perf/TDP, and we no longer claim that it does.

The iso-resource design does clear the bar. Optimizing Perf/TDP directly, under hard caps at TPU-v3's own area and TDP budget, selects a different chip. **CHEETAH-iso** carries the *same* 65,536 MACs as TPU-v3 at 0.82× its area (73.83 mm^2 against an 89.83 mm^2 cap), and provisions 128 GB/s of GDDR6 where TPU-v3 provisions 900 GB/s of HBM. Because it has no additional MACs, the provisioning factor f_2 of Appendix H.2 is eliminated *by construction*, so what it reports is attributable to the optimization rather than to silicon scale. Table 13 evaluates it against all three denominators.

Two properties make this the number we would have a reader carry. First, it survives the hurdle rate: at $S = 3.00$–$3.21\times$ against (D3), Eq. (6) gives a 1×-ROI break-even of **2,827–2,919** units, *inside* FAST's own reported 2,164–3,534 range, and we hold NRE at the larger family's \$50.7M rather than crediting the smaller die. Second, the design was recovered independently by multiple search arms (all four on DeepSeek-V3; arms 1, 2, and 4 on EfficientNet-B7), so it does not depend on the LLM-guided arm whose attribution we bound in Appendix J.1.

Where the iso-resource win comes from, and how fragile it is. The mechanism is bandwidth provisioning, not compute: with the same MAC count and both sides joint-scheduled, the performance ratio is ≈1 and essentially all of the 3.21× is the TDP denominator (78.07 W → 24.03 W, a 3.25× ratio). The joint schedule raises arithmetic intensity enough that 128 GB/s suffices where TPU-v3 provisions 900 GB/s, and the memory power follows. This makes the result sensitive to one [A]-flagged constant, the $0.07\,\mathrm{W}/(\mathrm{GB/s})$ memory-power coefficient, so we report its sensitivity rather than assert it: the TDP ratio falls to 2.80× at 0.05, 2.38× at 0.035, 1.88× at 0.02, and 1.47× at 0.01, reaching parity only as the coefficient reaches zero. The conclusion that CHEETAH-iso beats a joint-scheduled TPU-v3 on Perf/TDP is therefore robust to a 7× error in that constant; the specific break-even volume is not, and should be read as an order-of-magnitude figure.

Table 14: Did each decision fire? Per-decision plan-versus-execution counters from the DeepSeek-V3 tape replay ($C_{\text{GM}} = 8$ MiB). Every event the LP plans is executed by the RTL engine exactly once, the capacity row is checked in hardware at every step, and the decisions the LP chose not to use at this cell are confirmed absent while their opcodes are proven live on a synthetic tape. The bound says how good the plan is; the counters say the plan is what ran.

Decision	LP plan	RTL executed	Check
Tile choice (y)	903 EXECUTE	903	✓ ids match 903/903
Residency (p, p^W)	180 LOAD, 299 FREE, 2,709 pins	180, 299	✓ 2,709/2,709 pins
Fusion (operand kept resident)	418 of 903 ops	418	✓
Eviction / reload	0 / 0	0 / 0	✓ opcodes live
Rematerialization (r)	0	0	✓ opcode live
Capacity row	1,083 hardware checks	0 violations	✓ injected overflow flags 1
Peak occupancy	7,602,176 B	7,602,176 B	✓ byte-exact, trace within 4 B
Total cycles	1.6437×10^{12}	+16,431 cycles	✓ +0.000001%, accounted

B Implementation-Level Validation: Cycle-Accurate RTL

The results so far certify *model-level* quantities computed inside one cost-model convention; as we note in Appendix G.2, a real measured utilization would require an executable implementation of the selected hardware/schedule pair. This section supplies that check. We take the committed schedule and the datapath geometry the LP sizes, express them as *synthesizable* SystemVerilog, simulate them cycle-accurately with Verilator 5.050 [Snyder and contributors, 2025], and compare against a reference model written in C++. We call this *analytical-model-to-RTL cycle validation*: the RTL confirms that the LP's schedule is physically realizable, that its time-indexed memory constraints hold byte-for-byte in hardware, and that the utilization *mechanisms* the LP charges are the ones a real datapath exhibits. We do *not* treat RTL cycle counts as absolute ground truth (the analytical per-layer cache floors are uniformly conservative by $192\times$ to $2475\times$, quantified below), and we make no tape-out claim. Appendix B.1 then closes the remaining gap on the cost-model side, taking the sized datapath through logic synthesis and placement at a 7 nm predictive PDK to test the area and power coefficients themselves.

Experiment 1: schedule tape replay (realizability and memory legality). We compile the full committed DeepSeek-V3 [DeepSeek-AI, 2024] schedule at a TPU-v3-class configuration ($C_{\text{GM}} = 8$ MiB) into a 1,383-instruction event tape over the vocabulary {EXECUTE, LOAD, FREE, EVICT, RELOAD, REMATERIALIZE}, and execute it on a cycle-accurate SystemVerilog event engine (a Global-Buffer allocator, a bandwidth-limited DMA model, and a compute model) under Verilator. Each instruction is cross-checked against the analytical residency model layer-by-layer, and a deliberately over-committed tape confirms the capacity checker is live (it flags exactly one violation). Three pre-stated criteria pass (Table 15), and Table 14 reports, decision by decision, that every planned event executed. *Memory legality:* the tape runs to completion with **zero** capacity violations, and the RTL peak on-chip occupancy equals the analytical prediction *exactly* (7,602,176 B $= 7.25/8$ MiB, 9.4% headroom). *Cycles:* the total RTL cycle count lands within $+0.000001\%$ of the rounded LP objective, the entire gap being the 16,431 decoder/DMA sequencing cycles (10^{-8} of the total); i.e., the rounded schedule realizes with *no hidden serialization or stall* beyond the model's accounting. *Two-tape ratio:* against a stream-everything realization of the same operation order, the RTL-measured latency ratio is $1.0723\times$, matching the analytical ratio to 8 significant digits. This ratio is scoped to the pin/reload channel alone (both tapes hold the per-operation-optimal tiling fixed, so joint tile selection, the dominant channel elsewhere, is deliberately held equal). Together these establish that the LP's time-indexed residency model (Section 4.2) is not merely roofline-feasible but executes byte- and cycle-faithfully on hardware that enforces the capacity constraint at every step.

Experiment 2: datapath utilization mechanism. We next build one weight-stationary INT8 systolic PE cluster in synthesizable SystemVerilog (4×4 MAC array, INT8×INT8→INT32, with L1 buffers and a tile-loop FSM; plus a 16×16 variant), golden-verified against a plain-C++ reference convolution ($8/8$ runs bit-exact). Simulated on EfficientNet-B7 [Tan and Le, 2019] layers, the RTL reproduces the two utilization mechanisms the cost model charges for. First, the *depthwise structural ceiling*: on a broadcast systolic array a depthwise convolution can engage only 4 of 16 MAC lanes,

Table 15: Analytical-model-to-RTL cycle validation. Synthesizable SystemVerilog simulated cycle-accurately with Verilator [Snyder and contributors, 2025] and checked against a C++ reference. The RTL confirms schedule realizability, byte-exact memory legality, and the LP's utilization mechanisms; it is *not* used as an absolute cycle oracle.

Quantity (RTL vs. analytical)	**Measured**	**Verdict**
Exp. 1: DeepSeek-V3 schedule replay (1,383 *instr.,* $C_{GM} = 8$ *MiB)*		
Capacity violations	0	pass
Peak on-chip occupancy	7,602,176 B (exact)	byte-exact
Total cycles vs. rounded LP obj.	$+0.000001\%$	no hidden serialization
Two-tape ratio (pin/reload only)	$1.0723\times$ (8-digit match)	pass
Exp. 2: EfficientNet-B7 INT8 systolic cluster (golden $8/8$*)*		
Depthwise MAC-lane utilization	24.9% vs. 25% ceiling	mechanism confirmed
Systolic-vs-grid geometry contrast	$4.057\times$ vs. $4.00\times$	mechanism confirmed
Absolute cache cycle floors	192–$2475\times$ conservative	scope guard

and the RTL measures a steady-state depthwise utilization of 24.9% against the 25% analytic ceiling (6.13% vs. 6.25% on the 16×16 variant), while dense 1×1 layers hold 97–99% on the identical RTL. Second, the predicted *geometry contrasts*: the measured lane law reproduces the $4.00\times$ systolic-vs-grid contrast to $4.057\times$, and extends the cache's $18\times$ depthwise contrast to within $1.22\times$. Crucially, the same runs show the analytical cache cycle *floors* to be $192\times$ to $2475\times$ conservative (the RTL executes the identical work that much faster), which is precisely why RTL-derived claims are scoped to these utilization *contrasts* and to the schedule replay above, never to absolute cycle agreement. The mechanism is what matters here: the utilization losses the LP models (Section 4) are the ones the real datapath exhibits.

B.1 Physical-design validation: post-synthesis area and the linearity of the cost model

Experiments 1 and 2 validate the *schedule* the program commits to. A second, independent question is whether the *cost model* that evaluates the hardware is itself faithful: the program sizes a chip through the coefficients k_{MAC} (area per MAC) and π_{MAC} (power per MAC), and every manufacturability gate and the area and power caps are expressed in them. If those coefficients are wrong, the certified designs are certified against the wrong physics. We therefore take the datapath the program sizes, write it as synthesizable RTL, and push it through a full open-source RTL-to-placement flow at an advanced predictive node.

Setup. We implement a parameterized weight-stationary INT8 systolic tile (an $N \times N$ array of processing elements, each holding a resident 8-bit weight, streaming 8-bit activations east and 32-bit partial sums south, one MAC per PE per cycle in steady state) and synthesize, floorplan, place, and timing-repair it with OpenROAD [Ajayi et al., 2019] against the ASAP7 7 nm predictive PDK [Clark et al., 2016] at a 1 GHz target, for $N \in \{16, 32, 64\}$, that is 256 to 4,096 MACs. We report post-detailed-placement extracted areas. We do not run clock-tree synthesis or routing, and we make no tape-out claim: the quantity under test is the area coefficient and its scaling law, both of which are fixed at placement.

Result 1: the linear area model is structurally correct. Table 16 reports extracted cell area at the three sizes. A linear fit gives $A_{\mathrm{cell}} = 82.08\,\mathrm{MACs} - 206.5\ \mu\mathrm{m}^2$ with $R^2 = 0.999999$, and the extracted area *per MAC* varies by only 0.46% across a $16\times$ range in MAC count. This is a stronger statement than agreement on a single number: the program assumes area is *linear* in the MAC count, with the same slope at every chip size, and that assumption is exactly what a placed datapath exhibits. The linear area row in the program is therefore not a convenient approximation but a measured property of the datapath it sizes.

Result 2: the area coefficient is anchored and conservative. The fitted slope is $8.21 \times 10^{-5}\ \mathrm{mm}^2$/MAC of standard-cell area. Placed designs occupy a core larger than their cell area; our floorplans sit at 41% placement utilization, giving $2.0 \times 10^{-4}\ \mathrm{mm}^2$/MAC of core area. The value the program uses, $k_{\mathrm{MAC}} = 1.5 \times 10^{-4}\ \mathrm{mm}^2$/MAC, lies between these two and corresponds to the cell area at 54.7% placement utilization, a typical density for a datapath of this kind. The coefficient

is thus bracketed by extracted silicon area from below and by our own conservative floorplan from above.

Result 3: the power coefficient, and why we anchor it to silicon rather than to this flow. Power at the same operating point is instructive in a different way. Reporting placement-level power on this tile with flat random switching activity yields 2.0 to 6.5 pJ per MAC operation as the assumed input activity rises from 5% to 50%, against the 0.40 pJ/MAC-op implied by π_{MAC}. Rather than adjudicate this within the flow, we test it against an external physical bound. Shipped accelerators deliver INT8 arithmetic at 0.35 pJ/op (H100, 4 nm), 0.64 pJ/op (A100, 7 nm), and 1.27 pJ/op (TPU-v4i, 7 nm), and these are *whole-chip* figures that include memory, cache, and control. Our number is for the multiplier array alone, at the same node, and exceeds an entire A100 per operation by 3.2–$10.1\times$; a bare array cannot consume more energy per operation than the complete system that contains it. The measurement, not the coefficient, is therefore out of range: with a generic synthesized multiplier, no clock gating or operand isolation, and flat random activity in place of correlated INT8 data, this flow reports an upper bound rather than a representative figure. The program's $\pi_{\mathrm{MAC}} = 0.40$ pJ/MAC-op sits where an array-only coefficient should, at the same order as and slightly below whole-chip shipped silicon, and we anchor it there (Appendix Q).

Synthesis of the three hardware verification results. Design-space-exploration frameworks are conventionally validated in three ways: against measured taped-out chips, against post-synthesis extraction, and against other frameworks [Mei et al., 2021]. The Timeloop-derived Pareto data places this work in the third category and the silicon-anchored coefficients in the first; this section supplies the second. Concretely, it converts the area side of every gate-legality claim in this paper, and of the certified designs in Section 6, from a literature-shaped constant into a quantity extracted from RTL-to-placement at 7 nm, together with a validated scaling law; and it leaves the power side anchored to shipped silicon, with an explicit account of why this particular flow overstates it.

Table 16: Post-placement extracted area of the INT8 systolic tile at ASAP7 7 nm. Area per MAC is constant to within 0.46% over a $16\times$ range in MAC count; the linear fit has $R^2 = 0.999999$.

N	MACs	Cell area (μm^2)	Cell area/MAC (mm^2)	Core area/MAC (mm^2)
16	256	20,981	8.196×10^{-5}	1.986×10^{-4}
32	1,024	83,629	8.167×10^{-5}	1.986×10^{-4}
64	4,096	336,049	8.204×10^{-5}	1.998×10^{-4}
Linear fit ($R^2 = 0.999999$)			8.208×10^{-5}	
Program value k_{MAC}			1.5×10^{-4} (cell area at 54.7% utilization)	

B.2 Measuring the compute-energy coefficients

The area result above validates k_{MAC}. The program also charges for compute *energy*, and one property of that dual-guided scoring deserves a measurement rather than an assumption. Reducing arithmetic precision is usually modelled as a reduction in bytes moved, and in an earlier form of our program precision affected memory traffic alone. That is incomplete: a narrower multiplier also switches less, so precision should reduce compute energy as well. We add that term, and because a multiplier's energy is often argued to follow its partial-product count, we could have set the ratio analytically. Instead we measured it.

Method. We synthesize and place six variants of the tile at ASAP7: three operand-width combinations (W8A8, W4A8, W4A4) crossed with two power-management levels (ungated, and clock-gated with operand isolation). Power is reported at a matched input activity so that the ratios are comparable. As a check on the flow, the W8A8 ungated variant reproduces the independently synthesized tile of Table 16 to within 0.1% despite a different source file, module name and container.

Result: the analytic ratio is wrong in both directions. Table 17 reports the measurements. The partial-product model predicts energy ratios of 0.50 for W4A8 and 0.25 for W4A4 against the W8A8 baseline. The measured ratios are 0.376 and 0.358. Narrowing the *weight* saves more than the model predicts, because switching concentrates in the multiplier tree; narrowing the *activation* saves far less, because the 32-bit accumulator and the partial-sum pipeline registers do not narrow with the

operands, so the second halving buys only a further 5%. We therefore charge compute energy with the measured ratios rather than the analytic ones. The $\mathrm{W4A16}$ entry is not directly measured and is held at a conservative 0.5 rather than extrapolated in our own favour.

A negative result on gating, and why we do not include it. The same matrix was intended to characterize a clock-clock-gating decision, and it cannot. Under a flat synthetic activity model the gated variants measure as *more* expensive than the ungated ones ($1.40\times$ the power at W8A8), which is not physical. The cause is that a flat activity assumption charges for the gating cells while never crediting the idle cycles they exist to skip; the benefit of gating is visible only under a realistic idle pattern, which requires switching activity captured from a simulation of the actual workload. The area cost of gating is measured and real ($+22\%$ at W4A8, $+20\%$ at W4A4), but without a credible power benefit to weigh against it we do not introduce the decision. We report the attempt because the failure is informative: it delimits what this class of measurement can and cannot establish.

Scope. These measurements characterize the synthesized tile; they are not substituted into the cost model used for the reported speedups. They therefore provide coefficient sensitivity rather than a new whole-chip performance result. The best measured variant consumes 1.22 pJ per MAC operation against the 0.40 implied by π_{MAC}, a factor of three rather than the order of magnitude that an ungated tile driven by random data suggested, with the residual attributable to predictive-PDK pessimism, a generic rather than optimized multiplier, and synthetic activity. We continue to anchor π_{MAC} to shipped silicon.

Table 17: Measured compute energy and area of the tile at ASAP7 7 nm ($N = 16$, 1 GHz, matched input activity). Ratios are against the W8A8 ungated variant. The analytic partial-product model predicts 0.50 and 0.25 for the W4A8 and W4A4 energy ratios; both are wrong.

Variant	Power (W)	pJ/MAC-op	Energy ratio	Area ratio
W8A8, ungated	0.872	3.41	1.000	1.001
W4A8, ungated	0.328	1.28	**0.376**	0.726
W4A4, ungated	0.312	1.22	**0.358**	0.625
W8A8, gated	1.220	4.77	1.399	–
W4A8, gated	0.347	1.36	0.398	0.887
W4A4, gated	0.312	1.22	0.358	0.752

C Discussion and Plots

C.1 What a Bound Does and Does Not Establish

The LP lower endpoint covers only its identified model. A supported integral execution gives an upper endpoint after original-row checking and replay. A small working interval does not cover omitted implementations; a wide complete interval does not invalidate a separately admitted execution. The native feature result is architectural progress with a stated reference contract. Neither LLM guidance nor a favorable numerical objective supplies a global optimality proof.

C.2 Interpreting Hardware and Software Gains

The original blueprints expose a compute-bound regime after traffic reduction. The native decode task has different service and capacity bottlenecks. Software control rows should therefore be read at their own hardware and workload: retention savings can shrink as a link becomes wider, and batching can reduce array padding without changing an external load count. Feature effects can interact; nested KV placement controls are not independent contributions.

D Extended Related Work

Architecture and mapping search. Timeloop searches mappings for a supplied architecture [Parashar et al., 2019]. CoSA uses mixed-integer optimization for a constrained mapping objective; its guarantee does not make the resulting latency globally optimal under an arbitrary simulator [Huang et al., 2021]. ZigZag expands the architecture–mapping space [Mei et al., 2021], while DOSA places architecture and mapping variables in one differentiable program [Hong et al., 2023]. Gemini couples spatial mappings and architectural choices for multi-chiplet accelerators [Cai et al., 2024]. CODEBench uses multi-objective exploration over accelerator configurations [Tuli et al., 2023]. FAST combines hardware, mapping, and compiler search, including a fusion ILP [Zhang et al., 2022]. These methods motivate the controls in our original campaign. CHEETAH retains alternatives until shared storage, implementation cost, and hardware provisioning can be evaluated together.

Fusion, placement, and rematerialization. DeFiNES includes depth-first fused execution, caching, and overlap recomputation; its restricted placement and recomputation cells in Table 1 are therefore not marked absent [Mei et al., 2023]. Fast-and-Fusiest searches legal operation orders as well as fusion and mappings under its stated cost model [Andrulis et al., 2026]. These are substantial joint compiler capabilities, even though they do not provide CHEETAH's complete temporal state model and variable hardware domain. Checkmate formulates training checkpointing as an integer program [Jain et al., 2020]; POET combines rematerialization and paging [Patil et al., 2022]. CHEETAH uses paid recomputation, transfer completion, and readiness to represent these tradeoffs for inference. Representability alone is not a measured gain; our native controls exercise only the features identified in Section 6.6.

Polyhedral transformations. Pluto constructs affine schedules for parallelism and locality [Bondhugula et al., 2008]. Tiramisu exposes a richer programmable scheduling language, including storage and recomputation transformations [Baghdadi et al., 2019]. Their user-programmed capabilities should not be confused with an automatic joint search over every table axis. Conversely, CHEETAH's causal event choices should not be described as arbitrary loop transformation coverage: its implementation library and dependency contract bound the represented schedules.

Relaxations and first-order LP solvers. McCormick envelopes bound bilinear products over boxes [McCormick, 1976]; exactness for a single integral-factor product does not prove exactness of an entire fractional co-design program. PDLP demonstrates practical large-scale linear programming with first-order methods [Applegate et al., 2022]. GPU variants include cuPDLP.jl, cuPDLP-C, and cuPDLPx [Lu and Yang, 2024, Lu et al., 2024, 2025]. Our qualified OCS path uses centered Halpern PDHG, fixed diagonal scaling, structured forward/adjoint operators, and independent certificate checking. These mechanisms address memory and iteration cost; they do not imply that full-library pricing or global master optimality is complete.

LLM-guided search. FunSearch and AlphaEvolve combine model proposals with deterministic evaluation [Romera-Paredes et al., 2024, Novikov et al., 2025]. CHEETAH similarly uses an LLM to propose search interventions, with resource prices and failed-trial diagnostics as context. The original campaign edits Vizier search neighborhoods; current master experiments can also propose implementations and hardware alternatives directly. A proposal is admitted only after its numerical, dependency, and resource checks. The proposal mechanism is not an exhaustive separation oracle.

E Modeling Properties and Containment

The following statements concern represented feasible sets. They do not imply that a numerical solver has optimized the complete master or that arbitrary rounding preserves feasibility.

E.1 Bounded Products and Epigraphs

Proposition 1. *For $0 \le y, p \le 1$, the inequalities $0 \le w$, $w \le y$, $w \le p$, and $w \ge y + p - 1$ describe the convex hull of the graph $w = yp$. If either factor is integral, these inequalities force $w = yp$.*

Proof. The lower envelope is $\max(0, y + p - 1)$ and the upper envelope is $\min(y, p)$ [McCormick, 1976]. For $y = 0$ they both force $w = 0$; for $y = 1$ they force $w = p$. The cases for integral p are symmetric. □

This is an individual-product property, not a convex-hull theorem for the complete program. For example,

$$y_1 = y_2 = p = \tfrac{1}{2}, \quad w_1 = w_2 = \tfrac{1}{2} \tag{7}$$

satisfies the separate envelopes and $y_1 + y_2 = 1$, but violates the identity $w_1 + w_2 = p$ that holds for exact products. Linking rows

$$\sum_c w_{ick} = p_{\mathrm{assoc}(i,k)} \tag{8}$$

can strengthen a branch when its intended integral semantics justify them. They must not be added to an immutable compatibility branch while claiming containment of every original fractional point.

Corollary 1 (Disaggregated service epigraphs)**.** *For fixed resource costs a_c, b_c, the one-choice epigraph has the perspective form $\sum_c y_c = 1$, $y_c \ge 0$, $t_c \ge a_c y_c$, $t_c \ge b_c y_c$, $T \ge \sum_c t_c$. It gives $T \ge \sum_c y_c \max(a_c, b_c)$ and dominates $T \ge \max(\sum_c a_c y_c, \sum_c b_c y_c)$.*

Proof. For each c, minimizing t_c gives $y_c \max(a_c, b_c)$. Summing maxima is at least either sum separately. With a one-hot choice the two formulations coincide. For shared variable hardware, every additional coupling must also be disaggregated consistently; separate product envelopes alone do not prove the convex hull of the entire HW/SW problem. □

E.2 An Objective-Preserving Embedding

Definition 1. *An included schedule branch is a complete, identified program with its own workload, cost coefficients, hardware and boundary convention. A sequential control fixes its tile choices and other schedule decisions within that same contract. A method using different coefficients or unsupported hardware is not included merely because it has a similar pipeline.*

Write each included branch as $P_b = \{x : E_b x = e_b,\ G_b x \le g_b,\ \ell_b \le x \le u_b\}$. The master homogenizes all its rows and finite boxes with λ_b.

Theorem 1 (Containment of every included branch)**.** *For every included branch p and every $x \in P_p$, the assignment $\lambda_p = 1$, $x_p = x$, and $\lambda_b = 0, x_b = 0$ for $b \ne p$ is master-feasible, with the same objective and hardware readout.*

Proof. Every active-branch row and bound becomes its original constraint. Every inactive-branch row has zero left and right sides; zero satisfies each homogenized finite box. The branch weights sum to one, active hardware coordinates are unchanged, and $\sum_b c_b^T x_b = c_p^T x$. The argument applies to fractional points as well as integral schedules. □

Corollary 2. $\inf_{z \in P_M} c_M^T z \le \min_b \inf_{x \in P_b} c_b^T x$. *A fixed-tile sequential control represented within an included branch therefore has a master-feasible embedding at the same modeled cost.*

Remark. Containment is not automatic reproduction of every published chip: the actual branch and its complete contract must be imported. It also does not establish that the causal block alone contains the old fractional schedule block, or that a recovery heuristic returns a no-worse integral design. Retaining the old admitted execution as an eligible incumbent supplies the latter fallback under matched semantics. Unbounded branches can retain recession directions at zero weight; we claim the embedding above, not an unconditional closed-convex-hull identity.

E.3 Structure and Safe Pruning

Proposition 2. *For fixed graph incidence, variable domains, and candidate library, changes only to numerical costs and bounds preserve the structural row/column pattern.*

Proof. The same indexed row templates address the same coordinates. Hardware-dependent coefficients can change their values; changing the implementation library or introducing different lifetimes need not preserve the pattern. □

This permits reuse of indexing and warm starts. A new scaling or norm bound must still be checked when coefficients change. Pareto pruning similarly needs componentwise dominance in every resource and interface used by the branch; a two-axis latency plot alone is insufficient when workspace, traffic tiers, or numerical formats differ.

F Linear Program Formulation Details

F.1 Relaxation and Integral Recovery

The master is a linear relaxation of a finite implementation and hardware library. Its canonical form is

$$\min_x c^T x \qquad \text{subject to} \qquad \ell_c \le Ax \le u_c, \quad \ell_v \le x \le u_v. \tag{9}$$

McCormick envelopes and epigraphs retain linearity, but do not guarantee a small integrality gap. Integral recovery selects a concrete hardware and implementation assignment and checks the resulting schedule. Nearest-integer rounding alone is not a feasibility proof. Appendix O gives the explicit retained joint schedule rows and the expanded causal rows.

F.2 Original Campaign Matrix Sizes

The original controls use different encodings of restricted decision spaces. The fixed-tile temporal control stores separate input/output/weight placement and recomputation trajectories, giving approximately $4LT$ coordinates. The joint-tile control uses edge-indexed activation placement, weight placement, and recomputation, plus $O(LC)$ selection/product coordinates. For a mostly linear graph and $T = L$, its count is approximately $3L^2 + 4LC$. The joint control can therefore be smaller despite allowing more tile choices. These archived matrix counts describe the original campaign, not the current native source domain or the OCS supports in Appendix P.

Table 18: Single Workload per Chip LP matrix sizes ($n_{\text{constraints}} \times n_{\text{vars}}$, nnz) per model and formulation.

Model	Static	fixed-tile Dynamic	joint-tile Joint
EfficientNet-B1	803 × 576, nnz= 15,060	59,571 × 53,131, nnz= 119,485	62,728 × 47,035, nnz= 141,604
EfficientNet-B2	803 × 576, nnz= 15,060	59,571 × 53,131, nnz= 119,485	62,728 × 47,035, nnz= 141,604
EfficientNet-B3	908 × 651, nnz= 18,975	76,116 × 67,861, nnz= 152,620	77,743 × 59,020, nnz= 173,749
EfficientNet-B4	1,118 × 801, nnz= 28,155	115,281 × 102,721, nnz= 231,040	112,498 × 87,040, nnz= 247,489
EfficientNet-B5	1,356 × 971, nnz= 40,735	169,460 × 150,933, nnz= 339,500	159,503 × 125,324, nnz= 346,293
EfficientNet-B6	1,566 × 1,121, nnz= 53,755	225,905 × 201,153, nnz= 452,480	207,698 × 164,864, nnz= 446,913
EfficientNet-B7	1,909 × 1,366, nnz= 78,892	335,518 × 298,663, nnz= 671,853	299,965 × 241,059, nnz= 638,356
Llama-1 13B	2,518 × 1,801, nnz= 135,355	583,381 × 519,121, nnz= 1,167,840	505,198 × 411,840, nnz= 1,061,089
Llama-1 65B	5,038 × 3,601, nnz= 529,915	2,333,161 × 2,075,041, nnz= 4,668,480	1,917,658 × 1,601,280, nnz= 3,936,769
Llama-3 70B	5,038 × 3,601, nnz= 529,915	2,333,161 × 2,075,041, nnz= 4,668,480	1,917,658 × 1,601,280, nnz= 3,936,769
DeepSeek-V3	6,319 × 4,516, nnz= 829,852	3,669,793 × 3,263,443, nnz= 7,342,293	2,983,450 × 2,504,019, nnz= 6,094,156

Table 19: GeoMean Avg Maximum LP matrix size per model cluster (largest member LP).

Cluster	Static	fixed-tile Dynamic	joint-tile Joint
EfficientNet	1,909 × 1,366, nnz= 78,892	335,518 × 298,663, nnz= 671,853	299,965 × 241,059, nnz= 638,356
Llama	5,038 × 3,601, nnz= 529,915	2,333,161 × 2,075,041, nnz= 4,668,480	1,917,658 × 1,601,280, nnz= 3,936,769
Modern LLM	6,319 × 4,516, nnz= 829,852	3,669,793 × 3,263,443, nnz= 7,342,293	2,983,450 × 2,504,019, nnz= 6,094,156

F.3 LP Matrix Structure

Table 20 records the two controls' sparse patterns. Fixed incidence and menus permit index reuse. Numerical coefficient changes can still require new scaling and norm checks, and a changed library or dependency graph can change the pattern itself.

F.4 Current Solver Path

These matrix sizes describe the original restricted workloads, not the full native temporal expansion. Section 5 describes the OCS path; Appendix P lists accepted native working supports. No universal solver superiority or full-master convergence follows from these smaller cases.

Table 20: Structural comparison of the original fixed-tile and joint-tile controls. For Llama3-70b: $L = 720, T = 720, E \approx L$.

Property	fixed-tile control	CHEETAH
Variables	$T_i\ (L)$ + $p^I, p^O, p^W, r\ (4 \cdot L \cdot T)$ + $T_{\text{final}}\ (1)$ + $FT_i\ (L)\ \approx\ 4L^2 + 2L + 1$	$T_i\ (L)$ + $y_{i,c}\ (LC)$ + $p_{e,t}\ (ET)$ + $p^W, r\ (2LT)$ + $w_{i,c,k}\ (3LC)\ \approx\ 3L^2 + 4LC + L$
Llama-3 70B actual	L=720, T=720 $\rightarrow$ 2.07M vars, 2.33M constraints, 4.67M nnz	Similar order of magnitude; $p_{e,t}$ is edge-indexed (ET) instead of layer-pair-indexed, which saves $\approx L^2$ vars when DAG is mostly linear
Density	$\text{nnz}/(m \cdot n) \approx 9.6 \times 10^{-7}$ ($\approx$1 nnz per million entries)	Similar; row group (5) is the most dense
Avg. nnz/row	≈ 2.0	≈ 2.0 with bursts up to $4L + L$ on capacity rows
Constant pattern across runs	Yes, parametric in L. Every workload of L layers gets the exact same matrix template; only coefficient values (`T_max`, `T_min`, `d_I/O/W`) change.	Yes, parametric in (L, C, E, T). DAG topology lives in the (i, e, t) index map, but row template is invariant. Sparsity pattern is workload-agnostic.
Banded	Most rows touch only (i, t) and $(i, t{-}1)$ (persistence chains, weight monotone, FT recurrences). Bandwidth $\approx T$ along the time axis.	McCormick triples (4a/b/c) are 1–2 nnz; persistence on edge (6) couples (e, t) to $(e, t{-}1)$. Same time-band structure.
Exception to band-ness	Rows from group (C): global memory capacity per timestep span all L weights at that timestep $\rightarrow$ wide $\approx L{+}2$ nnz rows, T of them.	Rows from group (5): span all E edges + all L weight footprints at that timestep $\rightarrow\ \approx E{+}L \approx 2L$ nnz rows, T of them.
Why constant pattern works for solver	Per-row degree distribution is bimodal: $\mathcal{O}(L^2)$ rows with $\leq$3 nnz + $\mathcal{O}(T)$ fat capacity rows. Our solver handles this after preconditioning.	Same bimodal pattern, plus a small $(3 \cdot L \cdot C)$ dense McCormick block. Still sparse in aggregate.

G Designing for Manufacturability and Feasibility Audits

This appendix preserves the hardware audits and validation examples from the original campaign. Its RTL tapes, Timeloop mappings, and physical proxy gates retain that campaign's workload and hardware scope. The native full-inference extension has separate execution and service contracts in Appendix P; these earlier audits do not establish its memory-interface timing or physical implementation.

G.1 Five Levels of Validation

The original campaign uses five levels of modeled feasibility validation.

1. Roofline Auditor: A deterministic Roofline Auditor checks physical feasibility by comparing each LP-predicted latency against the lower bound

$$\mathrm{LB} = \max(t_{\mathrm{compute}},\ t_{\mathrm{memory}})\,, \tag{10}$$

   where t_{compute} is total FLOPs divided by peak FLOPs/s and t_{memory} is post-placement off-chip traffic divided by available memory bandwidth. The audit additionally reports $\mathrm{MFU}_{\mathrm{bound}}$ and memory utilization; values above 1 would indicate a physically impossible prediction. The original campaign's audited results have zero roofline lower-bound violations; Appendix G.2 gives the detailed checks.
2. Campaign Validator: A deterministic Campaign Validator legalizes LLM-proposed architectural neighbourhoods by enforcing hardware ranges, power-of-two constraints, TDP and area caps, MAC caps, CGM feasibility floors, and compute/bandwidth coupling constraints before Vizier is permitted to sample candidates.
3. Causal Ledger: The Causal Ledger records each LLM hypothesis, intervention, and observed outcome, making the search loop fully auditable and preventing repeated exploration of empirically invalid regions.
4. Solver diagnostics: The LP solver reports mathematical solver diagnostics such as primal and dual residuals, duality gap, rounding gap, and fractionality. In our experiments LP solutions are not treated as trustworthy unless both the relaxation and the rounded design are numerically well behaved.
5. Pre-RTL Feasibility Gates: Original-campaign designs pass six first-order screens for pin count, power density, logic depth, and related resource costs. Appendix G.3 provides their definitions. Passing these proxies does not constitute physical-design sign-off.

G.2 MFU and Latency Audits

Clarification on MFU In this work, $\mathrm{MFU} = \texttt{compute_floor}/\texttt{obj_cycles}$ is a *roofline tightness ratio*, not a measured hardware utilization. $\mathrm{MFU} = 1.000$ means the LP objective equals the ideal compute lower bound; it does not mean a real chip would achieve 100% utilization. This is since the searched accelerators are hypothetical datapaths rather than fabricated chips or RTL implementations, we cannot report measured hardware MFU; instead, we report MFU_bound only as a roofline lower-bound consistency check, where values above 1 would indicate an impossible prediction.

We define the idealized roofline saturation metric as:

$$\mathrm{MFU}_{\mathrm{audit}} = \frac{t_{\mathrm{compute}}}{t_{\mathrm{solver}}}, \tag{11}$$

where t_{compute} is the theoretical compute lower bound and t_{solver} is the LP-predicted latency. Thus, $\mathrm{MFU}_{\mathrm{audit}} = 1$ means the LP solution is *compute-floor tight* under the idealized accelerator model. It is not a claim that a real implementation would achieve 100% hardware utilization. Real chips would incur additional losses from pipeline bubbles, NoC contention, SRAM latency, control overhead, imperfect scheduling, and compiler/runtime effects.

The `binding` column reports whether the objective is sitting on the compute floor, the memory floor, or above both floors. The key sanity condition is that no row has $\mathrm{MFU}_{\mathrm{audit}} > 1$ and no fixed-tile/joint-tile row is unexpectedly loose above the lower bound. Values greater than 1 are physically

impossible and would indicate a bug or cost-model violation. Values equal to 1 are acceptable: they indicate that the idealized LP solution is exactly on the compute lower bound.

A true measured MFU would require an actual executable implementation of the selected hardware/schedule pair: either fabricated silicon, an FPGA/RTL prototype, or a calibrated cycle-accurate simulator with the selected joint-tile tiling/fusion/placement schedule implemented. One would need to run the workload and collect hardware counters such as active MAC cycles, issued FLOPs, memory stalls, NoC stalls, SRAM bank conflicts, pipeline bubbles, and total elapsed cycles. Measured MFU would then be:

$$\mathrm{MFU}_{\mathrm{measured}} = \frac{\text{useful FLOPs}}{\text{elapsed cycles} \times \text{peak FLOPs/cycle}}. \tag{12}$$

We define:

$$\mathrm{MFU}_{\mathrm{bound}} = \frac{t_{\mathrm{compute}}}{t_{\mathrm{solver}}}, \tag{13}$$

where t_{compute} is the theoretical compute lower-bound latency and t_{solver} is the LP-predicted latency. The roofline audit additionally checks the stronger lower bound:

$$t_{\mathrm{solver}} \;\geq\; \max(t_{\mathrm{compute}},\; t_{\mathrm{memory}})\,, \tag{14}$$

where the memory floor is total bytes moved divided by peak bandwidth. Concretely, the audit enforces `roof_ratio` $= \mathrm{LB}/\mathrm{obj} \leq 1.001$ as the pass criterion.

Interpretation of $\mathrm{MFU}_{\mathrm{bound}} = 1$**.** Since the searched accelerators are hypothetical datapaths evaluated by CORGIsolve rather than fabricated chips, RTL models, or calibrated cycle-accurate simulators, real measured MFU is unavailable. We therefore use $\mathrm{MFU}_{\mathrm{bound}} = t_{\mathrm{compute}}/t_{\mathrm{solver}}$ solely as a roofline consistency check. $\mathrm{MFU}_{\mathrm{bound}} = 1$ means the LP solution is *compute-floor tight* under the idealized accelerator model; it does not imply that real silicon would achieve 100% utilization.

What the roofline audit proves. The audit does not establish that real-world MFU would equal 1. It establishes the more basic and necessary condition that the reported LP latency is not physically impossible: no result is faster than either the compute or memory lower bound. In a real implementation, utilization would generally be lower due to pipeline bubbles, NoC contention, SRAM banking conflicts, control overhead, compiler effects, and imperfect scheduling.

Since our accelerators are simulated datapaths rather than fabricated chips, real measured MFU is unavailable. We use $\mathrm{MFU}_{\mathrm{bound}} = t_{\mathrm{compute}}/t_{\mathrm{solver}}$ only as a roofline consistency check. Values above 1, or LP latencies below $\max(t_{\mathrm{compute}}, t_{\mathrm{memory}})$, would indicate a physically impossible prediction. Values equal to 1 mean the LP solution is tight to the ideal compute floor, not that real hardware would achieve 100% utilization; real implementations would generally fall below this bound due to microarchitectural overheads. Our audit reports 31 plotted cells, 2 skipped infeasible cells, and 0 MFU or roofline violations, supporting that the reported fixed-tile/joint-tile speedups are physically feasible upper-bound estimates rather than below-roofline artifacts.

The below audit verifies that all reported fixed-tile/joint-tile results are roofline-feasible: the LP-predicted latency never falls below the compute or memory lower bound, so the speedups are not artifacts of violating basic physical throughput limits.

Table 21: Single Workloads of Vizier-discovered hardware configuration. $\mathrm{MFU}_{\mathrm{audit}} = t_{\mathrm{compute}}/t_{\mathrm{solver}}$ is a roofline tightness ratio (see Eq. 11), not a measured hardware utilization. $\mathrm{MFU} = 1.000$ means the LP hit the theoretical compute floor. *Compute-bound*: $\mathrm{obj} \approx t_{\mathrm{compute}}$. *Memory-bound*: $\mathrm{obj} \approx t_{\mathrm{memory}}$, $\mathrm{MFU} < 1$. *Above-roofline*: obj exceeds both floors; expected for Static on hard cells, unexpected for fixed-tile/joint-tile. Two cells (Llama-3 70B and DeepSeek-V3 under Static) had no physically valid trial and are marked infeasible.

Method	Model	MACs	BW	Speedup	Obj cycles	Comp. floor	Mem. floor	MFU	obj/lb	Binding	Verdict
Static FAST ILP											
Static	EN-B1	32,768	64	1.12	2.757×10^7	2.757×10^7	4.236×10^6	1.000	1.000	compute-bound	OK
Static	EN-B2	65,536	64	1.93	2.576×10^7	2.576×10^7	5.474×10^6	1.000	1.000	compute-bound	OK
Static	EN-B3	32,768	64	1.25	7.905×10^7	7.905×10^7	7.552×10^6	1.000	1.000	compute-bound	OK
Static	EN-B4	65,536	64	2.25	1.045×10^8	1.045×10^8	1.372×10^7	1.000	1.000	compute-bound	OK
Static	EN-B5	32,768	64	1.31	4.938×10^8	4.938×10^8	2.279×10^7	1.000	1.000	compute-bound	OK
Static	EN-B6	32,768	64	1.10	9.952×10^8	9.952×10^8	3.505×10^7	1.000	1.000	compute-bound	OK
Static	EN-B7	131,072	64	3.60	5.153×10^8	5.153×10^8	5.280×10^7	1.000	1.000	compute-bound	OK
Static	Llama-1 13B	32,768	1024	1.07	2.705×10^{10}	2.705×10^{10}	2.981×10^7	1.000	1.000	compute-bound	OK
Static	Llama-1 65B	65,536	128	0.61	2.377×10^{11}	6.744×10^{10}	9.514×10^8	0.284	3.525	above-roofline	loose (expected)
Static	Llama-3 70B	–	–	–	–	–	–	–	–	–	infeas
Static	DeepSeek-V3	–	–	–	–	–	–	–	–	–	infeas
Dynamic fixed-tile LP (time-indexed + rematerialisation)											
fixed-tile	EN-B1	65,536	64	2.24	1.379×10^7	1.379×10^7	4.236×10^6	1.000	1.000	compute-bound	OK
fixed-tile	EN-B2	131,072	64	3.87	1.288×10^7	1.288×10^7	5.474×10^6	1.000	1.000	compute-bound	OK
fixed-tile	EN-B3	131,072	64	4.99	1.976×10^7	1.976×10^7	7.552×10^6	1.000	1.000	compute-bound	OK
fixed-tile	EN-B4	131,072	64	4.51	5.225×10^7	5.225×10^7	1.372×10^7	1.000	1.000	compute-bound	OK
fixed-tile	EN-B5	131,072	64	5.24	1.234×10^8	1.234×10^8	2.279×10^7	1.000	1.000	compute-bound	OK
fixed-tile	EN-B6	262,144	64	8.82	1.244×10^8	1.244×10^8	3.505×10^7	1.000	1.000	compute-bound	OK
fixed-tile	EN-B7	524,288	64	14.39	1.288×10^8	1.288×10^8	5.280×10^7	1.000	1.000	compute-bound	OK
fixed-tile	Llama-1 13B	262,144	64	8.57	3.382×10^9	3.382×10^9	4.770×10^8	1.000	1.000	compute-bound	OK
fixed-tile	Llama-1 65B	262,144	64	8.57	1.686×10^{10}	1.686×10^{10}	1.903×10^9	1.000	1.000	compute-bound	OK
fixed-tile	Llama-3 70B	524,288	64	20.07	9.227×10^9	9.227×10^9	1.972×10^9	1.000	1.000	compute-bound	OK
fixed-tile	DeepSeek-V3	262,144	128	8.87	6.622×10^9	6.622×10^9	7.477×10^8	1.000	1.000	compute-bound	OK
Joint joint-tile LP (joint tiling + fusion)											
joint-tile	EN-B1	131,072	64	4.47	6.894×10^6	6.894×10^6	4.236×10^6	1.000	1.000	compute-bound	OK
joint-tile	EN-B2	262,144	64	7.73	6.440×10^6	6.440×10^6	5.474×10^6	1.000	1.000	compute-bound	OK
joint-tile	EN-B3	262,144	64	9.99	9.881×10^6	9.881×10^6	7.552×10^6	1.000	1.000	compute-bound	OK
joint-tile	EN-B4	262,144	64	9.01	2.612×10^7	2.612×10^7	1.372×10^7	1.000	1.000	compute-bound	OK
joint-tile	EN-B5	524,288	64	20.98	3.086×10^7	3.086×10^7	2.279×10^7	1.000	1.000	compute-bound	OK
joint-tile	EN-B6	524,288	64	17.64	6.220×10^7	6.220×10^7	3.505×10^7	1.000	1.000	compute-bound	OK
joint-tile	EN-B7	1,048,576	64	28.78	6.441×10^7	6.441×10^7	5.280×10^7	1.000	1.000	compute-bound	OK
joint-tile	Llama-1 13B	1,048,576	64	34.27	8.454×10^8	8.454×10^8	4.770×10^8	1.000	1.000	compute-bound	OK
joint-tile	Llama-1 65B	1,048,576	64	34.28	4.215×10^9	4.215×10^9	1.903×10^9	1.000	1.000	compute-bound	OK
joint-tile	Llama-3 70B	1,048,576	64	40.14	4.614×10^9	4.614×10^9	1.972×10^9	1.000	1.000	compute-bound	OK
joint-tile	DeepSeek-V3	1,048,576	64	35.50	1.656×10^9	1.656×10^9	1.495×10^9	1.000	1.000	compute-bound	OK
Summary: 31 cells plotted (MFU $\leq$ 1, roofline-feasible) · 2 cells skipped (no physically valid trial) · 0 MFU > 1 regressions · 0 fixed-tile/joint-tile loose rows.											

Table 22: Roofline lower-bound audit (compute floor + memory-BW floor). Lower bound: $\mathrm{LB} = \max\left(\frac{\text{total FLOPs}}{2\times\text{MACs}},\ \frac{\text{total bytes}}{\text{BW}}\right)$. Memory floor: bytes moved (HBM $\leftrightarrow$ PEs, all activations + weights) divided by BW (GB/s), converted to cycles at 1 GHz clock. Compute floor: total FLOPs $/(2 \times$ total MACs$)$ in cycles. The LP objective (obj cycles) must satisfy obj $\geq$ LB. `roof_ratio` $= \mathrm{LB}/\mathrm{obj} \leq 1.001$ is the pass criterion. Summary: 31 cells plotted · 2 skipped (no physically valid trial) · 0 violations.

Method	Model	MACs	BW	FLOPs	Bytes	Comp. LB	Mem. LB	Roofline LB	Obj	Roof ratio	Bottleneck	Verdict
Static FAST ILP												
Static	EN-B1	32,768	64	1.81×10^{12}	2.71×10^{8}	2.76×10^{7}	4.24×10^{6}	2.76×10^{7}	2.76×10^{7}	1.000	compute	OK
Static	EN-B2	65,536	64	3.38×10^{12}	3.50×10^{8}	2.58×10^{7}	5.47×10^{6}	2.58×10^{7}	2.58×10^{7}	1.000	compute	OK
Static	EN-B3	32,768	64	5.18×10^{12}	4.83×10^{8}	7.91×10^{7}	7.55×10^{6}	7.91×10^{7}	7.91×10^{7}	1.000	compute	OK
Static	EN-B4	65,536	64	1.37×10^{13}	8.78×10^{8}	1.04×10^{8}	1.37×10^{7}	1.04×10^{8}	1.04×10^{8}	1.000	compute	OK
Static	EN-B5	32,768	64	3.24×10^{13}	1.46×10^{9}	4.94×10^{8}	2.28×10^{7}	4.94×10^{8}	4.94×10^{8}	1.000	compute	OK
Static	EN-B6	32,768	64	6.52×10^{13}	2.24×10^{9}	9.95×10^{8}	3.50×10^{7}	9.95×10^{8}	9.95×10^{8}	1.000	compute	OK
Static	EN-B7	131,072	64	1.35×10^{14}	3.38×10^{9}	5.15×10^{8}	5.28×10^{7}	5.15×10^{8}	5.15×10^{8}	1.000	compute	OK
Static	Llama-1 13B	32,768	1024	1.77×10^{15}	3.05×10^{10}	2.71×10^{10}	2.98×10^{7}	2.71×10^{10}	2.71×10^{10}	1.000	compute	OK
Static	Llama-1 65B	65,536	128	8.84×10^{15}	1.22×10^{11}	6.74×10^{10}	9.51×10^{8}	6.74×10^{10}	2.38×10^{11}	0.284	compute	OK
Dynamic fixed-tile LP (time-indexed + rematerialisation)												
fixed-tile	EN-B1	65,536	64	1.81×10^{12}	2.71×10^{8}	1.38×10^{7}	4.24×10^{6}	1.38×10^{7}	1.38×10^{7}	1.000	compute	OK
fixed-tile	EN-B2	131,072	64	3.38×10^{12}	3.50×10^{8}	1.29×10^{7}	5.47×10^{6}	1.29×10^{7}	1.29×10^{7}	1.000	compute	OK
fixed-tile	EN-B3	131,072	64	5.18×10^{12}	4.83×10^{8}	1.98×10^{7}	7.55×10^{6}	1.98×10^{7}	1.98×10^{7}	1.000	compute	OK
fixed-tile	EN-B4	131,072	64	1.37×10^{13}	8.78×10^{8}	5.22×10^{7}	1.37×10^{7}	5.22×10^{7}	5.22×10^{7}	1.000	compute	OK
fixed-tile	EN-B5	131,072	64	3.24×10^{13}	1.46×10^{9}	1.23×10^{8}	2.28×10^{7}	1.23×10^{8}	1.23×10^{8}	1.000	compute	OK
fixed-tile	EN-B6	262,144	64	6.52×10^{13}	2.24×10^{9}	1.24×10^{8}	3.50×10^{7}	1.24×10^{8}	1.24×10^{8}	1.000	compute	OK
fixed-tile	EN-B7	524,288	64	1.35×10^{14}	3.38×10^{9}	1.29×10^{8}	5.28×10^{7}	1.29×10^{8}	1.29×10^{8}	1.000	compute	OK
fixed-tile	Llama-1 13B	262,144	64	1.77×10^{15}	3.05×10^{10}	3.38×10^{9}	4.77×10^{8}	3.38×10^{9}	3.38×10^{9}	1.000	compute	OK
fixed-tile	Llama-1 65B	262,144	64	8.84×10^{15}	1.22×10^{11}	1.69×10^{10}	1.90×10^{9}	1.69×10^{10}	1.69×10^{10}	1.000	compute	OK
fixed-tile	Llama-3 70B	524,288	64	9.68×10^{15}	1.26×10^{11}	9.23×10^{9}	1.97×10^{9}	9.23×10^{9}	9.23×10^{9}	1.000	compute	OK
fixed-tile	DeepSeek-V3	262,144	128	3.47×10^{15}	9.57×10^{10}	6.62×10^{9}	7.48×10^{8}	6.62×10^{9}	6.62×10^{9}	1.000	compute	OK
Joint joint-tile LP (joint tiling + fusion)												
joint-tile	EN-B1	131,072	64	1.81×10^{12}	2.71×10^{8}	6.89×10^{6}	4.24×10^{6}	6.89×10^{6}	6.89×10^{6}	1.000	compute	OK
joint-tile	EN-B2	262,144	64	3.38×10^{12}	3.50×10^{8}	6.44×10^{6}	5.47×10^{6}	6.44×10^{6}	6.44×10^{6}	1.000	compute	OK
joint-tile	EN-B3	262,144	64	5.18×10^{12}	4.83×10^{8}	9.88×10^{6}	7.55×10^{6}	9.88×10^{6}	9.88×10^{6}	1.000	compute	OK
joint-tile	EN-B4	262,144	64	1.37×10^{13}	8.78×10^{8}	2.61×10^{7}	1.37×10^{7}	2.61×10^{7}	2.61×10^{7}	1.000	compute	OK
joint-tile	EN-B5	524,288	64	3.24×10^{13}	1.46×10^{9}	3.09×10^{7}	2.28×10^{7}	3.09×10^{7}	3.09×10^{7}	1.000	compute	OK
joint-tile	EN-B6	524,288	64	6.52×10^{13}	2.24×10^{9}	6.22×10^{7}	3.50×10^{7}	6.22×10^{7}	6.22×10^{7}	1.000	compute	OK
joint-tile	EN-B7	1,048,576	64	1.35×10^{14}	3.38×10^{9}	6.44×10^{7}	5.28×10^{7}	6.44×10^{7}	6.44×10^{7}	1.000	compute	OK
joint-tile	Llama-1 13B	1,048,576	64	1.77×10^{15}	3.05×10^{10}	8.45×10^{8}	4.77×10^{8}	8.45×10^{8}	8.45×10^{8}	1.000	compute	OK
joint-tile	Llama-1 65B	1,048,576	64	8.84×10^{15}	1.22×10^{11}	4.22×10^{9}	1.90×10^{9}	4.22×10^{9}	4.22×10^{9}	1.000	compute	OK
joint-tile	Llama-3 70B	1,048,576	64	9.68×10^{15}	1.26×10^{11}	4.61×10^{9}	1.97×10^{9}	4.61×10^{9}	4.61×10^{9}	1.000	compute	OK
joint-tile	DeepSeek-V3	1,048,576	64	3.47×10^{15}	9.57×10^{10}	1.66×10^{9}	1.50×10^{9}	1.66×10^{9}	1.66×10^{9}	1.000	compute	OK

G.3 Six Physical-Design Feasibility Gates

The original campaign applies six physical-design proxy gates to its candidate accelerators. The gate thresholds are calibrated end-to-end to reproduce TPU-v3's published headline numbers ($700\,\mathrm{mm}^2$ die, $250\,\mathrm{W}$ TDP, $0.36\,\mathrm{W/mm}^2$ density at $65{,}536$ MACs / $900\,\mathrm{GB/s}$ / $32\,\mathrm{MiB}$ CGM) when fed TPU-v3 inputs. Therefore, they are not arbitrary but pinned to a known-quality silicon.

(1) Pin count. Converts off-chip bandwidth into HBM3-class signal wires at $6.4\,\mathrm{Gb/s/pin}$ ($0.8\,\mathrm{GB/s/pin}$) and rejects designs needing more than $4{,}096$ high-speed pins, which is the practical ceiling for four HBM3 stacks on a single package. This prevents "free-bandwidth" LP solutions that silently assume 8+ stacks.

(2) Reticle area. Evaluates the linear area model

$$A_{\mathrm{die}} \;=\; \mathrm{MACs} \times 1.5 \times 10^{-4} \;+\; \mathrm{CGM_MB} \times 1.5 \;+\; \mathrm{dram_channels} \times 8 \;+\; 50 \quad [\mathrm{mm}^2]$$

(calibrated to a 7 nm process) and rejects any die exceeding $600\,\mathrm{mm}^2$. TSMC's N7 hard reticle is $858\,\mathrm{mm}^2$, and our design choice of 600 leaves headroom for yield, redundancy rings, and IO. This prevents over-scaled compute that would otherwise require monolithic chiplet packaging or multi-die exposure.

(3) Logic depth. Estimates the combinational gate-level depth of a single MAC-tree as

$$D \;=\; \lceil \log_2(\mathrm{sys_x} \times \mathrm{sys_y} \times \mathrm{MAC_bits}) \rceil + 8$$

and rejects depths above 35 levels. Beyond $\sim$35 levels, the critical path cannot meet a $1\,\mathrm{GHz}$ clock at TSMC N7 nominal V_{dd} without aggressive pipelining we do not model. This prevents designs whose systolic array is too wide for one cycle.

(4) Crossbar / NoC routing. Computes the cell count of the PE-to-SRAM interconnect as

$$N_{\mathrm{NoC}} \;=\; n_{\mathrm{PEs}} \times n_{64\,\mathrm{KB\ banks}}$$

and rejects values above 5×10^6 cells. The cap is the empirical budget for a 2-level hierarchical NoC (cluster-local crossbar + inter-cluster mesh) at modern AI-accelerator scales: flat all-to-all crossbars cap around 10^6 cells before routing dominates the floorplan, and we use the hierarchical-NoC cap with industry precedent (TPU-v4 sparse-core mesh, H100 SM-cluster L2 partitions, MI300X chiplet-local + XGMI, Cerebras WSE-3 per-tile fabric).

(5) Power density. Computes dynamic power as

$$P_{\mathrm{dyn}} \;=\; \mathrm{total_MACs} \times 1\,\mathrm{GHz} \times 0.5_{\mathrm{util}} \times 0.4\,\mathrm{mW/MAC}$$

(the $0.4\,\mathrm{mW/MAC}$ constant is calibrated against TPU-v3's $250\,\mathrm{W}$ envelope, and bumps to $1.2\,\mathrm{mW}$ for bf16), divides by the same linear area model from gate (2), and rejects anything above $2.0\,\mathrm{W/mm}^2$. This is the realistic ceiling for an AIO-cooled package today (TPU-v3 0.36, TPU-v4 0.55, H100 0.7, MI300X 0.6, Cerebras WSE-3 1.5). This prevents "thin-but-hot" tiny-MAC chips that would melt without exotic cooling.

(6) SRAM macro feasibility. Requires the global memory to decompose into a banked array of standard $64\,\mathrm{KiB}$ SRAM macros (the workhorse macro at 7 nm), caps the bank count at $4{,}096$ (the practical NoC routing limit), and requires the bank-array aspect ratio to lie within $[1{:}4,\ 4{:}1]$ so the macro grid can actually be laid out. A $17\,\mathrm{MiB}$ CGM organized as a 1×272 strip fails this even though the byte-count is fine.

Outcome. Original-campaign designs pass all six declared screens before reporting. These bound reticle area, interface pins, logic depth, interconnect size, power density, and SRAM organization under the stated models. They do not prove routed timing, package feasibility, or achievable whole-chip power. The native extension's additional external-memory interface is a separately stated architectural assumption.

G.4 Why We Are Confident in the Software Side

The software claim is that the schedule the program commits to is the schedule that runs, and that its optimality is bounded rather than asserted. Five independent checks support it, ordered from strongest to weakest.

The committed schedule executes byte-exactly in RTL. The original DeepSeek-V3 validation schedule is compiled into a 1,383-instruction event tape and executed on a cycle-accurate SystemVerilog engine under Verilator (Appendix B). The run completes with **zero** capacity violations; the RTL peak on-chip occupancy equals the analytical prediction *exactly* at 7,602,176 bytes; and the total cycle count lands within $+0.000001\%$ of the rounded LP objective. This is the strongest form of the claim available without silicon: the plan is not merely feasible on paper, it is executable byte-for-byte on hardware that enforces the capacity constraint at every step of that tape. This is not an RTL execution of the later 61,985,244-invocation native workload.

The checker is live, not vacuous. A validation suite is only meaningful if it can fail. We therefore replay a deliberately over-committed tape, and the capacity checker flags exactly one violation. A test that never fails proves nothing; this one fails when it should.

Optimality is bounded, not claimed. An admitted integral execution supplies a feasible endpoint under its modeled contract. A lower bound applies only to the domain covered by its certificate; working-support bounds cannot be promoted to the full library. The current finite native family has a 21.6374% continuous interval (Appendix P). That interval is separate from the recovered chip's predicted latency and is not a performance error bar.

Rounding is feasibility-preserving and independently validated. In the original campaign, relaxed selectors are rounded by enumeration over the physically integral choices, and no pin that would violate a constraint is ever committed. Each rounded design is re-solved and checked against the constraint set independently of the solver that produced it, with residuals at the 10^{-13} level. We never report a relaxed objective as an achievable result.

Search configurations remain explicit. The original campaign uses the configuration in Appendix Q. The native extension separately records its reference, recovered design, and matched controls in Appendix P. Those controls share a chip and evaluator; they are not globally optimized feature-off designs.

What this does not establish. RTL cycle counts are not used as an absolute oracle: the analytical per-layer cache floors are conservative by $192\times$ to $2475\times$, so RTL evidence is scoped to schedule realizability, memory legality, and utilization *mechanisms*. We make no claim that the simulated datapath is a performance-optimal implementation.

G.5 Why We Are Confident in the Hardware Side

The hardware evidence supports the declared cost-model mechanisms; it is not a fabrication certificate for every modeled design. Design-space-exploration frameworks are conventionally validated three ways, against measured taped-out chips, against post-synthesis extraction, and against other frameworks [Mei et al., 2021]. We have all three.

Against post-synthesis extraction: the area coefficient and its functional form. We implement the scored datapath as synthesizable RTL and push it through OpenROAD to placement at the ASAP7 7 nm predictive PDK (Appendix B.1). Extracted cell area fits $A = 82.08\,\mathrm{MACs} - 206.5\ \mu\mathrm{m}^2$ with $R^2 = 0.999999$, and area per MAC varies by only 0.46% across a $16\times$ range in MAC count. The second number matters more than the first: the program assumes area is *linear with a size-independent slope*, and that is exactly what a placed datapath exhibits. The coefficient we use is bracketed from below by extracted cell area and from above by our own conservative floorplan.

Against shipped silicon: the energy coefficient. Our compute-energy coefficient corresponds to 0.40 pJ per INT8 operation. Whole chip figures for shipped accelerators are 0.35 (H100), 0.64

(A100) and 1.27 pJ/op (TPU-v4i), and an array-only coefficient must sit at or just below whole-chip values. It does. We additionally measured the precision-energy ratio directly rather than assuming it, and the measurement corrected our analytic estimate in both directions (Appendix B.2).

Against other frameworks: the mapping data. The original campaign's mapping data comes from Timeloop Pareto evaluations. The native extension uses explicit analytical dispatch, primitive-service, and memory-transfer contracts, as documented in Appendix P.

Declared feasibility is checked. The original campaign enforces its six proxy gates. The native extension checks its original LP rows, installed bank and staging resources, and complete execution accounting. These are different evidence scopes; neither replaces detailed physical implementation.

What this does not establish. We have not taped out, and we do not run clock-tree synthesis or routing, so we make no claim about achievable clock frequency in silicon. Absolute power from our own flow remains an upper bound rather than a prediction: an ungated tile driven by uncorrelated data on a predictive PDK reports roughly three times our coefficient, and we therefore anchor the coefficient to measured silicon rather than to our own synthesis. The gating measurement in particular is not yet trustworthy and no clock-gating decision is scored in the program as a result.

H Speedup Analysis: Baseline Deficit Attribution and Design-Parameter Sensitivity

A large end-to-end speedup number is only credible if two things are shown explicitly: (i) the naive baseline actually *carries* that deficit, and the deficit can be *localized* to a physical cause; and (ii) the reported gain can be *decomposed* into a small number of design decisions, each of which is separately sized. This appendix supplies both. We first attribute the naive-baseline per-operation runtime into compute-active versus memory-stall (versus communication) time (Appendix H.1); we then decompose the 34.6–$40.2\times$ co-design speedup into exactly two orthogonal factors (Appendix H.2); we sweep each design decision one at a time and report the objective's sensitivity (Appendix H.3); and we rank the decisions by objective sensitivity to guide where a chip designer should spend area and power (Appendix H.4). This section is a companion to the pre-RTL feasibility gates of Appendix G: the gates certify that a design is *buildable*; this section certifies where its speedup *comes from*.

Reading conventions (read before any number below). Throughout, **MFU is model-MFU**: the roofline-tightness ratio $\mathrm{MFU} = t_{\mathrm{compute}}/t_{\mathrm{solver}}$ of Appendix G.2 (cache compute floor over the LP/analytic latency within one cost-model convention), *never* a measured-silicon utilization; the searched datapaths are hypothetical, not fabricated. **Same-silicon** comparisons are equal-area and equal-TDP *by construction* (both sides share the PE grid, so the area/power model of Appendix G.3 returns identical budgets). The 97% pre-fusion memory-stall figures and the naive MFU values are *naive-baseline attributions*, not properties of the co-designed schedule. Every quantity is convention-tagged, and we never form a ratio across two conventions. The fixed-silicon algorithmic gain on EfficientNet-B7 is $2.21\times$ under this paper's naive convention (a $2.81\times$ value that appears in an earlier discussion note is a transcription error and is not used).

H.1 Per-Operation Deficit Attribution

To justify a custom-silicon speedup on the order of 34–$40\times$ we must first show that the naive baseline wastes that much time, and say *where*. We attribute the naive per-operation runtime into three buckets: *compute-active* (the systolic array is issuing useful MACs), *memory-stall* (the array is idle waiting on off-chip traffic), and, for multi-device / mixture-of-experts execution, *communication* (inter-device transfer of dispatched tokens or expert activations). On a single package the communication bucket is empty by construction: all eight blueprints of Appendix L hold their active working set on-package (for DeepSeek-V3, the active-expert set is resident and the routing layer, not the datapath, pages the full model), so no operation incurs cross-device traffic.

The deficit is memory stall, not communication. Two independent attributions, on two different silicon points, agree that the naive deficit is almost entirely memory stall.

(a) On the co-designed chip (blueprint convention). Before fusion and time-indexed placement, naive per-operation streaming (each operation reads all of its tensors from DRAM and writes them back) leaves the array stalled on memory for $\mathbf{97.1}$–$\mathbf{97.5}\%$ of naive time (Table 23; individual blueprints report 97.1% for Llama-1 13B/65B, 97.2% for DeepSeek-V3, 97.5% for Llama-3 70B). This is large precisely because the co-designed chip provisions 20–$27\times$ more MACs than TPU-v3: the compute floor shrinks, so any unfused, re-streaming schedule becomes memory-bound. Fusion plus time-indexed residency keep the reused tensors on-chip and recover the schedule to the compute floor, $\mathrm{MFU} = 1.000$ (0.998 on Llama-1 13B; Table 41).

(b) On fixed TPU-v3 silicon (grid convention). On the $65{,}536$-MAC baseline itself, naive streaming reaches only $\mathrm{MFU} = 0.40$–0.64 of the compute floor on TPU-v3 (Llama-3 70B 0.399, DeepSeek-V3 0.451, Llama-1 13B 0.467, SD1.5-UNet 0.643), and up to 0.78 across the compute-bound cells of the wider fixed-hardware grid; on the bandwidth-starved NPU (64 GB/s) it collapses to $\mathrm{MFU} = 0.02$–0.05. The joint schedule recovers every compute-bound cell to $\mathrm{MFU} \approx 1.000$, so the fixed-silicon speedup is a *pure utilization recovery*, not additional compute.

The stalled time is avoidable, not compulsory. The 97% stall is *re-streaming* of tensors that could have stayed on-chip, not irreducible DRAM traffic. Two floor analyses bound the compulsory component. First, on each blueprint the memory floor (compulsory bytes over the chip's own bandwidth) is only 0.10–0.33% of the compute floor (Llama-1 13B $2.27\times10^{7}/2.31\times10^{10} = 0.10\%$;

Table 23: Naive-baseline deficit attribution. Compute-active and memory-stall fractions are $\mathrm{MFU}_{\mathrm{naive}}$ and $1 - \mathrm{MFU}_{\mathrm{naive}}$ on fixed TPU-v3 silicon (grid convention, scale-$T_{\max}$ naive). Pre-fusion memory stall is the blueprint-chip convention (naive per-op streaming on the co-designed datapath). Communication is zero on a single package. The last column is the joint schedule's recovered model-MFU. All values are naive-baseline attributions under one cost-model convention (Appendix G.2); they are not measured-silicon utilizations.

	Naive baseline (TPU-v3, per-op)			**Pre-fusion stall**	**Joint**
Workload	Compute-active	Memory-stall	Comm.	(chip, no fusion)	MFU
Llama-1 13B	0.467	0.533	0	97.1%	1.000
Llama-1 65B	–[†]	–[†]	0	97.1%	1.000
Llama-3 70B	0.399	0.601	0	97.5%	1.000
DeepSeek-V3	0.451	0.549	0	97.2%	1.000
SD1.5-UNet	0.643	0.357	0	–	0.963

[†]Llama-1 65B is not one of the five workloads in the fixed-hardware MFU grid; its pre-fusion stall is read from Table 42. Across the compute-bound cells of the wider grid, naive MFU spans 0.40–0.78; the bandwidth-starved NPU cells sit at 0.02–0.05.

Llama-3 70B 0.31%; DeepSeek-V3 0.33%; Appendix L). Second, a mapping-independent attainable-traffic analysis in the style of Orojenesis [Huang et al., 2024] places the attainable per-Einsum DRAM floor at $\mathbf{0.1}$–$\mathbf{0.9}\%$ of the compute floor on every cell we measured (Llama-3 70B 0.116%, DeepSeek-V3 0.182%, SD1.5-UNet 0.896%), because at the deployed prefill-like batch the weight reuse is large. Both bound the same conclusion: the compulsory off-chip traffic that *no* schedule can remove is under 1% of the compute floor, so essentially the entire naive memory-stall deficit is recoverable by scheduling (fusion, time-indexed residency, and tile choice) and the recoverable ceiling is not gated by DRAM bandwidth or by network communication.

Planned figure: per-operation stacked time breakdown. We will add a per-operation stacked-bar figure (one bar per HLO operation, ordered by graph position; each bar split into compute-active / memory-stall / communication seconds) for the naive baseline and for the CHEETAH schedule side by side, on all four LLM blueprints, in the style of the FAST per-layer utilization plot [Zhang et al., 2022] and Eyeriss-style per-layer accounting [Chen et al., 2019]. The naive bars are dominated by the memory-stall segment ($\sim 97\%$ aggregate); the CHEETAH bars are almost pure compute-active (aggregate $\mathrm{MFU} \to 1$); the communication segment is empty on a single package and appears only in the multi-device / MoE variant of Appendix H.3. This figure is a *planned appendix experiment* (the underlying per-op compute/stall decomposition exists in the solver output; the figure is not yet rendered).

H.2 Two-Factor Speedup Decomposition

The headline speedup factors *exactly* into two orthogonal terms:

$$\underbrace{\frac{T_{\mathrm{naive}}(\text{TPU-v3})}{T_{\mathrm{joint}}(\text{chip})}}_{\text{headline}} = \underbrace{\frac{T_{\mathrm{naive}}(\text{TPU-v3})}{T_{\mathrm{joint}}(\text{TPU-v3})}}_{f_1\text{: scheduling recovery}} \times \underbrace{\frac{T_{\mathrm{joint}}(\text{TPU-v3})}{T_{\mathrm{joint}}(\text{chip})}}_{f_2\text{: MAC provisioning}}. \tag{15}$$

The first factor f_1 is a *fixed-silicon*, same-area / same-TDP algorithmic gain: it re-schedules the same $65{,}536$-MAC TPU-v3 and buys $\mathbf{1.48}$–$\mathbf{1.78}\times$. The second factor f_2 holds the joint schedule fixed and provisions the PE grid up to the co-designed chip's MAC count, buying $\mathbf{20.0}$–$\mathbf{26.9}\times$. Their product reproduces the per-workload Figure-2 headline (and the blueprint "achieved speedup vs. naive TPU-v3" rows) to within 1.4% (Table 24); the residual is the analytic chip-endpoint solve's utilization roll-off, cross-checked against the blueprints' Timeloop-in-loop objective.

Is the provisioning factor physically usable? A reader is entitled to ask whether f_2 is real parallelism or an artifact of a cost model in which compute time falls as $1/\mathrm{MACs}$ without bound. It is a fair question: an array can only speed an operation up as far as that operation supplies independent tiles to fill it, and a per-token decode GEMV supplies few. We measured the answer directly rather than argue it. For every event of DeepSeek-V3 we computed the *saturating* MAC

Table 24: Two-factor decomposition of the co-design speedup (Eq. (15)). f_1 is the fixed-silicon scheduling gain on TPU-v3 (same area/TDP), f_2 the MAC-provisioning gain to the co-designed chip. Reconstructed $= f_1 \times f_2$; headline is the blueprint value. Convention: config-0 naive; bf16 cycle basis; HiGHS-Optimal endpoints.

Workload	f_1 (sched., TPU-v3)	f_2 (MAC prov.)	Reconstructed	Headline	Recon./Headline
Llama-1 13B	1.662	20.31	33.75	34.22	0.986
Llama-1 65B	1.727	20.04	34.60	34.60	1.000[‡]
Llama-3 70B	1.484	26.85	39.84	40.16	0.992
DeepSeek-V3	1.784	19.97	35.63	35.68	0.999
Range	**1.48–1.78**	**20.0–26.9**	**34.6–40.2** (matches Fig. 2)		

[‡]Llama-1 65B's analytic chip-endpoint solve was degenerate under HiGHS time limits, so its f_2 uses the authoritative blueprint objective (exact by construction); the other three use the *independent* analytic chip solve and land within 1.4%.

count $M_{\text{sat}} = (\text{independent output tiles}) \times (\text{lanes per tile})$ under the same tile geometry the mapper uses, where independent tiles are row tiles times column tiles (k-tiles accumulate into one output and are not independent) times the number of GEMMs carrying work (expected active experts for routed FFNs). Beyond M_{sat} a chip has no more concurrent tiles to run for that event, and its compute time cannot keep falling. Table 25 gives the distribution in multiples of the TPU-v3 array.

Table 25: Saturating MAC count per event, DeepSeek-V3, as a multiple of TPU-v3's $65{,}536$ MACs. Prefill events saturate three to four orders of magnitude above the largest chip in this paper ($24.4\times$), so the provisioning factor f_2 is physically usable parallelism at prefill. Decode events saturate at 9–$14\times$ at the low end, which is where a bound on usable parallelism would bind, and only for chips beyond it.

Regime	Phase	min	p10	median
B=8, ctx 4096	prefill	3,584×	3,584×	16,384×
	decode	9×	14×	16×
B=32, ctx 4096	prefill	14,336×	14,336×	65,536×
	decode	14×	14×	32×
B=32, ctx 512	prefill	1,792×	1,792×	8,192×
	decode	14×	14×	32×

At the original prefill operating point, the table's tile geometry supplies many independent matrix tiles, so useful parallelism is less restrictive than it is for small decode matrices. The bound becomes relevant when installed arrays exceed the independent work available to an event. These counts describe the mapper's geometry, not placed-array timing, and do not account for growing non-matrix costs. The native full-inference model instead charges padded matrix service and explicit vector, local, and external service for each implementation (Section 6.5); its results must be read under that contract.

Use only the two-factor form; do not multiply the full decision ladder. It is tempting to read the software-ablation ladder (tile, placement, remat, datapath, datatype) as a chain of multiplicative factors and multiply them all. That *overshoots* the headline by 1.8–$2.1\times$ (a mechanical product of 70.7–$73.2\times$ versus the 34–$40\times$ headline), for two convention reasons, neither of them physics:

1. **The INT8 throughput label double-counts.** The blueprint headline T_{joint} is a *bf16 cycle count* at the chip's MAC count; INT8 (W8A8) is carried as a throughput / power label (roughly half the area and power per MAC), *not* as a separate cycle factor. Multiplying an extra $2\times$ for INT8 into a cycle product is a double-count. Dropping it lands the mechanical ladder at 35–$37\times$, within 3–12% of the headline.
2. **The software rows use a $\sim 1.3\times$ smaller naive baseline** (the scale-$T_{\max}$ naive) than the config-0 naive against which the headline is measured, so the two families of factors are not on the same numerator.

Equation (15) is the form that reconciles *exactly* because f_1 and f_2 share a single naive numerator and a single cycle basis. As a naming caution: the same fixed-silicon scheduling recovery is 1.48–$1.78\times$

against the config-0 naive (used above) and 2.14–2.51× against the grid's scale-$T_{\max}$ naive (the fixed-hardware grid convention of Appendix H.1); the two differ only by the naive-baseline definition, and must never be crossed within one ratio.

The provisioning factor is sub-linear (the anti-"just add MACs" result). f_2 is not free scaling. Measured as delivered throughput per provisioned MAC in an equal-area / equal-TDP resource-matched ablation, 20.0–27.0× more MACs realize only $\approx 16\times$ provisioning throughput, a linearity of $\mathbf{0.59}$–$\mathbf{0.81}$ (Llama-3 70B is worst, 27× MACs → 16.0×). Equivalently, the fixed-family MAC leg from 65,536 to 1,048,576 MACs (a 16× increase) yields only 14.0–15.7× cycle reduction, because a residual memory floor remains. Added silicon buys sub-linearly, which is exactly why capturing the free scheduling factor f_1 first (and provisioning MACs only up to the point of diminishing return) is the efficient order of operations.

H.3 One-Decision-at-a-Time Sensitivity Sweeps

We now hold the design fixed and sweep a single decision, reporting the objective (LP cycles) sensitivity. Two decisions have dedicated measured sweeps (global buffer, MAC count); the remaining fine-grained sweeps (per-unit peak-FLOPS, interconnect radix / link count) are framed here and marked as **planned** appendix experiments so they can be added to the paper's experiment list; we do not fabricate their numbers.

(a) Global-buffer size C_{GM}: measured, near-inert in the compute-bound regime. Table 26 sweeps C_{GM} from 1 to 64 MiB for DeepSeek-V3 on TPU-v3 (900 GB/s), for the joint (free-y) LP and for a fixed-tile FAST-subset LP. The joint LP is *flat*: its objective moves 0.39% across a 64× buffer sweep ($1.0723 \times 10^{12} \to 1.0682 \times 10^{12}$ cycles), and is bit-identical from 8 MiB upward. By contrast the fixed-tile schedule *needs* the buffer, improving 1.18× over the same sweep. The joint formulation substitutes tile choice for buffer capacity: when tiling is free, a larger global buffer buys almost nothing. The tile-dependent capacity extension is likewise inert here: it matches the retained joint model to within 0.077% across the full $C_{\mathrm{GM}} \in \{4, \ldots, 64\}$ MiB sweep. Under bandwidth pressure (64–128 GB/s) the buffer regains a modest decision ($\approx 1.19\times$ for Llama-1 13B over $8 \to 64$ MiB), but *only through placement*: with placement frozen (row A) the objective is constant across C_{GM}, so C_{GM} helps exclusively by enabling time-indexed residency, never on its own. The blueprints adopt $C_{\mathrm{GM}} = 64$ MiB.

Table 26: Global-buffer sweep, DeepSeek-V3 on TPU-v3 (900 GB/s, native cache convention, HiGHS-Optimal). The joint (free-y) LP is essentially C_{GM}-insensitive (0.39% over a 64× sweep); the fixed-tile FAST-subset LP is not (1.18×). Objective in LP cycles.

C_{GM} (MiB)	1	2	4	8	16	32	64	sweep (1→64)
Joint (free-y)	1.0723	1.0699	1.0690	1.0682	1.0682	1.0682	1.0682	$\times 10^{12}$; 1.004×
Fixed-tile (FAST-subset)	1.4627	1.4459	1.4327	1.4161	1.3863	1.3364	1.2371	$\times 10^{12}$; 1.182×

(b) Peak throughput / MAC-unit count: measured (count), planned (per-unit rate). The total-MAC provisioning sweep is measured: raising the PE grid from 65,536 to 1,048,576 MACs (a 16× increase) reduces the objective by 14.0–15.7× (Llama-1 13B 15.72×, DeepSeek-V3 14.98×, Llama-3 70B 14.04×, Llama-1 65B 14.02×), and the provisioning factor to the full co-designed chip is $f_2 = 20.0$–$26.9\times$ (Appendix H.2), both sub-linear (§H.2). This is the single largest decision on the objective. A *finer* sensitivity that varies per-unit peak throughput (systolic-array dimensions, vector-lane width, or clock) at a *fixed* MAC count and fixed area, separating "more units" from "faster units", is not yet run and is marked **planned**; the datapath template of Appendix G.3 and MAGNet-style generators [Venkatesan et al., 2019] already parameterize it.

(c) DRAM bandwidth: measured, inert in the compute-bound regime. Sweeping bandwidth from 900 to 3,350 GB/s at the fixed 65,536-MAC point moves the objective by $\leq 0.25\%$ (BW leg $= 1.000$–$1.0025\times$ for all four LLMs): at the deployed prefill-like batch these workloads are compute-bound, so bandwidth beyond the floor buys nothing. Bandwidth becomes the dominant decision only in the bandwidth-starved regime: at 64 GB/s (the compact NPU) the joint-vs-naive fixed-hardware gain is 5.9–31.4×, because there the roofline is memory-bound and residency / placement recover a

much larger share. Thus bandwidth sensitivity is regime-dependent: near-zero when compute-bound, first-order when memory-bound.

(d) Interconnect: on-chip NoC and inter-device topology (planned). Two interconnect decisions are framed but not yet swept. *On-chip*, the NoC / crossbar radix and link count set the PE-to-SRAM routing capacity; this is the resource bounded by the crossbar / NoC-routing gate of Appendix G.3 (gate 4, $N_{\mathrm{NoC}} = n_{\mathrm{PEs}} \times n_{\mathrm{banks}} \leq 5 \times 10^6$ cells), and its provisioning follows the network-on-package accounting of Simba [Shao et al., 2019]. In the single-chip compute-bound regime the on-chip NoC is provisioned to the compute floor and is not the binding resource, but a dedicated radix / link sweep (objective versus NoC cells at fixed MACs) is **planned**. *Across devices*, in the multi-GPU / mixture-of-experts regime the inter-device topology (radix and link count of the NVLink-class fabric) binds only when experts are dispatched *across* devices; when the active-expert set is package-resident (as in every blueprint here) the communication term is zero (Appendix H.1). A distributed / MoE topology sweep (objective and the communication bucket versus inter-GPU radix and link count, in the spirit of Chiplet-Cloud and wafer-scale serving analyses [Peng et al., 2024, Zhu et al., 2024] and co-optimized network provisioning [Rashidi et al., 2023]) is **planned**.

Table 27: Single-decision sensitivity summary (compute-bound LLM regime, TPU-v3 operating point). "Objective sensitivity" is the LP-cycle response to the stated sweep. Cells marked **planned** have no dedicated run yet and are listed as appendix experiments; their numbers are not fabricated.

Decision	Sweep	Objective sensitivity	Status
Global buffer C_{GM}	$1 \rightarrow 64$ MiB (BW$=900$)	$1.004\times$ (joint); $1.18\times$ (fixed-tile)	measured
	under BW pressure (≤ 128)	$\approx 1.19\times$, via placement only	measured
MAC count (total)	65,536 $\rightarrow$ 1,048,576 ($16\times$)	14.0–$15.7\times$ (sub-linear); $f_2 = 20.0$–$26.9\times$	measured
Peak rate / unit	systolic dims, clock @ fixed MACs	–	**planned**
DRAM bandwidth	$900 \rightarrow 3{,}350$ GB/s	$\leq 1.003\times$ (compute-bound)	measured
	64 GB/s (NPU)	5.9–$31.4\times$ (memory-bound)	measured
On-chip NoC radix / links	N_{NoC} at fixed MACs	– (gate 4 bounds it)	**planned**
Inter-device topology (MoE)	inter-GPU radix / links	– (binds on cross-device dispatch)	**planned**

H.4 Decision-Impact Ranking

Table 28 ranks the decisions by objective sensitivity in the compute-bound LLM regime and states *why* each sits where it does, so a chip designer can read off where area and power are best spent.

Table 28: Design decisions ranked by objective sensitivity in the compute-bound LLM regime (TPU-v3 operating point). "Cost" is what the decision spends on the die. The ranking inverts in the memory-bound / distributed regimes, as noted.

Rank	Decision	Sensitivity	Cost	Binds when
1	MAC-count provisioning	20.0–$26.9\times$ (sub-linear)	area + power	always (compute-bound); diminishing returns
2	Scheduling (tile+fusion+placement)	1.48–$1.78\times$	none (same silicon)	always; also a feasibility enabler at tight buffers
3	DRAM bandwidth	$\approx 1.0\times$; 5.9–$31.4\times$ if starved	pins + power	memory-bound regime only
4	Global buffer C_{GM}	$\approx 1.0\times$; $\leq 1.19\times$ pressed	area (SRAM)	buffer-pressed regime; only via placement
5	Interconnect (NoC / inter-GPU)	$\approx 1.0\times$ on-chip	area + links	distributed / MoE cross-device dispatch

Why this ordering, and what it tells a designer. In the compute-bound LLM regime the objective is set by how many useful MACs the schedule can keep busy. **(1) MAC-count provisioning** moves

the objective most (20–27×) but sub-linearly and at direct area/power cost, so it is the decision to spend on, up to the point of diminishing return, and never past the feasibility gates of Appendix G.3 (a bigger matrix unit can even degrade delivered performance, as observed for production TPU silicon [Jouppi et al., 2017]). **(2) Scheduling** is a *free* multiplier (1.48–$1.78\times$ at equal area and TDP) that must be captured first: it costs no silicon, it is what recovers naive MFU 0.40 to ≈ 1.0, and at tight buffers it is a feasibility enabler, not merely an optimization (on EfficientNet-B7 at TPU-v3's 32 MiB buffer the pure-streaming row is infeasible and time-indexed placement restores feasibility: placement alone at $2.03\times$, joint optimum $2.21\times$). **(3) DRAM bandwidth** and **(4) global buffer** are near-inert once compute-bound (bandwidth beyond the floor and buffer beyond a few MiB buy essentially nothing; Table 26), and the joint LP actively trades buffer for tile choice, so paying for HBM or a large global SRAM buys lower TDP-per-TFLOP, not more throughput, at these operating points. Both become first-order only when the workload is genuinely memory-bound (low bandwidth, or decode-time batch-1 operational intensity). **(5) Interconnect** is inert on a single compute-bound chip and matters only in the distributed / MoE regime, when experts are dispatched across devices and the communication bucket becomes nonzero. The practical guidance: *buy MACs (watching the sub-linear roll-off and the gates), capture scheduling for free, and invest in bandwidth, buffer, or interconnect only when the regime (memory- or communication-bound) actually makes them bind.*

Sweeps requiring new runs (for the paper's experiment list). The following are framed above but not yet run, and are proposed as appendix experiments: (i) a per-unit peak-throughput sweep (systolic dimensions / clock at fixed MAC count and area), isolating "faster units" from "more units"; (ii) an on-chip NoC radix / link-count sweep (objective versus N_{NoC} at fixed MACs, tied to gate 4); (iii) an inter-device / MoE topology sweep (objective and the communication bucket versus inter-GPU radix and link count, with experts dispatched across devices); and (iv) the per-operation stacked compute/stall/communication time-breakdown figure of Appendix H.1.

I Qualitative Design Discussion

Wall-clock time decomposition and search acceleration results are presented in Figure 4 and Section 6.

I.1 Interesting Empirical Design Results

Table 29: Surprise design pairs discovered by CHEETAH. Each pair achieves near-identical Perf/TDP via qualitatively different strategies that are not intuitive to human engineers.

Label	Workload	Trial	Perf/TDP ×	Speedup ×	MACs	CGM	L1 in/w/o (KB)	L1 cfg	L2 cfg	Batch
TPU-v3 ref	–	–	1.000	1.000	65,536	32 MB	64 / 64 / 64	Shared	Disabled	64
DeepSeek A	`deepseek_v3`	81	3.158	25.125	524,288	8 MB	512 / 1 / 256	Shared	Disabled	64
DeepSeek B	`deepseek_v3`	95	3.146	12.512	262,144	8 MB	1 / 256 / 512	Shared	Disabled	128
Llama-3 70B A	`llama3_70b`	88	3.557	14.152	262,144	32 MB	8 / 256 / 4	Private	Shared	64
Llama-3 70B B	`llama3_70b`	92	3.570	28.402	524,288	1 MB	512 / 128 / 512	Shared	Shared	64

We examine two designs on the DeepSeek-V3 (trials 81 vs. 95) and Llama-3 70B (trials 88 vs. 92) workloads. In each case, Perf/TDP scores are nearly identical yet design specifications are hugely different. DeekSeek's Design B uses 262K MACs vs. 524K in Design A. B compensates via richer search and larger batch. Pareto-tile space is 50× larger in B (500 vs. 10 configs); batch size doubles (128 vs. 64). Mirror-image L1 sub-buffer allocation in A keeps 512 KB input + 1 KB weight + 256 KB output. Design B keeps 1 KB input + 256 KB weight + 512 KB output. These two entirely opposite on-chip memory strategies yield identical system efficiency. On Llama-3 70B pair (trials 88 vs. 92), design A uses 262K MACs and 32 MB Global Buffer; Design B uses 524K MACs and only 1 MB Global Buffer. However Design A is weight-stationary, with a large 256 KB weight L1, tiny 8/4 KB input/output L1, and weight reuse amortised over a large on-chip scratchpad. Whereas Design B is activation-stationary, with 512/512 KB input/output L1, only 128 KB weight L1, and shared L1 configuration seen in doubled MACs allow CGM to collapse to 1 MB. This provides insight that additional compute in B substitutes for global memory capacity: higher throughput reduces the time activations must remain resident, relaxing the CGM capacity constraint.

I.2 Hardware and Vizier Search

The original campaign used 16 NVIDIA H100 GPUs, each with 80 GB of VRAM, and Vizier for hardware search following the experimental conventions of Zhang et al. [2022]. Table 30 records that campaign's parameter groups; Figure 9 depicts its accelerator template. The native full-inference extension instead uses the OCS, implicit-certificate and finite hardware-search paths in Section 5 and Appendix P.3; it is not an all-H100/Vizier experiment.

A Processing Element (PE) is a repeated compute tile in an accelerator template. The datapath can be modeled as a grid of PEs connected by a mesh on-chip network. PE systolic arrays perform a matrix-vector multiply each cycle. Vector and scalar MACs are modeled by setting systolic array X and Y dimensions to 1. PEs also contain a VPU for non-MAC vector ops. L2 and Global Memory structures are optional.

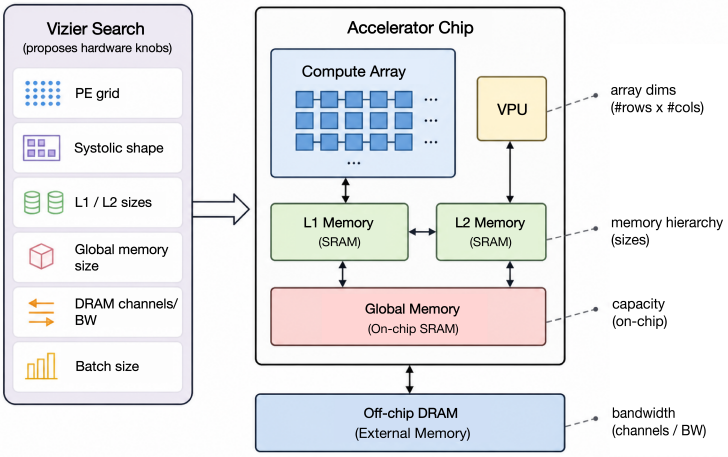


Figure 9: The accelerator datapath configuration. Processing elements (PEs) are arranged in a grid connected by a mesh network. Each PE contains a systolic array for matrix operations, a vector processing unit (VPU) for non-MAC operations, and private or shared L1/L2 memory. A Global Memory serves as the on-chip scratchpad shared across all PEs.

Table 30: Vizier search parameter groups and ranges: initial small search vs. expanded joint-tile-wide search.

Group	Parameter	Small (initial)	LLM-joint-tile-wide
A: Hardware datapath (16 params)	`PEs_x_dim`	pow2 in **4** … **32**	pow2 in **1** … **256**
	`PEs_y_dim`	pow2 in **4** … **32**	pow2 in **1** … **256**
	`Systolic_array_x`	pow2 in **4** … **32**	pow2 in **4** … **128**
	`Systolic_array_y`	pow2 in **4** … **32**	pow2 in **4** … **128**
	`Vector_unit_multiplier`	pow2 1 … 16	pow2 1 … 16
	`L1_buffer_config`	{Private, Shared}	{Private, Shared}
	`L1_input_buffer_size`	pow2 **1** KB … **4** MB	pow2 **1** KB … **1** MB
	`L1_weight_buffer_size`	pow2 **1** KB … **4** MB	pow2 **1** KB … **1** MB
	`L1_output_buffer_size`	pow2 **1** KB … **4** MB	pow2 **1** KB … **1** MB
	`L2_buffer_config`	{Disabled, Private, Shared}	{Disabled, Private, Shared}
	`L2_input_buffer_mult.`	pow2 1 … 128	pow2 1 … 128
	`L2_weight_buffer_mult.`	pow2 1 … 128	pow2 1 … 128
	`L2_output_buffer_mult.`	pow2 1 … 128	pow2 1 … 128
	`L3_global_buffer` (CGM)	{0}∪ pow2 **1** … **64** MB	{0}∪ pow2 **1** … **256** MB
	`GDDR6_channels`	pow2 **1** … **16** (→ 64–1024 GB/s)	pow2 **1** … **32** (→ 64–2048 GB/s)
	`Native_batch_size`	pow2 **1** … **1024**	pow2 **1** … **256**
B: Compiler passes (3 params)	`two_pass_softmax`	{disabled, enabled}	{disabled, enabled}
	`tensor_padding`	{disabled, enabled}	{disabled, enabled}
	`datatype`	{bfloat16, int8}	{bfloat16, int8}
C: fixed-tile rematerialisation (2 params)	`remat_enabled`	{disabled, enabled}	{disabled, enabled}
	`remat_cost_scale`	{0.1, 0.25, 0.5, 1.0, 2.0, 4.0}	{0.1, 0.25, 0.5, 1.0, 2.0, 4.0}
D: joint-tile joint-tiling (4 params)	`pareto_config_count_C`	{10, 50, 100, 250, 500, 1000}	{10, 50, 100, 250, 500, 1000}
	`loop_ordering_strategy`	{weight_stationary, output_stationary, both}	{weight_stationary, output_stationary, both}
	`pareto_pruning_strategy`	{tmax_workingset, tmax_tmin, tmin_workingset}	{tmax_workingset, tmax_tmin, tmin_workingset}
	`tile_workingset_modeling`	{fixed, config_dep}	{fixed, config_dep}
Post-Vizier gates (filter, not search)	`MIN_TOTAL_MACS`	32,768 (4×4×4×4=256 lower physical, 32K gate)	
	`MAX_TOTAL_MACS` cap	none: implicit ≤ 32×32×32×32 ≈ 1 M	1,048,576 explicit cap
	Implied total_MACs reach	≈ 256 … 1 M	256 … 1,048,576 (post-cap)
	Implied DRAM BW reach	64 … 1,024 GB/s	64 … 2,048 GB/s

J LLM-Architect Outer Loop Details

The LLM-Architect outer loop proceeds through the following steps for each campaign round:

1. **Extraction.** Deterministic scripts summarize recent trials into a *Causal Ledger*. Each record includes hardware parameters, validity, Perf/TDP, LP cycles, capacity duals/slacks, bandwidth sensitivity, MFU, memory utilization, roofline gap, failure class, and LP solver metrics.
2. **Context building.** The last N trials and the Causal Ledger are compressed into a compact JSON context for the Strategist. The context emphasizes empirical feasibility, observed invalid regions, best known valid designs, and physical audit signals.
3. **Architect.** The online Architect, implemented as Qwen2.5-7B-Instruct with a LoRA adapter trained on past trial logs, diagnoses the dominant bottleneck and emits a campaign patch. The patch constrains macro-architectural search decisions such as Global Memory, DRAM bandwidth, total MACs, systolic shape, PE grid, and batch size. A larger teacher model, DeepSeek-V3.2, was used offline to generate or critique campaign labels, but is not in the live loop.
4. **Campaign validation.** A deterministic validator clips or rejects the LLM patch to enforce legal hardware ranges, TDP and area limits, MAC caps, CGM feasibility floors, compute/bandwidth coupling, and roofline constraints. This prevents the LLM from proposing physically implausible regions such as maximum-MAC/minimum-bandwidth designs.
5. **Local optimization.** Vizier is initialized inside the validated patch and warm-started with prior trials that lie in the same region. Vizier then samples concrete hardware candidates within the approved campaign. The LLM chooses the high-level architectural neighborhood, while Vizier performs local adaptive optimization.
6. **Unified joint-tile Formulation.** The large-scale LP solver evaluates each candidate by solving the joint-tile joint tiling-fusion-placement LP using saved Pareto $\mathcal{C}$-sets. It returns the objective, cycles, selected tiling configurations, placement/rematerialization decisions, dual/slack summaries, and solver-quality diagnostics.
7. **Roofline audit.** A deterministic Roofline Auditor checks whether the predicted latency is physically plausible. For each candidate, we compute

$$t_{\text{compute}} = \frac{\text{useful FLOPs}}{\text{peak FLOPs/s}}, \qquad t_{\text{memory}} = \frac{\text{post-placement DRAM bytes}}{\text{peak bandwidth}},$$
$$t_{\text{ideal}} = \max(t_{\text{compute}},\, t_{\text{memory}})\,. \tag{16}$$

   A candidate is rejected if the solver-reported latency falls below this lower bound, or if MFU or memory utilization exceeds 1.0 within tolerance.
8. **Ledger update.** The loop records the campaign hypothesis, intervention, observed invalid-rate change, MFU change, roofline validity, and best Perf/TDP change. The ledger also records forbidden regions when a campaign produces excessive invalid trials.
9. **Shortlist Pareto refinement.** At the end of a campaign or study, only the top-ranked candidates are passed to a Pareto refinement run. This reruns targeted Timeloop refinement for selected configurations and re-solves joint-tile LP, providing a final trust check. This is an additional and optional sanity check step.

The Architect receives workload summaries, invalid-rate history, MFU, roofline signals, and prior valid regions, normalized LP duals and bandwidth sensitivities. The system starts from a safe empirically valid patch and allows duals only to make bounded edits, such as raising the CGM floor, increasing bandwidth, reducing MAC count, or changing systolic shapes. The validator rejects any dual-guided patch that excludes known high-performing valid regions or predicts a substantially worse feasible rate.

J.1 Profile-Conditioned LLM-Architect Application

We evaluate if the Architect can condition hardware search on deployment intent, since speech models may prioritize latency while data analytics models may prioritize bandwidth. For each workload,

we run the same Qwen2.5-LoRA Strategist, validator, random-in-patch sampler, and joint-tile LP evaluator while changing only a two-paragraph profile block describing the user's objective and constraints. The resulting best chips differ systematically by profile: overnight batch profiles select higher-bandwidth and larger-CGM regions, compact-edge profiles select smaller feasible regions with the lowest invalid rate, and interactive profiles favor small-CGM, extreme-aspect datapaths. Since the inner joint-tile LP and validator are identical across profiles, these differences isolate the effect of the LLM's profile-conditioned search patch. The absolute Perf/TDP often remains within a narrow band because the joint-tile evaluator reaches a similar structural ceiling, but the hardware families used to reach that ceiling differ substantially. We simulate two user profiles: (1) interactive speech is a spoken dialogue model where latency is more important than bandwidth, and (2) overnight finance batch where the user requires a large amount of financial data to be processed, but might leave it overnight to complete hence bandwidth is more important than latency.

The LLM-Architect clearly learned the profile distinction. Different profiles select meaningfully different chips for the same workload. The signal flows through the LLM's patch choice (steered by the profile prompt), and random-in-patch sampling finds the best chip within each profile-narrowed region. Table 31 gives examples of best designs per profile.

Table 31: Best hardware configuration discovered per (deployment profile, workload) cell. PE SA = PE grid $\times$ systolic array dimensions. MACs = total multiply-accumulate units. BW = DRAM bandwidth (GB/s). CGM = Global Buffer size (MB). Interactive speech = small low-latency-style chip, overnight finance batch = high-throughput, high-bandwidth, compact edge = compact constrained chip, balanced cloud = moderate/general design.

Workload	Profile	PE grid	Systolic	MACs	BW (GB/s)	CGM (MB)	Best Perf/TDP $\times$
Llama-1 13B	`interactive_speech`	1×128	16×16	32K	512	2	2.06
	`overnight_finance_batch`	16×8	128×32	524K	2048	256	2.73
	`compact_edge`	8×2	64×32	32K	512	4	2.06
	`balanced_cloud`	2×256	8×16	65K	64	128	2.97
DeepSeek-V3	`interactive_speech`	1×32	8×128	32K	2048	16	1.07
	`overnight_finance_batch`	8×256	16×8	262K	2048	1	2.55
	`compact_edge`	16×64	16×4	65K	128	2	2.99
	`balanced_cloud`	256×2	32×16	262K	1024	32	2.83

- `interactive_speech` repeatedly selects extreme-aspect PE arrays (1×32, 1×128, 1×256, 2×128); these favour narrow systolic streaming for low single-batch latency. CGM is consistently small (2–4 MB), trading cache capacity for compute density. Matches the profile intent: "minimize end-to-end latency, batch ≤ 8".

- `overnight_finance_batch` consistently selects high bandwidth (1024–2048 GB/s) and often large CGM (16–256 MB). Matches the profile intent: "maximize throughput, large batch, sustained bandwidth".

- `compact_edge` consistently selects the smallest chips (32–65K MACs is common vs. 524K for other profiles) with tiny CGM (1–4 MB). Matches the profile hard constraints: TDP $\leq 0.5\times$, area $\leq 0.5\times$, mobile deployment target.

- `balanced_cloud` selects medium-everything (middle-of-the-road MACs, bandwidth, and CGM), consistent with its unconstrained balanced objective.

Invalid-Rate Signal The absolute Perf/TDP $\times$ values are similar within a workload across profiles (e.g. Llama-3 70B: 3.38–3.53 across all four profiles, within $\approx 5\%$). This is not a failure: it reflects the LP's structural ceiling at fixed compute. *What differs is the chip family used to reach that ceiling.* This is precisely the design-space-narrowing benefit of profile conditioning: the same headline Perf/TDP number is achieved by hardware families with qualitatively different deployment characteristics (narrow streaming chips for interactive speech, large-bandwidth chips for overnight batch), demonstrating that profile conditioning the LLM-Architect steers the search toward different hardware trade-offs without sacrificing overall efficiency.

Table 32: Average invalid rate per deployment profile. The LLM responds to profile constraints: tighter profiles produce more LP-rejected proposals, while compact profiles are more feasible despite strict TDP/area limits.

Profile	Avg. invalid rate	Interpretation
`interactive_speech`	62%	Tightest profile (TDP $\leq 1\times$, batch ≤ 8): LLM proposes patches the LP frequently rejects.
`overnight_finance_batch`	53%	Loose TDP ($4\times$) but `min_batch` ≥ 32 forces large-batch hardware.
`compact_edge`	8%	Despite strict TDP/area constraints, the LLM finds feasible compact regions easily: best feasibility of all profiles.
`balanced_cloud`	36%	Middle-ground constraints produce intermediate invalid rate.

J.2 LLM in the Loop Prompts

The Strategist System Prompt

You are the Strategist agent in a hardware/software co-design loop.

Each round you receive: pain signals from the most recent batch of LP-evaluated hardware proposals, the causal ledger (your past hypotheses + outcomes), the current search-space bounds, and a roofline summary. You return **ONE JSON object** describing a campaign that narrows the search space for the next round.

You are a *campaign-level search-space editor*, not a performance predictor. Your job is to point Vizier at a small, physically-plausible neighborhood; the deterministic Validator + roofline auditor enforce hard constraints.

Important facts about the evaluator (CORGIsolve joint-tile). For each fixed hardware candidate, joint-tile already *jointly* optimizes:

- per-layer tiling selection $y_{i,c}$ over precomputed Pareto tiling sets $\mathcal{C}_i$
- time-indexed activation and weight placement
- rematerialization decisions
- tiling $\times$ placement interactions via McCormick variables
- fusion / DRAM-traffic reduction under the candidate's memory capacity

Therefore: do **not** assume low Perf/TDP is caused by a missing tiling search. The inner LP is already choosing the best available tiling/fusion/placement plan for the candidate. Your job is **only** to edit the *outer* hardware search region.

Pain signal keys.

- `invalid_rate`: fraction of round's trials not OK
- `mfu_med`, `memory_util_med`: median compute / memory utilization (≤ 1)
- `gap_to_floor_med`: median $t_{\text{solver}}/t_{\text{ideal}}$ (≥ 1; near-1 = at floor)
- `op_intensity_med`: median FLOPs/byte for OK trials
- `ridgepoint_med`: median chip ridgepoint (FLOPs/byte)
- `capacity_dual_p95`, `bandwidth_dual_p95`, `macfloor_dual_p95`: LP duals
- `best_perf_tdp_so_far`: running max Perf/TDP across the search
- `recent_roofline`: `{violation_rate, dominant_violation, ...}`

Causal reading.

1. `capacity_dual` high *or* failures are `global_memory_infeasible` $\Rightarrow$ increase CGM or restrict to configs with enough global memory. Do **not** blindly increase MACs.

2. `memory_util` high, `gap_to_floor` low, or `op_intensity_med` < `ridgepoint_med` ⇒ increase bandwidth, *reduce* MAC count, or avoid max-MAC/min-BW regions.
3. MFU low *and* `memory_util` also low ⇒ compute shape / systolic mismatch. Prefer different-shaped systolic arrays, **not** more total MACs.
4. Perf/TDP plateaued ⇒ propose an `efficiency_shrink` campaign that keeps $\geq 95\%$ of best Perf/TDP while reducing TDP / area / invalid rate.
5. Previous campaign had `invalid_rate` $> 50\%$ ⇒ do **not** repeat or further narrow into that region. Shift to the nearest empirically valid region.
6. *Empirical feasibility beats unverified dual reasoning.* Never choose a patch that excludes the best known valid configurations unless explicitly running a small `explore` campaign.

Duals are advisory, not absolute. If dual-based reasoning conflicts with empirical feasibility, prioritise empirical feasibility.

Anti-gaming policy (Validator enforces; obeying saves a wasted round):

- Do **not** propose the entire global search space; every patch must be a strict subset ($\leq 10\%$ volume by default; $\leq 20\%$ if `campaign_type` is `explore`).
- Do **not** pair max `total_macs` with min `GDDR6_channels`: ridgepoint cap 1000 FLOPs/byte.
- Do **not** propose tiny CGM (≤ 1 MB) for LLM workloads. Long-context workloads need ≥ 16 MB CGM (≥ 32 MB for 65B+).
- Do **not** re-propose regions marked forbidden in the ledger.

Patchable parameters (each must remain a non-empty subset of original powers-of-two): `PEs_x_dim`, `PEs_y_dim`, `Systolic_array_x`, `Systolic_array_y`, `Vector_unit_multiplier`, `L1_{input,weight,output}_buffer_size`, `L2_{input,weight,output}_buffer_multiplier`, `L3_global_buffer_size` (CGM in MB), `GDDR6_channels`, `Native_batch_size`.

Output: one JSON object on one line. No prose, no markdown fence.

`campaign_type` **semantics:**

- `exploit`: narrow region around best valid designs so far
- `repair`: targeted intervention for the dominant bottleneck
- `explore`: diverse region not covered by prior failed campaigns (validator allows up to 20% of global space)
- `efficiency_shrink`: Perf/TDP saturated; find a *smaller* chip that keeps $\geq 95\%$ of best Perf/TDP while reducing TDP / area / invalid rate

If no actionable hypothesis exists: return `campaign_name = "explore_current"`, `campaign_type = "explore"`, empty `search_patch`, `dominant_bottleneck = "invalidity"`, and reasonable prediction fields.

Teacher/Critic System Prompt

You are the **teacher/critic** for a hardware/software co-design search loop.

You are given:

- a *Trial Digest* (pain signals from a campaign round)
- a *deterministic oracle patch* (search-space narrowing that empirically worked in the next round of a previous run)
- the student-style template

Your task is to produce a **cleaned training example**. The patch and prediction **must** stay grounded in the deterministic oracle: do **not** invent new bounds. You may:

- rewrite `causal_hypothesis` to be precise and cite the dominant signal
- tighten `dominant_bottleneck` to the right category
- quantify `prediction` deltas (e.g. "drop below 0.25", "increase by 0.10")
- normalize `campaign_name` to `snake_case`
- reject the example by returning `{"keep": false, "reason": "..."}` if the deterministic oracle is internally inconsistent or unsafe

Return **one JSON object on one line**. Schema:

Accept:

```
{"keep": true, "campaign_name": "...", "causal_hypothesis":
"...", "dominant_bottleneck": "capacity|bandwidth|compute_shape|
tdp|invalidity|roofline_violation", "search_patch": <unchanged
from oracle>, "prediction": {...quantitative...}}
```

Reject:

```
{"keep": false, "reason": "..."}
```

Deployment Profile: `interactive_speech`

```
DEPLOYMENT PROFILE (active for this campaign)
  natural-language goal: Minimize end-to-end latency for streaming
                         speech inference. Throughput is secondary.
                         Power should stay moderate. Small batch sizes
                         only; single-request latency is the metric.
  objective_mode:        min_latency
  soft preferences:      prefer_batch_size=small,
                         prefer_latency=very_high,
                         prefer_throughput=medium,
                         prefer_bandwidth=medium,
                         prefer_area=medium
  hard constraints:      max_tdp_relative=1.0,
                         max_area_relative=1.0,
                         max_batch_size=8
  search hint (use as prior_centroid neighborhood):
    native_batch_size:       [1, 2, 4, 8]
    L3_global_buffer_size:   [16, 32, 64, 128]

Tailor ‘search_patch‘ to this profile. Do NOT always maximize
Perf/TDP; respect the profile's objective_mode. CORGIsolve V1
still optimizes inner-loop tiling/fusion/remat for each chip;
your job is to point it at the chip region whose trade-offs
match this user need.
```

Deployment Profile: `overnight_finance_batch`

```
DEPLOYMENT PROFILE (active for this campaign)
  natural-language goal: Maximize throughput for large overnight
                         historical-data analysis. Latency is not
                         important. Sustained throughput and memory
                         bandwidth matter most. TDP/power can be
                         high as long as the work completes by
                         deadline.
  objective_mode:        max_throughput_per_tdp
```

```
  soft preferences:      prefer_batch_size=large,
                         prefer_latency=low,
                         prefer_throughput=very_high,
                         prefer_bandwidth=very_high,
                         prefer_area=medium
  hard constraints:      max_tdp_relative=4.0,
                         min_batch_size=32
  search hint (use as prior_centroid neighborhood):
    native_batch_size:       [64, 128, 256]
    GDDR6_channels:          [16, 32]
    L3_global_buffer_size:   [128, 256]

Tailor ‘search_patch‘ to this profile. Do NOT always maximize
Perf/TDP; respect the profile’s objective_mode. CORGIsolve V1
still optimizes inner-loop tiling/fusion/remat for each chip;
your job is to point it at the chip region whose trade-offs
match this user need.
```

J.3 Resource Prices and Proposal Validation

Resource prices summarize the current identified program. Proposals can target expensive service, workspace, or retention constraints, but must be checked against their complete implementation contract. A proposal list is not an exhaustive pricing oracle: exhausting that list cannot establish full-master optimality. The native experiments use separately admitted schedules and complete-domain bound arguments as described in Appendix P.

K Two Optimal Tiling Examples: joint-tile LP Solution at a Glance

This section shows examples of optimal tiling, where accepting a locally slower/smaller tiling to enable better fusion or placement. This showcases the strength of the CHEETAH formulation, which discovers nontrivial tiling-placement-fusion trade-offs that less expressive methods would miss. The first example is Llama-3 70B on a modeled H100; the second is DeepSeek-V3 on a modeled RTX 5090.

By fixing the architecture, the inner loop derives optimal tiling and scheduling strategies that can act as custom speedups for targeted inference tasks. Therefore this section briefly provides motivation for future work on expanded hardware variants. This is motivated and similar in spirit to the speedups seen be FlashAttention [Dao et al., 2022] due to tiling optimization.

K.1 joint-tile LP Solution: Llama-3 70B on modeled H100

Problem dimensions: $L = 720$ layers, $C = 20$ tile configs, $T = 720$ timesteps, $E = 703$ activation edges. Total LP variables: 1,601,280. Objective: 542,790,817 cycles. `CONVERGED_STRICT` in 1,800 PDLP iterations.

Tile-Config Selection ($y_{i,c}$): The joint-tile LP heavily favours small/fast configurations. Table 33 groups tiles into three size bands, and Table 34 gives the per-config histogram.

Table 33: Tile-config band distribution across $L = 720$ layers.

Tile size band	Layers	% of model
Small ($c = 0 \ldots 4$): fast / large working set	427	59%
Medium ($c = 5 \ldots 14$)	275	38%
Large ($c = 15 \ldots 19$): slow / small working set	18	2%

Config histogram (picks per config index):

Table 34: Number of layers assigned to each tile config c. Configs with zero picks are omitted.

c	0	1	3	4	5	8	12	16	17
Layers	285	1	1	140	1	148	125	15	3

Config $c = 0$ (largest tile) was selected for $\approx 40\%$ of layers, with $c \in \{4, 8, 12\}$ covering most of the remainder. The LP decisively favours large tiles for this 70B model on H100: with 1.05M MACs and a 50 MB global buffer, reducing tile size to fit memory is unnecessary and incurs iteration overhead.

Per-Layer Execution Time T_i:

$$T_i^{\min} = 1{,}204 \text{ cycles}, \quad T_i^{\text{med}} = 524{,}288, \quad T_i^{\max} = 1{,}835{,}008, \quad \sum T_i = 542{,}790{,}817.$$

The ten longest layers (Table 35) all hit the same 1,835,008-cycle bound: these are the large FFN gate/up/down and Q/K/V/output projection sublayers in the transformer middle blocks:

Placement (p, p^W): Global Buffer Residency

- Activation edges pinned at $t = 0$: 0 / 703
- Weight layers pinned at $t = 0$: 0 / 720
- Total p-cells set across all t: 0 / 506,160 (0.00%)
- Total p^W-cells set across all t: 1 / 518,400 (0.00%)

The joint-tile LP makes almost no use of the global buffer for long-lived tensor residency and nearly everything streams through. With the L1/L2 buffer multipliers in this hardware configuration, **tiling alone is sufficient to keep the chip fed**; persistent pins are not needed.

Table 35: Ten longest layers. All hit the maximum $T_i = 1{,}835{,}008$ cycles and correspond to large FFN/attention projection sublayers.

Layer idx	T_i (cycles)	Selected config c
359	1,835,008	4
376	1,835,008	4
375	1,835,008	4
368	1,835,008	4
0	1,835,008	4
367	1,835,008	4
718	1,835,008	0
358	1,835,008	8
357	1,835,008	8
377	1,835,008	12

Rematerialization Events $r_{i,t}$

$$\text{Layers with} \geq 1 \text{ remat event} : 0 \;/\; 720. \qquad \text{Total remat events} : 0.$$

Zero Rematerialization. joint-tile has the freedom to recompute layers to free memory pressure, but on this cell determined it was not worth the additional cycles. Activation streaming combined with tile-config selection is sufficient.

Eviction / Restoration Timeline

- Activation evictions / restorations: $0 \;/\; 0$
- Weight evictions / restorations: $1 \;/\; 1$ (one weight churn across all 720 timesteps)

One Concrete Layer: Layer 360 (Deep Transformer Block)

$$\text{Selected config } c = 0, \qquad T_i = 524{,}288 \text{ cycles.}$$

$$\mathbf{y}_{\text{bin}} \text{ nonzero indices} : \{0,\ 16\}$$

The LP relaxation assigned nonzero weight to two equally-good configs

What This Reveals About joint-tile on Llama-3 70B / H100

1. **Tile choice does most of the work, not placement or remat.** Of 1.6M decision variables, only ≈720 (the $y_{i,c}$ selections) are doing real work; placement (p, p^W) and rematerialisation (r) variables are essentially all zero.
2. **Large tiles win because H100 is large.** With 1.05M MACs and 3.35 TB/s DRAM bandwidth, there is no memory slack pressure forcing small working sets. The LP exploits this by selecting the largest physically permissible tiles.
3. **The $1.53\times$ lift over the naive baseline comes entirely from per-layer tile selection.** Ten different configs are active across the 720 layers, versus the naive one-size-fits-all heuristic. On smaller chips (e.g. TPU-v3) this same mechanism yields a $2.11\times$ lift, because the memory slack disappears and per-layer tile choice becomes correspondingly more impactful.

K.2 joint-tile LP Solution: DeepSeek-V3 on modeled RTX 5090

This example shows the LP makes real per-sublayer scheduling decisions that are directly readable from layer names.

As a qualitative example of joint-tile's solution structure, we inspect the rounded joint-tile LP solution for DeepSeek-V3 on an RTX-5090-like hardware point. The model contains L=903 layers, including 174 MoE sublayers, and the joint-tile LP has 2.5M variables. Although the Pareto cache exposes 20 tile configurations per layer, the solution uses only five configs (Table 36), 0,4,8,12,16, suggesting

that joint-tile compresses the available tiling space into a small workload-specific alphabet. The most revealing behavior occurs in the FFN sublayers: ffn_up, ffn_gate, and ffn_down have identical compute time, yet joint-tile assigns them different tile configs in a rotating pattern (Table 37). This indicates that the LP is not merely selecting the fastest per-layer tile; it is interleaving large- and small-working-set choices across adjacent sublayers to manage memory pressure. joint-tile also automatically identifies structural outliers: the only seven layers assigned the smallest tile c=16 correspond to unusual sublayers such as rotary-key projections, low-rank Q projections, shared/expert gates, and expert down-projections (Table 38). Finally, weight churn is highly localized: only four attention-path weights in the first block are evicted/refetched (Table 39), while the MoE sublayers stream without churn. This case illustrates how joint-tile exposes interpretable schedule structure that is unavailable in a sequential Timeloop→fusion pipeline.

Problem Dimensions

- $L = 903$ layers (67-block DeepSeek-V3 with router/expert/shared sublayers per MoE block)
- $C = 20$ tile configs, $T = 903$ timesteps, $E = 886$ activation edges
- 2,504,019 LP variables, objective $= 517{,}356{,}013$ cycles
- Hardware (RTX 5090): 419K MACs, 1,792 GB/s DRAM, 98 MB CGM

Tile-Config Selection: Only 5 of 20 Configs Ever Picked Configs are spaced exactly every 4 indices. joint-tile does not sweep the full continuum but locks onto a discrete alphabet of five shapes (Table 36).

Table 36: Tile-config distribution for DeepSeek-V3 on RTX 5090. Only 5 of 20 candidate configs appear in the rounded solution.

Config c	**Layers**	**% of model**	
0	363	40.2%	(largest tile)
4	192	21.3%	
8	172	19.0%	
12	169	18.7%	
16	7	0.8%	(smallest tile: tightest working set)
others	0	unused	

Key Finding: joint-tile Staggers Tile Sizes Across the FFN Trio Every transformer block contains three FFN sublayers (up, gate, down) with identical compute time ($T_i = 2{,}580{,}482$ cycles each). joint-tile assigns *different* tile configs to the three so their working sets do not peak simultaneously in the global buffer. Table 37 shows the pattern across the ten hottest layers.

Table 37: Top-10 longest layers. Despite identical T_i, joint-tile assigns different tile configs to the FFN up/gate/down trio within each block, staggering working-set pressure across timesteps.

Layer idx	**Name**	T_i **(cycles)**	**Config** c
8	`layer0_ffn_up`	2,580,482	0
9	`layer0_ffn_gate`	2,580,482	12
10	`layer0_ffn_down`	2,580,482	8
19	`layer1_ffn_up`	2,580,482	4
20	`layer1_ffn_gate`	2,580,482	0
21	`layer1_ffn_down`	2,580,482	12
30	`layer2_ffn_up`	2,580,482	8
31	`layer2_ffn_gate`	2,580,482	4
32	`layer2_ffn_down`	2,580,482	0

This is the scheduling intelligence in joint-tile: same compute cost, different tiles, rotated so that any window of timesteps contains at most one large-working-set tile and two smaller ones. A naive

mapper assigns one tile size to all FFN layers; joint-tile detects the working-set conflict and breaks the symmetry automatically.

Structural Outliers: The 7 Layers That Pick Config $c = 16$ The LP assigned the tightest working-set tile ($c = 16$) to exactly the sublayers with unusual shapes (rotary embeddings, shared-expert gates, and expert down-projections) without any hand-labeled outlier feature (Table 38).

Table 38: The seven layers assigned config $c = 16$ (smallest tile / tightest working set). All correspond to structurally atypical sublayers in the DeepSeek-V3 MoE architecture.

Layer idx	Name	T_i (cycles)	Reason
202	`layer14_K_rope`	1,146,881	Rotary-embedded K: small inner dim
238	`layer16_shared_gate`	286,720	Shared-expert gate: narrow
322	`layer22_K_rope`	1,146,881	Same shape as layer 202
394	`layer27_Q_b`	737,281	Q lower-rank decomposition
573	`layer39_Q_a`	215,040	Q upper-rank decomposition
706	`layer47_expert_gate`	286,720	Per-expert gate
812	`layer54_expert_down`	286,720	Per-expert down-projection

Weight Evictions: Localized to Transformer Block 1 joint-tile evicts weights for only four layers, all in the first transformer block's attention path (Table 39).

Table 39: Weight eviction/restoration events. Only transformer block 1 attention weights are evicted; all 174 MoE blocks have zero churn.

Layer	Name	Evict	Restore	T_i (cycles)	Config c
12	`layer1_Q_b`	1	0	737,281	0
13	`layer1_KV_a`	1	1	71,680	12
14	`layer1_KV_b`	1	1	327,680	8
15	`layer1_K_rope`	1	1	1,146,881	4

The LP determined that block 0/1 attention weights are cheaper to refetch than to evict the FFN trio they compete with in the global buffer.

Relative Layer Costs Table 40 summarizes the execution time of key sublayer types.

Table 40: Relative execution time of key sublayer types. MoE routers are $\approx 72\times$ cheaper than FFN sublayers yet still receive varied tile assignments because 67 of them serialize in the dependency chain.

Sublayer type	T_i (cycles)	Ratio vs. FFN
FFN up / gate / down	2,580,482	$1.00\times$ (long pole)
Q/K/V projections	700K–1,200K	≈ 27–47%
MoE routers	35,840	1.4% ($72\times$ smaller)

What This Demonstrates About joint-tile

1. **joint-tile performs scheduling, not just per-layer tile picking.** The FFN trio tile rotation is a genuine scheduling decision: no naive per-layer mapper can produce this.
2. **joint-tile discovers structural outliers automatically.** The seven $c = 16$ picks land exactly on `K_rope`, `Q_a`, `Q_b`, `expert_gate`, and `expert_down` sublayers without any hand-labeled outlier feature in the input.
3. **joint-tile commits to a 5-config alphabet, not a 20-config sweep.** This is actionable for hardware designers: only 5 of 20 candidate tile shapes ever matter for this workload.

4. **Memory churn is highly localized.** Only 4 evictions occur, all in transformer block 1, all in the attention path. The 174 MoE blocks have zero churn: joint-tile streams them entirely.

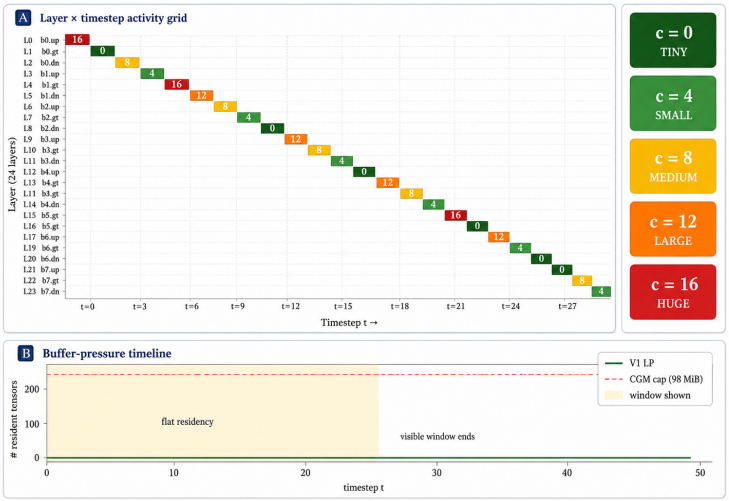


Figure 10: Visualization or rotating tiles selected by joint-tile LP. There exists 903 FFN sublayers in this workload, tile config C=20. The LP only picks 5 of them (since the rest are Pareto-dominated).

L Examples of CHEETAH Design Blueprints

We provide detailed blueprints for eight CHEETAH designs below. For each workload we give two variants: **Variant 1** (TSMC N7 + GDDR/LPDDR commodity memory) and **Variant 2** (TSMC N5 + HBM3 upgraded memory). Within a workload, the two variants share the same PE grid, so both inherit the same compute floor under the tile-dependent joint-tile LP, and achieve the same speedups. The variants differ only in area, TDP, and memory-subsystem cost. Every blueprint passes the roofline check and all six pre-RTL manufacturability gates of Appendix G.

Compute boundedness and the bandwidth choice. The binding floor is compute, not bandwidth, at the target workloads examined. At the chip's peak DRAM bandwidth (range 544 GB/s LPDDR5X (N7) to 2,457 GB/s HBM3 (N5)), we measure $T_{\text{DRAM}}/T_{\text{compute}} \leq 3.3 \times 10^{-3}$ on every blueprint. Total per-forward DRAM bytes are (13B / 65B / 70B / DS-V3) = (30.5 / 121.8 / 126.2 / 95.7) GB, and production inference serving amortizes these reads across many forward passes per weight load (typical batches-per-load ≥ 100 in vLLM and TGI). Therefore the steady-state DRAM traffic the LP charges per forward is far below the full weight tensor. The two TSMC N7 and TSMC N5 variants presented in Subsections L.1 and L.2 aims to make the latency–cost–yield trade-off explicit. TSMC N7's commodity memory subsystem still meets the bandwidth floor with $\geq 200\times$ headroom (the LP consumes $< 0.4\%$ of peak BW), so paying for HBM3 (TSMC N5) buys lower TDP per delivered TFLOP, not higher compute throughput.

L.1 Variant 1: TSMC N7 + GDDR/LPDDR

Table 41: Workload: `llama1_13b` · TSMC N7 · $V_{dd} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.46\times$ (= 364 W / 250 W)
Normalized Area	$0.61\times$ (= 427 mm^2 / 700 mm^2)
Peak Compute	2,703 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	1,344 GB/s (GDDR6X $\times$16)
Memory capacity	32 GB
Batch size	256
Num PEs	1320 (33×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	2,010 FLOP/byte
Fused model OpInt	2.05×10^6 FLOP/byte
Performance (QPS)	11.08 QPS @ batch=256
Inference latency	90.22 ms / sample
Normalized Perf/TDP	$23.51\times$
Achieved speedup vs. naive TPU-v3	$34.22\times$
MFU (Eq 4)	0.998
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 1,680 / 4096
reticle_area	PASS: 427 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,351,680 cells / 5,000,000
power_density	PASS: 0.63 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,351,680
DRAM bandwidth	1,344 GB/s
DRAM channels	16
PE array	33×40 (1,320 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	2.312×10^{10}
Compute floor cycles	2.308×10^{10}
Memory floor cycles	2.272×10^7 (at 1,344 GB/s)
TPU-v3 naive baseline	7.918×10^{11} cycles
Tile alphabet C	16
Layers L	360
LP solver	`custom JAX PDHG`
LP solve time	1.7 s

Table 42: Workload: `llama1_65b` · TSMC N7 · $V_{dd} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.20\times$ (= 300 W / 250 W)
Normalized Area	$0.51\times$ (= 357 mm^2 / 700 mm^2)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	544 GB/s (LPDDR5X $\times 8$)
Memory capacity	128 GB
Batch size	256
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	4,818 FLOP/byte
Fused model OpInt	4.02×10^6 FLOP/byte
Performance (QPS)	1.37 QPS @ batch=256
Inference latency	729.94 ms / sample
Normalized Perf/TDP	$28.83\times$
Achieved speedup vs. naive TPU-v3	$34.60\times$
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 680 / 4096
reticle_area	PASS: 357 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: 0.74 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	544 GB/s
DRAM channels	8
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	1.868×10^{11}
Compute floor cycles	1.867×10^{11}
Memory floor cycles	2.238×10^8 (at 544 GB/s)
TPU-v3 naive baseline	6.424×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	6.7 s

Table 43: Workload: `llama3_70b` · TSMC N7 · $V_{dd} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.57\times$ (= 392 W / 250 W)
Normalized Area	$0.61\times$ (= 425 mm^2 / 700 mm^2)
Peak Compute	3,539 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	544 GB/s (LPDDR5X $\times 8$)
Memory capacity	128 GB
Batch size	256
Num PEs	1728 (48×36)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.5%
Fusion efficiency	100.0%
OpInt ridgepoint	6,505 FLOP/byte
Fused model OpInt	2.07×10^6 FLOP/byte
Performance (QPS)	3.46 QPS @ batch=256
Inference latency	288.77 ms / sample
Normalized Perf/TDP	$25.61\times$
Achieved speedup vs. naive TPU-v3	$40.16\times$
Match to target	100.2%
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 680 / 4096
reticle_area	PASS: 425 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,769,472 cells / 5,000,000
power_density	PASS: 0.83 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,769,472
DRAM bandwidth	544 GB/s
DRAM channels	8
PE array	48×36 (1,728 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	7.385×10^{10}
Compute floor cycles	7.398×10^{10}
Memory floor cycles	2.320×10^8 (at 544 GB/s)
TPU-v3 naive baseline	2.965×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	

Table 44: Workload: `deepseek_v3` · TSMC N7 · $V_{dd} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.20\times$ (= 300 W / 250 W)
Normalized Area	$0.51\times$ (= 357 mm^2 / 700 mm^2)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	544 GB/s (LPDDR5X $\times 8$)
Memory capacity	128 GB (active-expert resident)
Batch size	128
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.2%
Fusion efficiency	100.0%
OpInt ridgepoint	4,818 FLOP/byte
Fused model OpInt	1.46×10^6 FLOP/byte
Performance (QPS)	2.40 QPS @ batch=128
Inference latency	417.32 ms / sample
Normalized Perf/TDP	$29.73\times$
Achieved speedup vs. naive TPU-v3	$35.68\times$
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 680 / 4096
reticle_area	PASS: 357 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: 0.74 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	544 GB/s
DRAM channels	8
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	5.341×10^{10}
Compute floor cycles	5.370×10^{10}
Memory floor cycles	1.758×10^8 (at 544 GB/s)
TPU-v3 naive baseline	1.906×10^{12} cycles
Tile alphabet C	16
Layers L	903
LP solver	`custom JAX PDHG`
LP solve time	

L.2 Variant 2: TSMC N5 + HBM3

Table 45: Workload: `llama1_13b` · TSMC N5 · $V_{dd} = 0.65\,\text{V}$ · 1.0 GHz.

Field	Value
Normalized TDP	$1.31\times$ (= 328 W / 250 W)
Normalized Area	$0.52\times$ (= 363 mm^2 / 700 mm^2)
Peak Compute	2,703 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	819 GB/s (HBM3 $\times 1$)
Memory capacity	24 GB
Batch size	256
Num PEs	1320 (33×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	3,300 FLOP/byte
Fused model OpInt	2.05×10^6 FLOP/byte
Performance (QPS)	11.08 QPS @ batch=256
Inference latency	90.22 ms / sample
Normalized Perf/TDP	$26.08\times$
Achieved speedup vs. naive TPU-v3	$34.22\times$
MFU (Eq 4)	0.998
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 1,024 / 4096
reticle_area	PASS: 363 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,351,680 cells / 5,000,000
power_density	PASS: 0.75 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,351,680
DRAM bandwidth	819 GB/s
DRAM channels	8
PE array	33×40 (1,320 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	2.312×10^{10}
Compute floor cycles	2.308×10^{10}
Memory floor cycles	3.726×10^7 (at 819 GB/s)
TPU-v3 naive baseline	7.918×10^{11} cycles
Tile alphabet C	16
Layers L	360
LP solver	`custom JAX PDHG`
LP solve time	1.7 s

Table 46: Workload: `llama1_65b` · TSMC N5 · $V_{dd} = 0.65\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.74\times$ ($= 434\,\text{W} / 250\,\text{W}$)
Normalized Area	$0.69\times$ ($= 485\,\text{mm}^2 / 700\,\text{mm}^2$)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	2,457 GB/s (HBM3 $\times 3$)
Memory capacity	72 GB
Batch size	256
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	1,067 FLOP/byte
Fused model OpInt	4.02×10^6 FLOP/byte
Performance (QPS)	1.37 QPS @ batch=256
Inference latency	729.94 ms / sample
Normalized Perf/TDP	$19.93\times$
Achieved speedup vs. naive TPU-v3	$34.60\times$
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 3,071 / 4096
reticle_area	PASS: $485\,\text{mm}^2 / 600\,\text{mm}^2$
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: $0.54\,\text{W/mm}^2$ / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	2,457 GB/s
DRAM channels	24
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	1.868×10^{11}
Compute floor cycles	1.867×10^{11}
Memory floor cycles	4.957×10^7 (at 2,457 GB/s)
TPU-v3 naive baseline	6.424×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	6.7 s

Table 47: Workload: `llama3_70b` · TSMC N5 · $V_{dd} = 0.65\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$2.10\times$ ($= 526\,\text{W} / 250\,\text{W}$)
Normalized Area	$0.79\times$ ($= 553\,\text{mm}^2 / 700\,\text{mm}^2$)
Peak Compute	3,539 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	2,457 GB/s (HBM3 $\times 3$)
Memory capacity	72 GB
Batch size	256
Num PEs	1728 (48×36)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.5%
Fusion efficiency	100.0%
OpInt ridgepoint	1,440 FLOP/byte
Fused model OpInt	2.07×10^6 FLOP/byte
Performance (QPS)	3.46 QPS @ batch=256
Inference latency	288.77 ms / sample
Normalized Perf/TDP	$19.09\times$
Achieved speedup vs. naive TPU-v3	$40.16\times$
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 3,071 / 4096
reticle_area	PASS: $553\,\text{mm}^2 / 600\,\text{mm}^2$
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,769,472 cells / 5,000,000
power_density	PASS: $0.64\,\text{W/mm}^2 / 2.0$
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,769,472
DRAM bandwidth	2,457 GB/s
DRAM channels	24
PE array	48×36 (1,728 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	7.385×10^{10}
Compute floor cycles	7.398×10^{10}
Memory floor cycles	5.137×10^7 (at 2,457 GB/s)
TPU-v3 naive baseline	2.965×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	

Table 48: Workload: `deepseek_v3` · TSMC N5 · $V_{dd} = 0.65\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.74\times$ ($= 434\,\text{W} / 250\,\text{W}$)
Normalized Area	$0.69\times$ ($= 485\,\text{mm}^2 / 700\,\text{mm}^2$)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	2,457 GB/s (HBM3 $\times 3$)
Memory capacity	72 GB (active-expert resident)
Batch size	128
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.2%
Fusion efficiency	100.0%
OpInt ridgepoint	1,067 FLOP/byte
Fused model OpInt	1.46×10^6 FLOP/byte
Performance (QPS)	2.40 QPS @ batch=128
Inference latency	417.32 ms / sample
Normalized Perf/TDP	$20.55\times$
Achieved speedup vs. naive TPU-v3	$35.68\times$
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 3,071 / 4096
reticle_area	PASS: $485\,\text{mm}^2 / 600\,\text{mm}^2$
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: $0.54\,\text{W/mm}^2 / 2.0$
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	2,457 GB/s
DRAM channels	24
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	5.341×10^{10}
Compute floor cycles	5.370×10^{10}
Memory floor cycles	3.894×10^7 (at 2,457 GB/s)
TPU-v3 naive baseline	1.906×10^{12} cycles
Tile alphabet C	16
Layers L	903
LP solver	`custom JAX PDHG`
LP solve time	

M Limitations, Future Work, and Broader Impact

The model optimizes a declared workload, finite implementation library, and analytical hardware contract. The frozen 21-branch native master is certified to a 0.0850% LP objective gap (rounded); a larger implementation universe, general pricing and global discrete optimality remain open. Continuous relaxation bounds, feasible schedules, numerical inference equivalence, and physical timing are distinct claims. The full-input DeepSeek result uses captured routes and expanded attention under its recorded precision policy; its workload is reproducible, not statistically representative of every deployment. The large speedup depends on the reference and custom memory-interface assumptions. Same-chip controls isolate software effects but are not globally reoptimized ablations.

The current execution credits compatible batching, KV retention/placement, and SwiGLU storage fusion. Larger native fusion groups, unrestricted overlap, quality-changing menus, and general rematerialization gains need additional experiments. RTL and synthesis studies cover selected schedules and datapaths, not complete-chip calibration. Extending the framework to multiple chips, robust workload distributions, or training introduces further communication and numerical constraints. More efficient inference designs may reduce energy per task; deployment scale and manufacturing costs determine their broader resource impact.

N Historical Shape-Tied Control and Its Limits

An earlier scalability experiment shared decisions among operations with identical Timeloop coefficient rows. This is a separate restricted control; it is not the representation used to claim coverage of the current native master. The latter preserves independent invocation choices and deduplicates coefficient recipes rather than silently identifying their variables.

Coefficient reuse versus shared decisions. If operations i and j have identical dimensions and implementations, the same coefficient recipe may generate both sets of rows. This reduces storage without imposing $y_i = y_j$. Replacing their selectors by one shape selector y_s, however, removes solutions in which they choose different tiles. Identical local coefficients do not imply interchangeable decisions: the operations can have different dependencies and live-memory pressure.

Temporal coupling. A product $w_{ict} = y_{ic}p_{it}$ depends on both implementation and temporal state. A shape-indexed selector and a coarsened residency trajectory cannot represent all per-instance products. The historical control grouped identical Pareto rows, multiplied compute, traffic, and recomputation coefficients by multiplicity, charged tile bytes once, and coarsened $T = N$ to $T = S$. Decision tying restricts the domain while removing capacity rows can relax it; there is no general ordering between its optimum and the original optimum. An objective-preserving reduction requires an explicit lift satisfying all original rows, not just agreement of shape dimensions.

Observed small-instance agreement. The retained comparison below solves both programs with HiGHS on the same truncated operation set and with the original linking rows enabled. The objectives agree to ten significant figures in these cases, with up to an $89\times$ reduction in columns. Capacity is slack in this experiment. These observations support the control at the tested points, not an equivalence claim for capacity-constrained or full native inference.

Table 49: Historical shape-tied control on the first N DeepSeek operations, 16 MiB global memory. Columns are LP variables; objectives use the original cycle convention. Agreement on these cases is not a general equivalence proof.

N	S	cols (unagg.)	cols (agg.)	unaggregated	aggregated	error
16	10	9,729	3,981	$3.848169063 \times 10^{10}$	$3.848169063 \times 10^{10}$	0.0000%
32	10	37,377	3,981	$1.027407872 \times 10^{11}$	$1.027407872 \times 10^{11}$	0.0000%
64	13	146,433	6,540	$1.397050573 \times 10^{11}$	$1.397050573 \times 10^{11}$	0.0000%
128	13	579,585	6,540	$2.088766669 \times 10^{11}$	$2.088766669 \times 10^{11}$	0.0000%

Current scalability path. Section 5 instead uses structured temporal operators and compact working supports, retaining the source identity and checking omitted-domain obligations separately.

Shared hardware is global, but independent operation choices remain independent. A working certificate applies to its identified support; only a complete-domain check can extend a lower bound to the source.

O Complete Rows of the Master Blocks

Equation (Master) is a perspective formulation of a declared finite family. If branch b has equality matrix E_b, inequality matrix G_b, right-hand sides e_b, g_b, and coordinate box $[\ell_b, u_b]$, its complete block form is

$$\begin{aligned} \min_{\lambda,x} \quad & \sum_{b\in\mathcal{B}_C} \kappa_b \sum_{e\in\mathcal{E}_b} d_{be} + \sum_{b\in\mathcal{B}_P} \alpha_b \left(\sum_i T_{bi} + \sum_{i,t} c^{\mathrm{remat}}_{bi} r_{bit} \right) \\ \text{s.t.} \quad & E_b x_b = e_b \lambda_b, \quad G_b x_b \le g_b \lambda_b, \\ & \ell_{b\nu} \lambda_b \le x_{b\nu} \le u_{b\nu} \lambda_b, \quad \lambda_b \ge 0, \quad \sum_b \lambda_b = 1. \end{aligned} \tag{17}$$

This is the explicit coefficient expansion of $\sum_b c_b^T x_b$: duration coordinates carry cost κ_b in causal branches; operation latency and rematerialization carry their original costs in retained schedule branches. All other coordinates in these blocks have zero objective cost. Branch weights are not multiplied into this expression a second time. A native-only instance has $\mathcal{B}_P = \varnothing$. The declared row matrices include the direct physical constraints below precisely when that versioned branch includes them. Only finite endpoints generate box rows. A branch at weight one retains its original constraints and objective; setting every other branch to zero gives an explicit inclusion map. This does not transfer a lower bound from one branch to a larger family. The retained schedule block below explains inclusion of the supplied joint LP when it is configured as a branch. The certified 21-branch native instance uses the causal blocks and their identified extensions; its bound is not asserted for a union with additional schedule models or a larger implementation library.

O.1 The Retained Joint Tiling–Placement Block

For original operation i, configuration c, edge e, and fixed time label t, let $P_{ik} = p_{\mathrm{assoc}(i,k),o(i)}$ for an associated activation, $P_{iW} = p^W_{i,o(i)}$ for weights, and $P_{ik} = 0$ when no placement association exists. The block's contribution to the master objective is $\alpha_b(\sum_i T_i + \sum_{i,t} c^{\mathrm{remat}}_i r_{it})$; α_b converts its cost units. Its explicit rows, in the original joint-LP notation, are

$$\begin{aligned} & \sum_c y_{ic} = \lambda_b, \qquad h_b = h_b^P \lambda_b, \\ & T_i \ge \sum_c T_i^{\min}(c) y_{ic}, \qquad T_i \ge \sum_c T_i^{\max}(c) y_{ic} - \sum_{c,k} t_i^k(c) w_{ick}, \\ & 0 \le w_{ick} \le y_{ic}, \quad w_{ick} \le P_{ik}, \quad w_{ick} \ge y_{ic} + P_{ik} - \lambda_b, \\ & \sum_e d_e p_{et} + \sum_i d_i^W p_{it}^W \le C_{GM,b} \lambda_b, \\ & p_{et} \le p_{e,t-1} + r_{\mathrm{src}(e),t} \quad (t > o(\mathrm{src}(e))), \\ & p_{it}^W \le p_{i,t-1}^W \quad (t > o(i)), \qquad r_{it} = 0 \quad (t \le o(i)), \\ & r_{it} \le p_{et}, \quad r_{it} \le p_{it}^W \quad (e \in \mathrm{in}(i),\ t > o(i)), \\ & T_i \ge 0, \qquad 0 \le y, w, p, p^W, r \le \lambda_b. \end{aligned}$$

These rows preserve the original fractional savings convention. Extra cold-start, copy, or exact-product constraints are not appended to this block. Causal execution uses its own complete contract instead.

O.2 Native Cover, Lifecycle and Storage

All following equations use scaled coordinates in one native branch. The event range is $0 \le e < E$, with storage/completion states at $0 \le e \le E$. A column covers its original nodes S_k, including any

conversion nodes; all cover, work-mass, active-state, typed $w = f$, control and operand-readiness rows are those in Eq. (Master). In particular $\sum_k a_{ke} = A_e \le \lambda_b$ is an aggregate active-mass restriction. Distinct invocations retain distinct coverage constraints even when coefficients are reused.

Let O_k be implementation outputs and τ_k their declared destination. For initial indicators $I_j, I_{j\tau}$, the birth and tier recurrences are

$$\begin{aligned}
&\beta_{j0} = I_j \lambda_b, \qquad p_{j\tau 0} = I_{j\tau} \lambda_b, \\
&\beta_{j,e+1} = \beta_{je} + \sum_{k: j \in O_k} f_{ke}, \\
&p_{j\tau,e+1} = p_{j\tau e} - v_{j\tau e} + \sum_{\sigma:\sigma\to\tau} q_{j,\sigma\to\tau,e} + \sum_{k: j\in O_k, \tau_k=\tau} f_{ke} + \mathbf{1}_{\{\tau=D,\ j\in\mathcal{J}_{\text{stage}}\}}\delta_{je}.
\end{aligned} \tag{18}$$

H, D, S denote host, DRAM and SRAM. The drain term occurs only in staging branches. Ordinary copy links are $H \leftrightarrow D \leftrightarrow S$. A copy retains its source and adds destination state at the next event; it is not a destructive move or instantaneous readiness. State, eviction, copy, choice and lifecycle masses have boxes $[0, \lambda_b]$.

For a nonterminal object's original users $\mathcal{U}_j$, backing requires

$$\begin{aligned}
&\beta_{je} \le p_{jHe} + p_{jDe} + p_{jSe} + D_{ie} && (i \in \mathcal{U}_j,\ 0 \le e \le E), \\
&\beta_{je} \le p_{jHe} + p_{jDe} + p_{jSe} && (j \text{ terminal},\ 0 \le e \le E), \\
&\beta_{jE} \le p_{jHE} + p_{jDE} && (j \text{ terminal}).
\end{aligned} \tag{19}$$

Terminal backing implies the corresponding user rows because $D_{ie} \ge 0$; their exact elimination changes no feasible point. KV appends use original object identities, complete prefix operands and these same persistence rules. The native certified family adds no recomputation columns. A separate recomputation extension must charge repeat work, gate it on prior production, restore tier state without a second birth/completion, and explicitly identify which backing obligations can be discharged by recomputation.

Selected numerical representation. A raw FP8 parameter and its converted BF16 parameter are different objects. For converted object j, raw owner $r(j)$, and consumer i, define $R_{ij} = \sum_{k\ni i:\, k \text{ reads } r(j)} y_k$. Selected-reader branches replace the corresponding nonterminal user row by

$$\beta_{je} \le \sum_\tau p_{j\tau e} + D_{ie} + R_{ij}, \qquad R_{ij} \le \sum_\tau p_{r(j),\tau e} + D_{ie}. \tag{20}$$

The actual selected implementation still requires readiness in its declared input tier. Original conversion nodes remain covered. Earlier branches retain their original backing rows; Eq. (20) is an identified alternative contract, not an equivalence transformation of them.

O.3 Capacities, Staging and Installed Hardware

Define the transient ordinary occupancy

$$o_{j\tau e} = p_{j\tau e} - v_{j\tau e} + \sum_{\sigma:\sigma\to\tau} q_{j,\sigma\to\tau,e}.$$

Let $N = h_{b,\text{arrays}}$, $S = h_{b,\text{SRAM}}$ in MiB, and $V = h_{b,\text{vector}}$. The source introduces $\chi_e \approx N A_e$ with the four linear inequalities

$$n_- A_e \le \chi_e \le n_+ A_e, \qquad N + n_+ A_e - n_+ \lambda_b \le \chi_e \le N + n_- A_e - n_- \lambda_b. \tag{21}$$

Let W_N, W_V be the original per-array and active-vector workspace constants, P_k private panel SRAM, D_k private DRAM workspace, and $O_{k\tau} = \sum_{j\in O_k:\tau_k=\tau} m_j$ output reservation. The newest native branch uses

$$\begin{aligned}
&\sum_j m_j o_{jSe} + W_N \chi_e + W_V A_e + \sum_k (P_k + O_{kS}) a_{ke} + (C_{\text{stage}} + C_{\text{receiver}})\lambda_b \le 2^{20} S, \\
&\sum_j m_j o_{jDe} + \sum_k (D_k + O_{kD}) a_{ke} + \sum_{j\in\mathcal{J}_{\text{stage}}} m_j \delta_{je} \le C_D \lambda_b, \\
&\sum_j m_j o_{jHe} + \sum_k O_{kH} a_{ke} \le C_H \lambda_b.
\end{aligned} \tag{22}$$

Branch-specific absent terms are zero. Entry and exit capacity rows count the resident state and installed dedicated reservations, without active work or in-flight transfers. Installed bank, register, controller and receiver requirements also give affine hardware rows, alongside area and power proxies. Host/local channel counts and receiver variants are whole-program choices; arrays, vector lanes, SRAM and DRAM bandwidth remain shared continuous coordinates inside each branch.

Explicit parameter staging. For each stageable owner j, the additional coordinates obey

$$\begin{aligned}
&\eta_{j0} = \eta_{jE} = 0, \quad \eta_{j,e+1} = \eta_{je} + \rho_{je} - \delta_{je},\\
&0 \le \delta_{je} \le \eta_{je} \le \beta_{je}, \qquad 0 \le \rho_{je} \le p_{jHe} - v_{jHe},\\
&\sum_j m_j \eta_{je} \le C_{\text{stage}} \lambda_b,\\
&\sum_j m_j(\eta_{je} - \delta_{je} + \rho_{je}) + \sum_{j,\ H\in\{\tau,\tau'\}} \min(m_j, C_{\text{stage}}) q_{j,\tau\to\tau',e} + \sum_k R_k^{\text{stage}} a_{ke} \le C_{\text{stage}} \lambda_b.
\end{aligned} \tag{23}$$

The state bound applies through exit; the transition row applies to action events. R_k^{stage} is the original implementation's reservation: ordinary host service uses $\min(B_k^H, C_{\text{stage}})$, while full-K and receiver implementations use their explicit panel/buffer reservation. The drain enters DRAM state and transient capacity in Eqs. (18)–(22). Dedicated staging and receiver SRAM cannot simultaneously hold ordinary data.

O.4 Native Resource Demands and Linear Service Envelopes

For original stream bytes B_k, MAC demand M_k, and vector work V_k, the native variable-rate demands are

$$\begin{aligned}
Q_e^B &= \sum_{j,\tau\to\tau'} m_j q_{j,\tau\to\tau',e} + \sum_k B_k w_{ke} + \sum_{j\in\mathcal{J}_{\text{stage}}} m_j \delta_{je},\\
Q_e^N &= \sum_k (M_k/256) w_{ke}, \qquad Q_e^V = \sum_k V_k w_{ke}.
\end{aligned} \tag{24}$$

Here one native array has 16×16 MACs. All traffic on the same DRAM service pool enters Q_e^B before applying its epigraph. Different resource services each satisfy $Z_e^r \le d_e$. For a rate $r \in [\ell, h]$, $u_- = 1/h$, $u_+ = 1/\ell$, the ideal homogenized reciprocal and product rows are

$$\begin{aligned}
&\ell\lambda_b \le r \le h\lambda_b, \quad u_-\lambda_b \le u \le u_+\lambda_b,\\
&u \ge 2\lambda_b/a - r/a^2 \quad (a \in \mathcal{A}), \qquad u \le ((\ell+h)\lambda_b - r)/(\ell h),\\
&0 \le Q \le \bar{Q}\lambda_b, \quad Z \ge 0, \quad Z \ge u_- Q,\\
&Z \ge \bar{Q}u + u_+ Q - \bar{Q}u_+\lambda_b, \quad Z \le u_+ Q, \quad Z \le \bar{Q}u + u_- Q - \bar{Q}u_-\lambda_b.
\end{aligned} \tag{25}$$

The implemented LP uses outward-rounded reciprocal endpoints and row coefficients derived from exact rational interpretations of supplied binary64 values. Each coefficient error is paid against its original coordinate box in the row RHS, rounded upward; errors on unbounded coordinates are refused. Thus stored rows contain the physical graph without asserting equality to it. The source demand bounds $\bar{Q}$ cover the full declared generator domain, not just activated working columns.

Fixed-rate ports use integer-scaled service rows. With B_k^H, B_k^L the original host/local byte coefficients, t_k^{setup} host setup ticks, B_k^{stage} the staging service bytes and t_k^{recv} receiver ticks,

$$\begin{aligned}
&\sum_{j,\ H\in\{\tau,\tau'\}} m_j q_{j,\tau\to\tau',e} + \sum_j m_j \rho_{je} + \sum_k (B_k^H + r_H t_k^{\text{setup}}) w_{ke} \le r_H d_e,\\
&\sum_{j,\ S\in\{\tau,\tau'\}} m_j q_{j,\tau\to\tau',e} + \sum_k B_k^L w_{ke} \le r_L d_e,\\
&\sum_j m_j(\rho_{je} + \delta_{je}) + 2 \sum_{j,\ H\in\{\tau,\tau'\}} m_j q_{j,\tau\to\tau',e} + \sum_k B_k^{\text{stage}} w_{ke} \le r_{\text{stage}} d_e,\\
&\sum_k t_k^{\text{recv}} w_{ke} \le d_e, \qquad \sum_k t_k^{\min} w_{ke} \le d_e.
\end{aligned} \tag{26}$$

Stage-owner sums are restricted to $\mathcal{J}_{\text{stage}}$. Ordinary kernels pay $B_k^{\text{stage}} = 2B_k^H$; streamed implementations substitute their paid staging recipe. Absent receiver/setup/floor terms are zero. The current source retains the floor row when its implementation contract requires it. The objective is $\kappa \sum_e d_e$, with κ the exact stored binary64 conversion from ns to ms. Integral replay adds its declared clock, descriptor and dispatch accounting; LP event sharing alone does not establish realizable kernel overlap.

O.5 Direct Roofline and Physical Constraints

The strengthened fixed-chip implementation uses one installed chip $\bar{h}_b$ per whole-program branch. Its scaled coordinates satisfy $h_b = \lambda_b \bar{h}_b$; no operation chooses a separate chip. Let $\bar{C}_b$ be installed BF16 matrix throughput in FLOP/ns, and $\bar{B}_{br}$ the delivered byte/ns rate of interface r. With original useful work $F_k^{\text{useful}} \le F_k^{\text{issued}}$ and linear traffic incidence B_{bre}, the source adds

$$\begin{aligned} &\sum_k F_k^{\text{issued}} w_{ke} \le \bar{C}_b d_e, && \mathsf{B}_{bre}(q,w,\rho,\delta) \le \bar{B}_{br} d_e, \\ &\sum_k V_k w_{ke} \le \bar{V}_b d_e, && b \in \mathcal{B}_{\text{phys}},\ e \in \mathcal{E}_b. \end{aligned} \tag{27}$$

For the suite's 16×16 arrays at 1 GHz, $\bar{C}_b = 512\bar{N}_b$ and $F_k^{\text{issued}} = 512\,\text{jobs}_k\,\text{bundle}_k\,\text{reductionCycles}_k$; one MAC is two FLOPs. The demand sums include every concurrently active implementation and transfer sharing a resource. HBM, host, local SRAM, DMA staging and aggregate bank ports have separate incidences and limits. The suite's external-copy staging row charges at least $64\lceil m/32 \rceil$ bytes for a transfer of m bytes, accounting for its 32-byte-word write and read. This is an alignment-independent necessary bound; replay pays actual alignment, commands and queue costs. Kernel local traffic, SRAM-copy endpoints and DMA staging enter one aggregate bank-port row.

For a selected branch ($\lambda_b = 1$), summing over the same event interval with total duration $T = \sum_e d_e > 0$ gives

$$\text{MFU} = \frac{\sum_{e,k} F_k^{\text{useful}} w_{ke}}{\bar{C}_b T} \le \frac{\sum_{e,k} F_k^{\text{issued}} w_{ke}}{\bar{C}_b T} \le 1, \qquad \text{MBU}_r = \frac{\sum_e \mathsf{B}_{bre}}{\bar{B}_{br} T} \le 1. \tag{28}$$

Traffic includes the represented reads, writes, repeated loads and transfers; vector arithmetic is not counted as BF16 matrix work. No tolerance declares a positive resource residual zero at exact admission. These inequalities remain linear because the installed rates are branch constants. In a variable-hardware relaxation, $C(h)d_e$ is bilinear; its McCormick envelope does not provide this exact fixed-chip guarantee. Fractional branch mixtures and intermediate solver iterates are not reported as fabricated or executable chips.

Write installed-component area and all-components-active peak power as $\mathsf{A}_b^\lambda = a_{b0}\lambda_b + a_b^T h_b$ and $\mathsf{P}_b^\lambda = p_{b0}\lambda_b + p_b^T h_b$. The shared constraint compiler adds the following exact affine rows:

$$\begin{aligned} &\mathsf{A}_b^\lambda \le 600\lambda_b, \quad \mathsf{P}_b^\lambda \le 300\lambda_b, \quad \mathsf{P}_b^\lambda \le 2\mathsf{A}_b^\lambda, \\ &\frac{h_{b,B}}{\eta_H \beta_H} \le n_H^{\max}\lambda_b, \quad \frac{h_{b,B}}{\eta_H \beta_H} + \frac{\bar{B}_{b,\text{host}}}{\eta_{\text{host}}\beta_{\text{host}}}\lambda_b \le n_{\text{pkg}}^{\max}\lambda_b, \\ &h_{b,B} \le \bar{B}_{b,\text{controller}}\lambda_b, \quad 2^{20} h_{b,S} = 65536\, n_b \lambda_b, \quad n_b h_{b,N} \le 5{,}000{,}000\lambda_b. \end{aligned} \tag{29}$$

Here area is in mm^2, power in W, β is byte/ns per signal pin, η is delivered efficiency, n_b is the fixed installed bank count, and $h_{b,B}, h_{b,S}, h_{b,N}$ denote bandwidth, SRAM MiB and arrays. The declared BF16 scenario uses at most 4,096 combined host/HBM signal pins, whole 64-KiB banks with $n_b \le 4{,}096$, bank-array aspect between $1/4$ and 4, and a logic-depth proxy at most 35. Each bank supplies a combined 32-byte/cycle port at 1 GHz. Installed-buffer lower bounds prevent hardware arrays, vector registers or DMA storage from borrowing memory already assigned to live data. Bank organization, pipeline depth and interface choices are fixed template constants; an invalid template cannot become an admitted positive-weight branch.

Peak power uses 1.2 mW per BF16 MAC in the revised analytical scenario, with separately priced SRAM, vector, controller and interface components. It is not multiplied by MFU. Inherited area coefficients and their sensitivity factors remain declared assumptions. The NoC-size and chip-average

density rows are proxies, not detailed network congestion or hotspot models; arbitrary bank/link overlap additionally needs its actual resource incidence. LP construction and recovery use the same identified coefficient set and gates.

Implementation and certificate scope. These rows are compiled by `native_physical_contract.py` and `native_suite_physical_rows.py`; the fixed-rate native wrapper also adds direct compute, bandwidth and vector ceilings. The complete saved Llama-13B execution passes 520,216,367 added scalar rows and four hardware boxes. Integration into every other master construction path is unfinished. The 21-branch 0.0850% certificate in Appendix P.4 retains its original rows and costs; it is not a certificate for the strengthened physical family.

O.6 Feature Scope

Table 50: Master encoding and the current native performance result.

Family	Encoding	Native result credited here
Hardware sizing	Shared resources, installed banks, area/power rows	One chip; explicit interface assumptions
Tiling and batching	Independent cover and work coefficients	Compatible shared-weight GEMMs and per-use tiles
Fusion/registers	Original ports, outputs and private workspace	SwiGLU and elementwise storage choices
Temporal placement	Tier states, paid copies, transient capacity	246 KV chains vs. 193 in a fixed partition
KV persistence	Append identities, prefix readiness, terminal backing	Full history and paid restoration
Parameter supply	Staging, panel buffers, raw/converted backing	Full-K panels and paid FP8-to-BF16 receiver
Rematerialization	Retained schedule or explicit repeat extension	No native repeat-column gain claimed
Precision/sparsity	Separately qualified implementation contracts	Fixed capture precision; no pruning/quality gain
Overlap/reordering	Event placement and resource epigraphs	Serial inter-kernel replay; internal panel pipelines

The generic formulation can express further clock, memory technology or reuse contracts, but they are not thereby members of the certified finite native pool. Adding a branch, implementation, or hardware range requires a new instance identity and new bounds. Quality-changing alternatives additionally need their own numerical contract.

P Native Full-Inference Provenance and Reproduction

P.1 Baseline and BF16 physical-cost reconciliation

The native ratios below retain their original analytical contracts. They do not yet establish comparisons against FAST's modeled TPU-v3 baseline, which starts from ordinary XLA fusion regions on two cores with two 128×128 MXUs per core, 16 MiB SRAM per core, 123 BF16 TFLOP/s and 900 GB/s per chip. FAST's additional fusion/scheduling is a separate optimized arm. Its internal simulator and compiler artifacts are not available here; the replacement is an explicit reconstruction with complete device residence, matrix mapping and nonmatrix service, rather than a claim to reproduce that internal simulator.

The archived native MAC peak-power slope is 0.4 mW, whereas the appendix specifies 1.2 mW for BF16. Replacing only that slope makes all twelve saved chips exceed 300 W. Area, banks, pins and NoC costs also require explicit shared assumptions. A new versioned analytical scenario applies the BF16 power slope, a shared 4,096-signal-pin budget for host and HBM traffic, and declared SRAM port/geometry assumptions. The inherited area slope is varied by factors of 1, 2 and 4; these are sensitivities, not calibrated BF16 bounds. New whole-chip branches use identical constraints for LP construction and execution admission. A passing scenario is not silicon sign-off.

For fixed chip h, each event satisfies total issued matrix work $F_e \le P_h t_e$ and total accounted interface traffic $D_e \le B_h t_e$. Summing over the same phase proves aggregate MFU and MBU at most one; padding, concurrent work and copies cannot be omitted. Variable-hardware McCormick points remain relaxation bounds, not physical latency predictions. The new contract does not relabel the frozen native LP certificate. Perf/TDP comparisons await matched process, workload and peak-power scope.

The construction and evidence are recorded in `systems/cheetah_joint/audits/20261026/BASELINE_FEASIBILITY_REPAIR.md` and `systems/cheetah_joint/audits/20261026/BASELINE_EXECUTION_CONTRACT.md`.

The corrected DeepSeek chip and its full-domain fixed-chip LP interval are reported in Appendix P.5. Its retimed workload remains separate from the historical comparisons below.

P.2 Archived native results

Frozen input. DeepSeek-V3 is pinned to checkpoint revision `e815299b0bcbac849fa540c768ef21845365c9eb`. Thirty-two nonoverlapping validation-text windows from Wikitext-2 produce 4,096-token prompts using the pinned tokenizer. Generation is greedy with 512 outputs per request. All 61 layers and 58 routed MoE layers execute; all route IDs and aligned weights are retained. The observation policy dequantizes original FP8 blocks to BF16, preserves higher-precision tensors, disables TF32 and reduced-precision BF16 accumulation, and uses the pinned normalization/router behavior. The workload is reproducible, not a language quality or representativeness study. Capture and solver GPU timings are not custom-chip performance estimates.

Archived chip and controls. The earlier result uses source identity `831edc6f8cde9bd2091541b55beaa00fae2736035badb3937236ba32c66104e3` and family `c2dbac2a699129be05060253c6d44292644ee7fa244f3cd9590bf5da6c113dd8`. Its 21 branches retain prior alternatives. The 320-array execution predicts $31.705749\times$ overall, $19.852844\times$ prefill and $37.931777\times$ decode speedup. Joint weight/KV recovery retains 281,872 weight gaps and 246 prefix chains. Its conservative execution reservation uses 68,697,653,440 of 68,719,476,736 DRAM bytes. The separately admitted continuous point was 1,146,545.738382 ms. Subsequent optimization of the same finite master gives a continuous upper of 734,720.389756 ms and the certificate in Appendix P.4. Direct original-row embedding of the physical-duration execution is not established by these separate points. The historical four-branch lower below was not transferred to this expanded family; its new lower is derived over all 21 branches.

The archived controls use this same chip, weight plan, KV policy and causal progress. Removing batching expands original groups into serial members inside each original dependency envelope; prefill members retain their actual routed row counts. Each streamed member pays a complete weight receive. Independent checks recount 31,706,528 decode and 1,441,733 prefill matrix members,

and verify every selected receiver template. Fusion controls restore original paid primitive spills, preserving local service and numerical boundaries. The added private scratch is 335,872 DRAM bytes and fits beyond the retained execution pool. Disabling full-K prefill panels reselects among the five ordinary tiles within the original workspace allowance. All other owners, dependencies and terminal obligations inherit the admitted execution. These controls are analytical substitutions, not new continuous-source admissions or feature-off optima.

The archived report is `systems/cheetah_joint/audits/20261023/NATIVE_JOINT_WEIGHT_KV.md`. Source, exact ledgers, independent checks and versioned failures are preserved under `native_joint_weight_kv_20261024_v1/`: `publish_v1`, `kv_ablation_replay_v1`, `storage_check_v1`, and `batching_check_v1`, and `family_tile_check_v1`. The tile control independently selects each of 45,091 parameter families' best constant decode tile while retaining membership, source kind, panel boundaries, buffers, leases and hardware. Its 0.865673-second penalty covers weighted decode; it is not a complete ablation of every time-indexed master choice. The current controls supersede the older chip's feature ratios; they do not erase the historical results below.

Earlier retained execution. The following four-branch record and 384-array controls are historical; the preceding archived result is described above.

The source identity is `6e216623a18c33290a6a072f5dab38f7d5bd1e130bee7f61b011b4caca44185d`, and the four-branch family is `261a86dcd42ffdbee930db921f1e7de6d879a17b7289f42a797e6cc74ea92654`. The exact predicted duration is $60733247481458753/20864$ ns. The complete-source admission checks all changed implementations and preserved boundary/state fields against original rows; a fresh replay independently confirms the execution. A separate cost checker recounts every group and member and reconstructs matrix service formulas. Seven semantic regressions cover the feature path. The result and twelve controls are retained in `systems/cheetah_joint/audits/20261023/native_feature_assets/`, including `chip.json`, `schedule.json`, `replay.json`, `result.json`, `features.csv`, and `cases.csv`.

Reference sensitivity. The headline uses a declared 32-GB/s external link for the native TPU-v3 extension and 1,024 GB/s for the custom design. The preceding matched interface-sensitivity study, on the pre-feature custom execution, reduces its $22.56\times$ comparison to 4.54–$4.80\times$ when the same naive reference is assigned 1,024 GB/s. This is evidence that interface assumptions materially affect the headline, not a new ratio for the final feature-enabled chip. The original study varies vector/local service as documented in `audits/20261023/NATIVE_REFERENCE_SENSITIVITY.md`. A well-scheduled, fully calibrated TPU serving stack is not established by the naive analytical reference.

Solver scope and current bound. Three accepted OCS working supports are separately identified:

Working support	Columns	Rows	Checked interval
Fixed serial policy	922,183	2,305,510	0.009473%
Hardware/service	36,350	310,354	0.065888%
Expanded residency	36,508	313,061	0.077406%

The first combines saved OCS iterates with exact reconstruction and a small CPU dual completion; the latter two use no CPU optimizer. The hardware/service and residency producers take 188.598 and 313.248 seconds respectively, including preparation and checking. These are not full-library solve times.

For the earlier four-branch native family, complete checking admitted a continuous lower of $1{,}059{,}841.723689$ ms against a continuous upper $1{,}352{,}483.524969$ ms, a 21.6374% interval. The new fusion alternatives invalidate direct transfer of older prices; the bound is rederived for all 1,964 fused coefficient classes. Request-specific decode prices then exploit actual expert imbalance while paying for mixed prefill/decode groups. Signed cover and work-mass equality multipliers remain legal; all 45,091 parameter families and their original request counts are independently checked. This covers the declared library and original event domain, not every intended future implementation. It is separate from both integral latency and the later 21-branch main-text family. The derivation and independent replay are in `audits/20261023/NATIVE_REQUEST_COVER_BOUND.md`.

The upper transfers the already admitted fused columns into the complete fractional-pass witness, preserving storage, event labels, and hardware; every changed shared row and box is rechecked. This improves the continuous upper by 11,857.091 ms without changing the integral result; see `audits/20261023/NATIVE_FUSED_WAVE_TRANSFER.md`.

Negative scheduling result. Two additional current-head batching policies preserve all decode invocations and pass independent dependency checks, but their predicted costs increase by 1,829.259 and 1,840.501 seconds relative to the retained execution. They are rejected. This motivates joint pricing of waiting, batch alignment, and parameter reuse; it does not prove that all alternative schedules are worse.

Additional array allocation. Keeping the admitted cover, kernel tiles, KV policy, fusion, and interfaces fixed, we vary the integer array count and pay for the additional SRAM. Dispatch time is nonincreasing in array count; exact budget enumeration therefore identifies the best feasible endpoint of this restricted change. The following candidates pass independent arithmetic and original-row composition checks for the entire native input:

Power reserve (W)	Arrays	SRAM (MiB)	Total latency (s)	Speedup
10	429	98.3125	2,908.727	$22.897\times$
5	473	106.5625	2,888.207	$23.060\times$
0	518	115.0625	2,874.771	$23.168\times$

At 518 arrays, prefill and decode improve to $13.607\times$ and $28.497\times$ against the same declared reference. Its modeled peak power is 299.932 W; this is a proxy budget, not physical power sign-off. The 384-array chip remains the common experimental point for the twelve controls on the earlier chip. These additional points improve latency by at most 1.24% without new scheduling or implementation choices; they do not establish global hardware optimality. The exact field substitution and checks are retained in `audits/20261023/NATIVE_ARRAY_ALLOCATION.md`.

Joint array and tile recovery. Reselecting among all five original tiles at each array endpoint improves the 518-array candidate to 2,871.252 seconds: $23.196\times$ overall, $13.607\times$ prefill, and $28.551\times$ decode. Fifty-six expert gate/up batch classes change tiles, covering 8,550,454 decode groups and four prefill groups. The additional saving is 3.519 seconds, almost entirely in decode; the 429- and 473-array candidates retain their tiles. Independent reconstruction and original-row admission pass with unchanged KV, fusion, copies and causal order. Every added group choice remains independently addressable. This extends the candidate family to five branches; the four-branch lower bound above is not transferred. The exact witness and replay are recorded in `audits/20261023/NATIVE_ARRAY_TILES.md`. This is a restricted joint recovery result, not full-master pricing or global hardware/software optimality.

Persistence. Source versions, failed preparations, exact checks, controls, and final results are content-verified on local external storage and the A100 host. Evidence archives use external content manifests. No original OCS project code is modified; experiments use the copied `systems/OCS_0.3.2` implementation.

P.3 End-to-End Solution and Recovery Workflow

The continuous objective certificate and the integral performance result solve different tasks. We specify the finite branch family and original coordinates, use working LPs and structured transformations to construct source points, and certify their objectives against a complete-family bound. Hardware search and schedule recovery produce a separate analytical execution. The following is the implemented workflow, not a proposed complete branch-and-price algorithm.

1. Freeze the mathematical instance. The native family has 21 branches, 61,985,244 independent invocations, 62,030,334 logical nodes including conversions, and 68,907,890 storage allocations. Its largest current branch has 15,044,386,833 implementation choices before temporal expansion; the event domain has 5,809,955,157 labels. These are logical domain counts, not an allocated sparse matrix. Invocation membership, types, demand caps, hardware ranges, costs and row recipes identify

the LP. A generator may reuse a coefficient formula without forcing its invocations to share decisions. The more general implementation resolver can describe choices outside this frozen pool; those choices are outside the present certificate.

2. Solve manageable working supports with OCS. Activated columns and temporal fields define a restricted working LP with a checked lift to original coordinates. Let $f(x) = c^T x + I_{[\ell,u]}(x)$, let Kx stack equality and inequality row activities, and let $g(z) = I_{\{z_{\mathrm{eq}}=b_{\mathrm{eq}},\ z_{\mathrm{ineq}}\le b_{\mathrm{ineq}}\}}(z)$. The selected fixed-step OCS kernel performs

$$\begin{aligned}
\widehat{v}^{k+1} &= \mathrm{prox}_{\Sigma g^*}(v^k + \Sigma K \bar{x}^k),\\
\widehat{x}^{k+1} &= \mathrm{prox}_{Tf}(x^k - TK^T \widehat{v}^{k+1}),\\
(x^{k+1}, v^{k+1}) &= \alpha_k(x^{\mathrm{anchor}}, v^{\mathrm{anchor}}) + (1-\alpha_k)(\widehat{x}^{k+1}, \widehat{v}^{k+1}), \qquad \alpha_k = \frac{1}{k+2},\\
\bar{x}^{k+1} &= 2x^{k+1} - x^k.
\end{aligned} \tag{30}$$

Each block restarts from its average. Centered arithmetic computes differences from the anchor to avoid repeatedly summing a large baseline. The primal proximal map is $\Pi_{[\ell,u]}(x - Tc)$; equality dual coordinates receive the RHS shift, while $\le$-row multipliers additionally project onto the nonnegative orthant. Reversible scaling and diagonal steps are verified against the original operator. The tested paged adapter moves bounded vector pages between host storage and GPU; it reduces GPU memory but still stores the working iteration vectors somewhere.

3. Improve a full-domain primal without full vectors. The final continuation starts from an admitted implicit source point. It retains identified parameter owners, changes legal original weighted implementations, retains the embedding after its first real import, and retires converted-host backups only after verifying their remaining consumers. These are searches for better points of the same source LP, not new constraints restricting all source solutions. The accepted recipe translates a compact parent of H events over T delays, using weights $\omega_j = 2(T-j)/(T(T+1))$. Each linear state and lifecycle field is represented by its weighted sum; duration is the maximum of the resulting resource-service fields. The retained parent can exceed DRAM capacity, so convexity of feasible parents is not the argument for feasibility. The checker proves the mixed capacity: the largest mass of any H consecutive delays is $H(2T-H+1)/(T(T+1))$ for $H \le T$, with initial/static reservations accounted for separately. Original readiness, boundary and service rows are also checked. GPU blocks evaluate the two edges of the mixture; rational affine sums handle the middle. An outward arithmetic allowance is included in the objective upper.

4. Certify the full LP objective. For $\min c^T x$ subject to $Ex = e$, $Gx \le g$, $x \in [\ell, u]$, any equality multipliers ν and legal inequality multipliers $\mu \ge 0$ give

$$L = -e^T\nu - g^T\mu + \inf_{\ell \le x \le u} (c + E^T\nu + G^T\mu)^T x.$$

Box support and every residual sign are checked in original coordinates; a small negative residual on an unbounded nonnegative axis cannot be rounded away. A working-set dual is insufficient when omitted columns change this infimum. The final certificate instead uses explicit coverage prices, work-mass equalities and original service rows. Generator-wide inequalities establish the required residual signs for all implementations; constant temporal multipliers apply to every event. The minimum branch bound certifies the perspective family. Appendix P.4 gives the actual multipliers and endpoints. This provides approximate continuous objective closure without completing a general arbitrary-multiplier pricing coordinator.

5. Search discrete hardware and recover software. The hardware branch-and-bound tree is separate from the whole-program branch index b in the LP. A tree node is a rectangle in a declared finite menu of array counts, vector lanes, DRAM rates, host channels and local-service units. Minimum-resource budget violations prune infeasible rectangles. Optimistic resource values, relaxed private-capacity constraints and repriced local implementations provide latency lower estimates. In a joint variant, each parameter family has a two-state retained/streamed dynamic program. Nonnegative prices on shared capacity rows yield the conditional Lagrangian bound

$$L(h,\mu) = \sum_f \min_{\pi \in \mathcal{P}_f} \left[t_f(h,\pi) + \sum_t \mu_t B_{ft}(\pi) \right] - \sum_t \mu_t C_t$$

plus costs outside those paths. Here f identifies an original parameter family, π an allowed retention path, and B_{ft} its storage use. GPU scans evaluate the independent paths; shared prices preserve the capacity coupling in the bound. A path mixture or optimistic rectangle is not itself an executable schedule.

The demonstrated menu contains 536,380,416 hardware combinations: 64–1,024 arrays in steps of two, 1,024–16,384 vector lanes in steps of 128, 512–2,560 GB/s DRAM in steps of 128 plus 2,608 GB/s, 1–64 host channels, and 1–8 local-service units. An earlier fixed-software tree exhausted that menu; the joint capacity-path tree made 1,190 GPU oracle calls and retained 142 unresolved schedule leaves, reduced to 78 by its subsequent replayed incumbent. Its bounds were numerical and conditional on the then-current cover, progress and implementation menu. They are not the final LP certificate, and subsequent menu expansions require fresh tree bounds.

Only a replayed feasible design supplies a pruning incumbent; an unverified candidate does not. Surviving hardware choices undergo integral cover/path selection, capacity repair and transfer scheduling. Replay then computes the predicted phase latencies using the selected chip's actual modeled rates, clocks, formats and buffer reservations. This is how an integral hardware/software result is obtained without claiming a completed tree over every master selector.

What an exhaustive discrete proof would additionally require. One would need complete lower bounds at every remaining hardware/schedule node, pricing over every allowed implementation under each branching decision, and a fully explored or pruned frontier. Making tile or branch choices integral alone does not make continuous rate/product envelopes exact. Spatial branching to enforce physical products addresses a stronger nonlinear design problem; it is not necessary to solve the frozen LP relaxation and is not claimed here. Analytical replay supports the reported design latency, while fabrication and global discrete optimality remain separate questions.

P.4 Archived certificate for the finite native master

The identified 21-branch native master retains all 61,985,244 independent invocations and 5,809,955,157 original event labels. Its current finite implementation generators remain fixed. We construct a feasible implicit continuous point and a complete-domain lower-bound certificate, obtaining

$$L = 734{,}095.7109040404\ \text{ms}, \qquad U = 734{,}720.3897564713\ \text{ms},$$
$$\frac{U-L}{U}\,100\% = 0.0850226646\%.$$

Thus the LP objective gap is certified to approximately 0.0850% (rounded), or strictly less than 0.085023%. This concerns the continuous finite master, not global optimality of the recovered chip and schedule. It retains the original physical-cost and service-envelope contract. The direct roofline and revised BF16 physical rows in Appendix O.5 define strengthened branches and require their own admission and bounds.

Explicit dual certificate. Use the notation of Eq. (Master). Let κ be the stored duration-cost coefficient, u_b^- the stored inverse-bandwidth lower-support coefficient in branch b, B_k the original stream demand of implementation k, and $\pi_i \ge 0$ a logical node's coverage price, including conversion nodes. We verify $\sum_{i\in S_k}\pi_i \le B_k$ for every implementation in every frozen generator. At branch weight one, under the convention $c^Tx + \sum_r \mu_r(A_rx - b_r)$, the nonzero row multipliers are

$$\begin{array}{ll}
Z_e^B - d_e \le 0: & \mu = \kappa,\\
u_b^- Q_e^B - Z_e^B \le 0: & \mu = \kappa,\\
Q_e^B - \sum_k B_k w_{ke} - \text{transfer bytes}_e = 0: & \mu = -\kappa u_b^-,\\
\sum_e w_{ke} - y_k = 0: & \mu = -\kappa u_b^- B_k,\\
\sum_{k:i\in S_k} y_k = 1: & \mu = -\kappa u_b^- \pi_i.
\end{array}$$

All other multipliers are zero. The duration, service, demand and work coefficients cancel. Choice residuals are $\kappa u_b^-(B_k - \sum_{i\in S_k}\pi_i) \ge 0$; transfer residuals are nonnegative. Original nonnegative coordinate bounds therefore give $L_b = \kappa u_b^- \sum_i \pi_i$. For the perspective branch union, the multiplier $-\min_b L_b$ on $\sum_b \lambda_b = 1$ leaves residual $L_b - \min_b L_b \ge 0$, proving $L = \min_b L_b$ for the full family. The multiplier identities apply to every original event, including events outside the primal's

compact parent support. Here $\kappa = 4722366482869645/472236648286964521369 6$ is the exact stored binary64 value of 10^{-6}; u_b^- likewise uses the stored outward coefficient. Ideal decimal constants are not substituted into the certificate.

Primal certificate and verification. The saved rational field recipe translates a parent of $H = 126{,}469{,}531$ events over $T = 5{,}683{,}485{,}627$ admissible delays with weights $2(T-j)/(T(T+1))$, $0 \le j < T$. The weights sum to one. Duration is the maximum of eight mixed resource fields. Original lifecycle and implementation admissions are retained as source-bound proof dependencies; the mixed DRAM capacity is checked using its maximum translation-window mass. A separate checker verifies the generator price inequalities, exact dual cancellation and rational resource moments. It checks the central affine envelope exactly and re-evaluates all 252,939,060 edge events in CPU extended precision with an outward error allowance. It shares the original source compiler and earlier lifecycle lemmas, rather than claiming a separately implemented frontend or proof-assistant formalization.

The certificate changes no source coefficients, implementation choices or event labels, and requires no new inference or optimization. It does not cover implementations absent from the frozen finite generators and does not establish convergence of a generic OCS primal–dual iteration. Exact fractions, source identities, multiplier tables, verifier source and replay receipts are retained in the accompanying native LP certificate artifacts.

P.5 Corrected fixed-chip full-domain LP

The revised BF16 physical contract admits the chip in Table 10. Its new source is `577942e1db9ffbe4f6582d27a69a418765e60deec584e04465f70b4f7a558604`. All 15,044,386,833 implementation choices, 61,985,244 invocations and 5,809,955,157 event labels remain independent. Hardware is fixed to this chip; this result does not certify optimization over all hardware branches. The complete software/temporal LP has checked endpoints

$$L = 1{,}180{,}606.049320422\ \text{ms}, \qquad U = 1{,}181{,}606.6831816933\ \text{ms},$$
$$100(U-L)/U = 0.0846841741\%.$$

The lower certificate reuses the checked original coverage prices π_i, whose inequalities still hold for every implementation's unchanged stream demand. It removes the old reciprocal-support multiplier and uses the new exact resource row $Q_e^B \le B_h d_e$, giving $L = \kappa \sum_i \pi_i / B_h$ with $B_h = 1630$. This transfers the row proof, not the old scalar bound.

The upper retains the admitted parent point's choices, lifecycle, storage and temporal translation mixture. It recomputes resource services for the new rates and regenerates all receiver alternatives. Nonnegative setup costs permit a conservative host-service factor $8/5$; twice the old receiver service bounds each new twenty-channel option. Duration is the complete resource maximum after mixing. The checker verifies 197,075 source service cases, 791,997 shared row checks and all 19,317 point service classes; the unchanged local rows follow their saved exact lift. A fresh source reconstruction reproduces both exact endpoints. Admission takes 94.626 s; the replay's admission stage takes 96.503 s, excluding source reconstruction. No new optimizer call is used for this certificate.

The continuous upper is distinct from the recovered schedule's conservative 3,196,885.598-ms latency. The latter preserves the admitted outer order and rechecks all 384 changed internal pipelines; the previous state/dependency replay supplies the unchanged lifetimes. This is a full-workload concept result under declared physical assumptions, not silicon validation or a discrete optimality proof.

Numerical reconstruction progress. The specialized GPU path executes candidate lifecycle updates and all shared candidate products over the same complete candidate/event domain. Four updates plus products take 56.21 s on an A100 and 23.14 s on an RTX 5090; their shared products agree. Source-bound checkpoints preserve independent owner values and permit continuation from disk. These runs keep external prices fixed. They do not establish convergence of the coupled master; readiness/storage coupling and a complete native residual pass remain integration obligations. The objective certificate above establishes its stated LP interval independently of that unfinished iteration engine.

Q Coefficient Audit Trail: Where Every Constant Comes From

The following inventory records the *historical analytical extension* coefficients and menu assumptions. It is not the numerical contract for the native full-input experiment, which is specified in Appendix P. R denotes a reported measurement or silicon anchor, A an assumption, and D a derived value; these labels describe provenance, not validation of every proposed chip or precision configuration. Timeloop-derived costs are modeled estimates. Quantization and prefix-cache menu entries below receive no native speedup credit. Source paths refer to the original released builders, whose historical `socp` filenames do not change the linear form of the exported program.

Constant	Value (units)	St.	Anchor
`RETICLE_AREA_MM2` (A_{cap})	600 (hard limit 858) mm^2	A	ASML/TSMC 26×33 mm reticle field is 858 mm^2 (the standard lithography limit); 600 leaves yield headroom, the same cap style as FAST [Zhang et al., 2022]
`MM2_PER_MAC` (k_{MAC})	1.5×10^{-4} mm^2/MAC (INT8, 7 nm)	R	**Post-synthesis extracted** (Appendix B.1): OpenROAD/ASAP7 7 nm placement of an INT8 systolic tile at 256/1024/4096 MACs gives 8.21×10^{-5} mm^2/MAC of cell area ($R^2 = 0.999999$ linear, 0.46% spread), i.e. this value at 54.7% placement utilization; also calibrated so TPU-v3 inputs (65,536 MACs) reproduce its core area, and consistent with Simba 8-bit PE area at 16 nm scaled to 7 nm [Shao et al., 2019]
`MM2_PER_MB_SRAM` (k_{SRAM})	1.5 mm^2/MB (7 nm)	R	TSMC N7 SRAM bitcell 0.027 μm^2/bit (ISSCC 2017 N7 test chip) plus macro overhead gives 1.4–1.6 mm^2/MB
`MM2_PER_DRAM_CHANNEL`	8.0 mm^2/channel	R	HBM PHY plus controller floorplan area on TPU/GPU die shots (TPU-v4i die analysis) [Jouppi et al., 2021]
`MIN_CHIP_AREA_MM2`	50.0 mm^2	A	IO ring, control, and power-grid floor observed on small accelerator dies (Simba chiplet package overheads) [Shao et al., 2019]
`W_PER_MAC_AT_FULL_UTIL`	0.4×10^{-3} W/MAC at 1 GHz (7 nm INT8)	R	Calibrated to TPU-v3 250 W TDP at 65,536 MACs × 700 MHz, then roughly 5× 7 nm INT8 efficiency (Horowitz energy scaling) [Jouppi et al., 2021]
`DEFAULT_UTILIZATION`	0.5 (fraction)	A	Measured LLM-serving MFU of 30–50% on production accelerators (vLLM and TPU serving reports) [Kwon et al., 2023]
$k_{\text{HBM,byte}}$ (HBM-resident capacity area)	$k_{\text{SRAM}}/10 = 1.43 \times 10^{-7}$ mm^2/byte	A	DRAM roughly 10× denser than SRAM per bit (standard memory-technology density gap)
k_{BW} (bandwidth area)	0.08 mm^2/(GB/s)	D	`MM2_PER_DRAM_CHANNEL` (Real) divided by roughly 100 GB/s per HBM3 channel (819 GB/s per stack over 8 channels, JEDEC JESD238)
k_{LD} (load-engine area)	$= k_{\text{BW}} = 0.08$ mm^2/(GB/s)	A	Load/DMA engine area assumed comparable to PHY area at matched bandwidth
π_{MAC} (power per MAC at operating util.)	2.0×10^{-4} W/MAC	D	Product of `W_PER_MAC_AT_FULL_UTIL` (TPU-v3 calibrated, Real) and `DEFAULT_UTILIZATION` [Jouppi et al., 2021]. **Silicon-anchored**: 0.4×10^{-3} W/MAC at 1 GHz is 0.40 pJ/MAC-op, bracketed by whole-chip shipped INT8 silicon (H100 0.35, A100 0.64, TPU-v4i 1.27 pJ/op), which an array-only coefficient must sit at or below (Appendix B.1)
P_{cap} (TDP cap)	700 W	A	NVIDIA H100 SXM5 TDP is 700 W (vendor datasheet)
π_{bit} (DRAM access energy)	4.0×10^{-12} J/bit	A	HBM2 access energy roughly 3.9 pJ/bit (O'Connor et al., Fine-Grained DRAM, MICRO 2017)

Table 51: Silicon cost model: area and power coefficients. Sources: `feasibility_checks.py` lines 32–48 and the LP builders (`deepseek_socp_codesign.py` lines 341–350, `deepseek_socp_serving_codesign.py` lines 147–152).

The full inventory, including per-coefficient file and line provenance, is maintained in `systems/deepseek_socp_formulation.md` in the released code; constants marked A above are

Constant	Value (units)	St.	Anchor
`COOLING_W_PER_MM2` (power-density gate)	2.0 W/mm^2	A	Envelope over real silicon: TPU-v3 0.36, TPU-v4 0.55, H100 0.7, MI300X 0.6, WSE-3 1.5 W/mm^2; 2.0 corresponds to aggressive AIO/liquid cooling
`MAX_LOGIC_DEPTH` (logic-depth gate)	35 gate levels at 1 GHz	A	FO4-per-cycle timing literature: roughly 25–35 logic levels per 1 ns cycle at 7 nm (Harris/Horowitz FO4 method)
`SRAM_BANK_KB` (sram-macro gate)	64 KB per macro	R	Standard foundry SRAM compiler macro size; Simba uses 64 KB-class buffers per PE cluster [Shao et al., 2019]
`MAX_BANKS` (sram-macro gate)	4096 banks	A	Practical routing limit for a banked global buffer; conservative floorplanning rule
`MAX_BANK_ASPECT` (sram-macro gate)	4.0 (aspect in [1:4, 4:1])	A	Standard physical-design floorplanning guideline for macro arrays
`HBM3_GBPS_PER_PIN` (pin gate)	0.8 GB/s per pin (6.4 Gbps / 8)	R	JEDEC HBM3 standard JESD238: 6.4 Gb/s per pin
`MAX_PIN_COUNT` (pin gate)	4096 pins (4 stacks $\times$ 1024 data pins, roughly 3.3 TB/s)	R	JEDEC HBM3 1024-bit interface per stack; 4-stack package standard (TPU-v3 uses 4 HBM stacks) [Jouppi et al., 2021]
Crossbar routing-cell cap (crossbar gate)	5×10^6 cells ($n_{\mathrm{pes}}\times n_{\mathrm{banks}}$; flat-crossbar bound 10^6, relaxed $5\times$ for a 2-level hierarchical NoC)	A	Industry hierarchical-NoC precedent: TPU-v4 mesh plus crossbar, H100 GPC/L2 partitions, MI300X chiplet XGMI, WSE-3 tile fabric [Jouppi et al., 2021]

Table 52: Pre-RTL manufacturability gate thresholds. Sources: `feasibility_checks.py` lines 57–64 and 174.

the explicit modeling assumptions a reviewer can vary, and every one enters the program linearly or through a one-hot menu, so sensitivity to any single assumption is a re-solve, not a re-derivation.

Constant	Value (units)	St.	Anchor
`MAC_WIDTH_BY_DTYPE`	int8 = 8, bf16/fp16 = 16, fp32 = 32 bits	R	Datatype bit-widths by definition (IEEE 754 / bfloat16)
Precision-menu byte fractions (b_W, b_A, b_{KV})	W8A8 = (1,1,1); W4A8 = (0.5,1,1); W4A8-KV4 = (0.5,1,0.5); W4A16 = (0.5,2,1)	A	Illustrative settings motivated by AWQ (W4), GPTQ (W4), and KVQuant (KV4); not quality-qualified here [Lin et al., 2024, Frantar et al., 2023, Hooper et al., 2024]
Array-aspect menu dims	sq16x16 = (16,16); wgmma8x32 = (8,32); tcgen05_32x8 = (32,8) rows × cols	R	NVIDIA Hopper WGMMA and Blackwell tcgen05 native MMA tile shapes (PTX ISA); ThunderKittens documents the H100 tile constraint [Spector et al., 2024]
Utilization $\eta(\text{spec}, a_r, a_c)$	$\eta = (\text{rows}/\lceil\cdot\rceil_r)(\text{cols}/\lceil\cdot\rceil_c)$, clipped to $[10^{-3}, 1]$	A	Analytic ceil-quantization model of array underfill; reproduces the Timeloop mapper low-GEMV-utilization artifact [Parashar et al., 2019]
DVFS menu γ (clock scale)	[0.7, 1.0, 1.3] for lo/nom/hi	A	Standard CMOS DVFS operating points (±30% around nominal); f proportional to V
DVFS menu p_f (dynamic-power factor)	[0.5, 1.0, 1.9] for lo/nom/hi	A	P proportional to fV^2 (roughly cubic in f), the CMOS dynamic-power law
Radix-cache menu sizes	[0, 64, 256, 1024] MB	A	Cache-size sweep grid; an on-chip-SRAM-scale prefix cache [Zheng et al., 2024]
Radix-cache hit-rate profile η_s	[0.0, 0.30, 0.50, 0.65] (saturating), fraction of prefill reuse	A	SGLang RadixAttention measured prefix-cache hit rates (saturating in cache size) [Zheng et al., 2024]

Table 53: Precision, datatype, and decision menus. Sources: `feasibility_checks.py` lines 65–73 and `deepseek_socp_codesign.py` lines 228–280.

Constant	Value (units)	St.	Anchor
Timeloop Pareto tile menu $T_{\min}(c)$, $T_{\max}(c)$, $t_{I/O/W}(c)$, $d_{I/O/W}(c)$	per-op per-tile arrays (cycles; bytes)	R	Measured Timeloop mapper Pareto frontier for DeepSeek-V3 layers [Parashar et al., 2019]
f_e routing frequency (balls-in-bins base)	$f_{\text{unif}} = 1 - (1 - 8/256)^B$ (0.983 at B=128; 0.638 at B=32)	D	Distinct-expert balls-in-bins derivation from the DeepSeek-V3 top-k=8 of n=256 routed experts [DeepSeek-AI, 2024]
Zipf routing-skew exponent	0.6 (mean-normalized tilt on f_{unif})	A	Mild expert-load imbalance consistent with the MoE load-balancing literature (DeepSeek-V3 aux-loss-free routing reports residual skew) [DeepSeek-AI, 2024]
E_{distinct} (expected distinct experts per decode step)	$256(1 - (1 - 8/256)^{32}) \approx 163$ at batch 32	D	Balls-in-bins expectation at the decode operating batch; f_e renormalized to this mean [DeepSeek-AI, 2024]
W_e (per-expert weight bytes, 8-bit)	$3 \times 7168 \times 2048 = 44.0$ MB (up, gate, down)	D	DeepSeek-V3 config: hidden 7168, MoE intermediate 2048 [DeepSeek-AI, 2024]
d_{KV} (MLA KV bytes per token, 8-bit)	$61 \times (512 + 64) = 35{,}136$ B/token	D	DeepSeek-V3 MLA: $n_{\text{layers}} = 61$ times (kv_lora_rank 512 plus qk_rope_head_dim 64) cached per token [DeepSeek-AI, 2024]
`DEEPSEEK_V3_CONFIG`	n_{layers}=61, hidden 7168, 256 routed experts, k=8 active, kv_lora_rank 512, qk_rope_head_dim 64, MoE intermediate 2048	R	DeepSeek-V3 technical report architecture table [DeepSeek-AI, 2024]

Table 54: Workload-derived quantities (DeepSeek-V3). Sources: `deepseek_socp_codesign.py` lines 180–183 and 283–363, `timeloop_full.py` lines 137–156.

Constant	Value (units)	St.	Anchor
M_{ref} (reference MAC count)	65,536 MACs	R	Declared TPU-v3 reference MAC count [Jouppi et al., 2021]
$\mathrm{BW}_{\mathrm{ref}}$ (reference bandwidth)	900 GB/s	A	TPU-v3 4× HBM2 headline bandwidth [Jouppi et al., 2021]
MAC design envelope $[M_{\min}, M_{\max}]$	[16,384, 4,194,304] MACs	A	Spans $1/4$ TPU-v3 to 64× TPU-v3 within reticle; a design-space envelope (FAST-style search bounds) [Zhang et al., 2022]
BW design envelope $[\mathrm{BW}_{\min}, \mathrm{BW}_{\max}]$	[100, 8000] GB/s	A	Low end GDDR-class single channel; high end HBM3E multi-stack (roughly 8 TB/s, B200-class)
`cgm_grid_mb` (CGM one-hot menu)	[1, 2, 4, 8, 16, 32, 64, 128, 256] MB	A	Power-of-two sweep bracketing TPU-v3's 32 MB on-chip global memory [Jouppi et al., 2021]
`BW_GRID` (bandwidth one-hot grid)	[225, 450, 675, 900, 1350, 1800, 2400, 3200, 4200, 5400, 6700, 8000] GB/s	A	Enumerated grid from quarter-TPU-v3 HBM2 to HBM3E multi-stack; one-hot branches are solved integrally (exact per branch), and the pin gate prunes entries above 3276.8 GB/s at the 4096-pin cap

Table 55: Solver grids and operating-point references. Sources: `deepseek_socp_codesign.py` lines 220 and 326–334, `deepseek_socp_serving_codesign.py` lines 72–129.